

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Кафедра «Програмна інженерія та
інтелектуальні технології управління»

ЗВІТ
з лабораторної роботи № 2
з навчальної дисципліни
«WEB-ДОДАТКИ НА ОСНОВІ JAVA»

ВИКОНАВ

Студентка групи КН-222а

Інесса РЕПЕШКО

ПЕРЕВІРИВ

Доцент кафедри ПІІТУ

кандидат технічних наук

Владислав КОВТУН

Харків 2024

ДОСЛІДЖЕННЯ ПРИНЦИПІВ ЗАСТОСУВАННЯ HIBERNATE ДЛЯ ДОСТУПУ ДО ДАНИХ У ВЕБЗАСТОСУНКАХ

Мета

1. Навчитись створювати вебзастосунок, що відображає на вебсторінці інформацію, яка міститься у таблиці бази даних, а також забезпечує виконання операцій CRUD із використанням Hibernate.
2. Набути навичок підключення БД до Java-проєкту та реалізації CRUD із застосування технології об'єктно-реляційного мапінгу із застосуванням класів та методів бібліотеки Hibernate.
3. Удосконалити навички створення вебсторінок на основні технології Java Server Pages (JSP) та застосування Java Standart Tag Library (JSTL) при їх розробці.
4. Удосконалити навички розробки зручних для користувача додатків.
5. Удосконалити навички створення сервлетів, враховуючи особливості використання даних.
6. Набути навичок реалізації контролю коректності даних, що вводяться до бази даних.

Завдання роботи

Розробити вебзастосунок, який отримує та відображає дані з однією таблицею реляційної бази даних, забезпечує пошук даних та виконання операцій CRUD із обраною таблицею, застосовуючи можливості об'єктно реляційного мапінгу.

Застосунок має забезпечити:

- відображення вмісту таблиці на вебсторінці;
- додавання записів до таблиці;
- редагування даних обраного запису;
- вилучати записи з таблиці;
- здійснювати пошук даних за семантичними полями.

Реалізацію завдань можна поділити на декілька завдань:

1) створення проєкту для роботи із базою даних із використанням бібліотеки реляційного відображення об'єктів Hibernate, створення відображення «застосунок - база даних»;

2) створення Entity-класу, створення відображення «клас – таблиця бази даних», створення таблиці у базі даних; створення та тестування класу із методами виконання операцій CRUD (create, read, update, delete);

3) створення JSP для виведення всього переліку записів із таблиці та реалізація функціоналу управління відображення даних (сортування, фільтрація, пошук);

4) створення форми для виконання додавання та редагування записів до таблиці, а також сервлетів для здійснення додавання та редагування.

Умови варіанту № 1 індивідуального завдання для реалізації у даній лабораторній роботі представлені на рис. 1.

Варіант №1 (ЗВО №1)			
Опис структури полів класу «Abiturient» (суттєвість 1)			
Ім'я поля	Тип даних	Опис	Властивості поля
id	Long	Сурогатний ключ	PK
p_code	String	Номер особової справи	Маска номеру "KH23-NNNN", унікальне для кожного студента
surname	String	Прізвище	Not null
name	String	Ім'я	Not null
pname	String	По-батькові	Not null
birthday	LocalDate	Дата народження	Not null
gender	boolean	Стать	Not null; true – чоловіча, false - жіноча
rating	double	Рейтинговий бал	Not null від 120.01 до 200.00
Опис структури полів класу «StudentDopInfo» (суттєвість 2)			
Ім'я поля	Тип даних	Опис	Властивості поля
id	Long	Сурогатний ключ	PK
student	Abiturient	Студент-власник особистої інформації	@OneToOne (FK до Student)
contract	boolean (можна String)	Контракт чи бюджет?	Not null. Перелік значень: «контракт»; «бюджет»
scholarship	boolean (можна String)	Є стипендія чи немає?	Not null. Перелік значень: «стипендія»; «ні». Якщо студент на контракті, то встановлюється значення «ні».
email	String	Адреса у корпоративній пошті НТУ «ХП»	Not null. Unique. Складається з транскрипції імені та прізвища студента, домен - @cs.khpi.edu.ua

Рисунок 1 – Умова варіанту № 1 індивідуального завдання

Хід роботи

1 Створення проєкту з Hibernate і відображення «застосунок - база даних»

Налаштування проєкту. У результаті створення Maven проєкту було додано необхідні теги залежностей для підключення до проєкту бібліотек драйверу JDBC для обраної СКБД MySQL, Servlets API, JSP, JSTL, що представлені у додатку А.1 у файлі конфігурації Maven проєкту pom.xml. Даний файл визначає наступний стек технологій розробки:

- JDK 17;
- Jakarta EE 9.1;
- Apache Tomcat 10.1.28;
- Servlets API 6.1.0;
- JSP (JavaServer Pages) та JSTL (JSP Standard Tag Library) 3.0.1;
- Lombok 1.18.34;
- Junit 5.10.3;
- DBMS MySQL 8.0.30;
- Hibernate 6.5.2.Final;
- Java Persistence API 3.2.0;
- Log4j 2.23.1 та SLF4J;
- Apache Commons Lang 3.12.0;
- Jakarta EL (Expression Language) 6.0.1;
- JBoss Logging;
- Hibernate Validator 8.0.1.Final.

Архітектура проєкту. У проєкті реалізовано багатошарову архітектуру (Multi-layered Architecture) що представлена на рис. 2 далі в тексті роботи та організована на основі шарів відповідальності із та розподілом внутрішніх пакетів за бізнес-логікою застосунку.

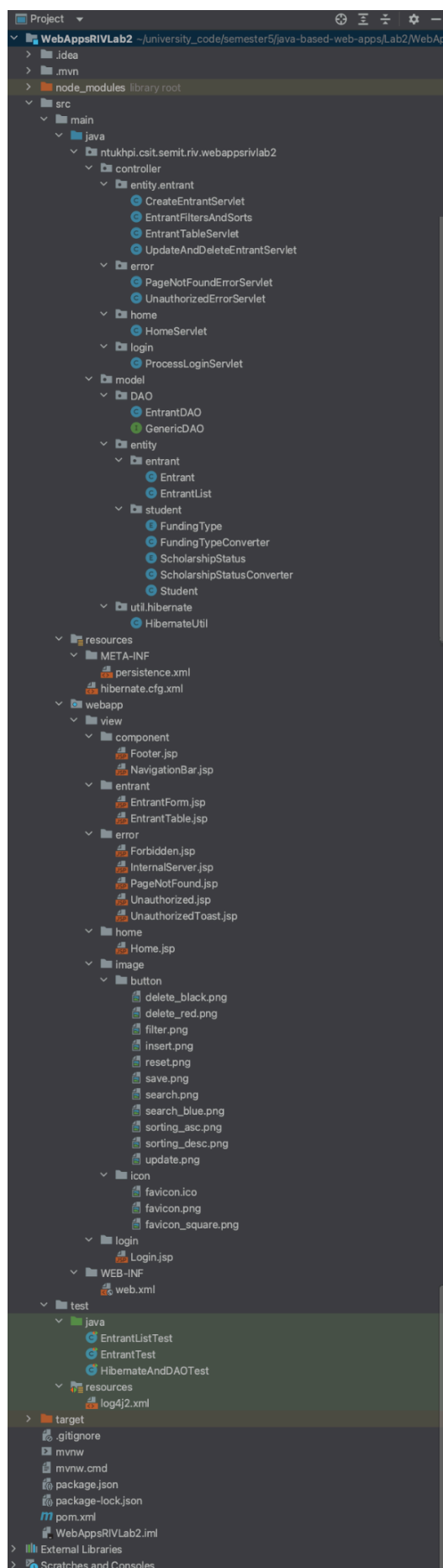


Рисунок 2 – Структура проекту WebAppsRIVLab2

Основні шари проєкту можна визначити наступним чином описаним нижче.

1 Контролери (Controller Layer). У пакеті controller є кілька підпакетів для різних сутностей:

- а) entity.entrant – сервлети для управління таблицею з абітурієнтами, включаючи створення, оновлення та видалення;
- б) error – сервлети для обробки помилок (сторінки з помилками);
- в) home – обробка домашньої сторінки;
- г) login – обробка входу в систему.

2 Модель (Model Layer):

- а) DAO (Data Access Object) – у пакеті model.DAO знаходяться класи для доступу до бази даних (EntrantDAO, GenericDAO);
- б) сутності (entity) – пакети entrant та student містять сутності для абітурієнтів та студентів – це Java-класи, які відображаються на таблиці бази даних;
- в) пакет util.hibernate містить клас HibernateUtil для управління сесіями Hibernate та конфігурацією.

3 Інфраструктура (Infrastructure Layer): файли конфігурації Hibernate та з'єднання з базою даних (hibernate.cfg.xml, persistence.xml) знаходяться в каталозі resources/META-INF.

4 Вебчастина (Presentation Layer). У директорії webapp/view знаходяться JSP-сторінки для представлення даних користувачу:

- а) EntrantForm.jsp і EntrantTable.jsp для роботи з абітурієнтами;
- б) сторінки для помилок (Forbidden.jsp, PageNotFound.jsp тощо);
- в) сторінка для управління входом в систему (Login.jsp);
- г) домашня сторінка вебзастосунку (Home.jsp);
- д) відображення кнопок та іконок (image/button, image/icon).

Реалізований проєкт також містить супровідні компоненти описані далі в тексті.

1 Тести (Test Layer). Пакет test містить JUnit тести для перевірки функціональності різних компонентів:

- а) EntrantListTest, EntrantTest для перевірки логіки роботи з абітурієнтами;
- б) HibernateAndDAOTest для тестування взаємодії з базою даних через Hibernate.

2 Конфігурація та допоміжні файли:

- а) log4j2.xml для налаштування логування;
- б) web.xml у папці WEB-INF для конфігурації сервлетів і маршрутизації.

Схема бази даних. Схема створеної бази даних на основі DBMS MySQL 8.0.30 відображена на рис. 3. Структура створених таблиць entrants та students наведена на рис. 4 та рис. 5.

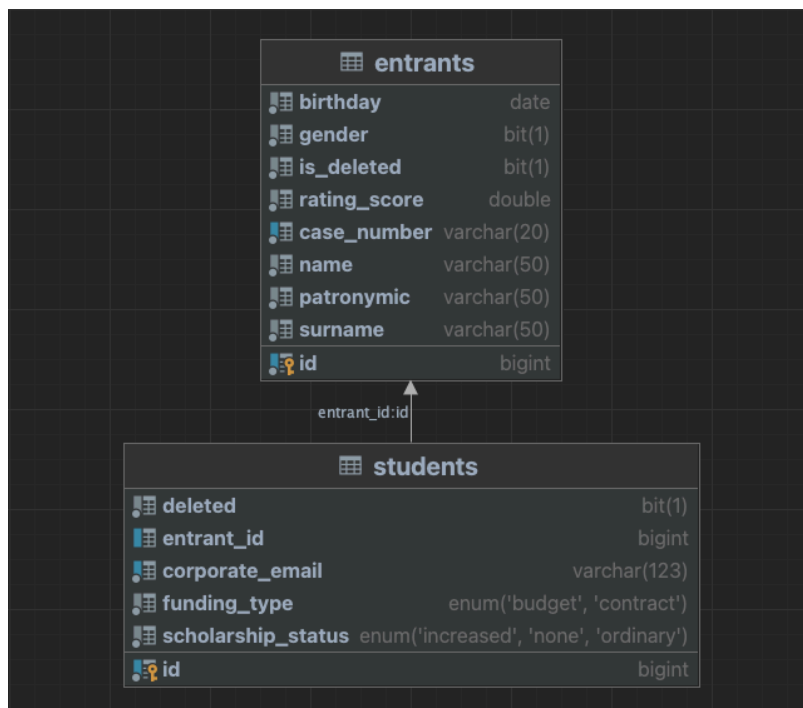


Рисунок 3 – Схема бази даних на основі DBMS MySQL 8.0.30

Файл hibernate.cfg.xml. Створений файл hibernate.cfg.xml для конфігурації Hibernate та з'єднання з базою даних наведено у додатку А.2.

columns (entrants)											
	Name	Comment	Data Type	Not Null	Auto Increment	Column Kind	Default Expression	Hidden	On Update	Position	Collation
1	birthday		date	true		NORMAL		false		1	null
2	gender		bit(1)	true		NORMAL	b'1'	false		2	null
3	is_deleted		bit(1)	true		NORMAL	b'0'	false		3	null
4	rating_score		double	true		NORMAL		false		4	null
5	id		bigint	true	33	NORMAL		false		5	null
6	case_number		varchar(20)	true		NORMAL		false		6	null
7	name		varchar(50)	true		NORMAL		false		7	null
8	patronymic		varchar(50)	true		NORMAL		false		8	null
9	surname		varchar(50)	true		NORMAL		false		9	null

keys (entrants)	
Name	Primary
1 PRIMARY	true
2 UKJ36ja5h4xym91tmpdw81xno84	false

indexes (entrants)				
Name	Comment	Unique	Type	
1 PRIMARY		true	btree	
2 UKJ36ja5h4xym91tmpdw81xno84		true	btree	

checks (entrants)	
Name	Predicate
1 entrants_chk_2	('rating_score' > 120.00) and ('rating_score' <= 200.00)
2 entrants_chk_1	'birthday' between _utf8mb4\1914-01-01\ and _utf8mb4\2008-01-01\
3 entrants_chk_3	regexp_like('case_number', _utf8mb4\'^([0-9]{1,10}[0-9]{2}-[0-9]{1,4}\$\', _utf8mb4\c') = 1
4 entrants_chk_4	regexp_like('name', _utf8mb4\'^{1,50}\$([0-9]{1,10}[0-9]{2}-[0-9]{1,4}\$\', _utf8mb4\c') = 1
5 entrants_chk_5	regexp_like('patronymic', _utf8mb4\'^{1,50}\$([0-9]{1,10}[0-9]{2}-[0-9]{1,4}\$\', _utf8mb4\c') = 1
6 entrants_chk_6	regexp_like('surname', _utf8mb4\'^{1,50}\$([0-9]{1,10}[0-9]{2}-[0-9]{1,4}\$\', _utf8mb4\c') = 1

Рисунок 4 – Структура колонок таблиці entrants

columns (students)											
	Name	Comment	Data Type	Not Null	Auto Increment	Column Kind	Default Expression	Hidden	On Update	Position	Collation
1	deleted		bit(1)	true		NORMAL	b'0'	false		1	null
2	entrant_id		bigint	false		NORMAL		false		2	null
3	id		bigint	true	1	NORMAL		false		3	null
4	corporate_email		varchar(123)	true		NORMAL		false		4	null
5	funding_type		enum('BUDGET', 'CONTRACT')	true		NORMAL		false		5	null
6	scholarship_status		enum('INCREASED', 'NONE', 'ORDINARY')	true		NORMAL	'NONE'	false		6	null

keys (students)	
Name	Primary
1 PRIMARY	true
2 UKeqgu37Lcg62sduw3n7d7gre0v	false
3 UKng7nfdg9laddmqsl1be38mkx	false

indexes (students)				
Name	Comment	Unique	Type	
1 UKeqgu37Lcg62sduw3n7d7gre0v		true	btree	
2 PRIMARY		true	btree	
3 UKng7nfdg9laddmqsl1be38mkx		true	btree	

Рисунок 5 – Структура колонок таблиці students

Клас `HibernateUtil.java`. Клас `HibernateUtil` використовується для управління сесіями `Hibernate` та створення конфігурації підключення до бази даних у додатках `Java` з використанням `Hibernate` як `ORM` (Object-Relational Mapping).

Основне призначення цього класу – створення, керування та закриття об'єкта `SessionFactory`, який є центральним механізмом роботи `Hibernate` для отримання сесій (об'єктів `Session`) і виконання операцій з базою даних.

Код реалізованого класу `HibernateUtil.java` представлений у додатку А.3.

2 Створення `Entity`-класів, таблиць у базі даних та `CRUD` операці

Результатом виконання даного завдання є реалізований `Entity`-клас відповідно до індивідуального завдання наведеного на рис. 1 у даній роботі шляхом додавання до полів анотацій `Java Persistence API`.

Клас `Entrant`. Клас `Entrant` є сутністю, що представляє запис у таблиці “entrants” бази даних. Він зберігає таку інформацію про вступника:

- унікальний ідентифікатор запису в базі даних (числове значення);
- номер особової справи вступника (рядковий літерал);
- прізвище вступника (рядковий літерал);
- ім'я вступника (рядковий літерал);
- по-батькові вступника (рядковий літерал);
- дата народження вступника (локальна дата);
- стать вступника (булеве значення);
- рейтинговий бал вступника (числове значення);
- статус запису в базі даних – видалений або не видалений (рядковий літерал).

Клас використовує `Hibernate` для `ORM` (`Object-Relational Mapping`) і включає валідаційні обмеження для різних полів, таких як номер справи, складові імені та оцінка рейтингу.

Основні особливості включають підтримку логічного видалення записів за технікою `soft-delete` через поле `isDeleted`, валідацію полів для дотримання правильного формату, такі як для номера справи та імені, а також фільтри для виключення логічно видалених записів з результатів запитів. Клас також надає методи для форматування дат та представлення полів у різних форматах, що спрощує маніпуляцію даними.

Код класу Entrant представлений у додатку Б.1.

Клас EntrantList. Клас EntrantList реалізує шаблон Singleton для зберігання списку об'єктів типу Entrant (абітурієнтів). Він успадковує функціональність від класу ArrayList та забезпечує доступ до єдиного екземпляра цього списку впродовж життєвого циклу застосунку.

Клас ініціалізує список абітурієнтів із заздалегідь визначеними даними, що використовуються для тестування та демонстрації функціональності. Серед додаткових можливостей є зберігання записів про видалені абітурієнти та підтримка прикладів із різними форматами імен. EntrantList служить для моделювання бази даних, що полегшує роботу з колекцією даних у проєкті.

Код класу EntrantList представлений у додатку Б.2.

Клас Student. Клас Student є сутністю, що представляє студента, який перейшов зі статусу вступника. Він містить таку інформацію про студента:

- унікальний ідентифікатор запису в базі даних про студента (числове значення);
- унікальний ідентифікатор запису в базі даних про даного студента у таблиці вступників (числове значення);
- тип фінансування – бюджет чи контракт (набір константних значень);
- статус нарахування стипендії – підвищена, звичайна чи відсутня (набір константних значень);
- адреса корпоративної пошти (рядковий літерал);
- статус запису в базі даних – видалений або не видалений (рядковий літерал).

Клас мапується до таблиці “students” у базі даних і використовує анотації JPA та Hibernate для визначення відносин, обмежень і фільтрів.

Основні характеристики включають зв'язок “один до одного” з сутністю Entrant, що представляє дані про вступника, та поля для зберігання типу фінансування і статусу стипендії. Поле з адресою корпоративної пошти має шаблон валідації, що забезпечує відповідність формату корпоративної адреси.

Логічне видалення реалізоване за технікою soft-delete з використанням поля deleted, яке дозволяє виключати видалені записи із запитів.

Код класу Student представлений у додатку Б.3.

Для мапінгу створених класів сутностей у файлі hibernate.cfg.xml було додано наступні властивості відображені на рис. 6.

```
<!-- HBM mappings or annotated classes -->
<mapping class="ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant"/>
<mapping class="ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.student.Student"/>
<mapping class="ntukhpi.csit.semit.riv.webappsrivlab2.model.util.hibernate.HibernateUtil"/>
```

Рисунок 6 – Теги у файлі hibernate.cfg.xml для включення класів сутностей у мапінг бази даних

Перелік FundingType. Цей клас є переліком (enum), яке представляє тип фінансування студента: контракт або бюджет.

Він включає в себе методи для отримання рядкового представлення кожного типу фінансування з використанням методу getValue() та пошук відповідного типу за рядком з використанням методу fromValue(String).

Кожен елемент перерахування відповідає певному рядку на українській мові, що зручно для відображення цих даних в інтерфейсі користувача.

Код переліку FundingType представлений у додатку Б.4.

Клас FundingTypeConverter. Цей клас є конвертером JPA для перетворення між типом даних FundingType і його рядковим представленням у базі даних.

Метод convertToDatabaseColumn перетворює enum у рядок для збереження в базу даних, а метод convertToEntityAttribute перетворює рядок із бази даних назад у відповідний enum.

Конвертер автоматично застосовується до всіх полів типу FundingType завдяки параметру autoApply=true у анотації @Converter.

Код класу FundingTypeConverter представлений у додатку Б.5.

Перелік ScholarshipStatus. Цей клас представляє статус стипендії студента, який може бути: звичайна, підвищена або відсутня.

Включає методи для отримання рядкового представлення кожного статусу з використанням методу `getValue()` та пошуку відповідного статусу за рядком з використанням методу `fromValue(String)`.

Це зручний інструмент для зберігання статусу стипендії у зручному для читання форматі та для взаємодії з базою даних.

Код переліку `ScholarshipStatus` представлений у додатку Б.6.

Клас ScholarshipStatusConverter. Це конвертер JPA, який відповідає за перетворення між типом `ScholarshipStatus` та його рядковим представленням у базі даних.

Конвертер автоматично конвертує `enum` у рядок для збереження в базу даних та назад у `enum` при читанні з бази даних.

Завдяки параметру `autoApply=true`, цей конвертер автоматично застосовується до полів типу `ScholarshipStatus`.

Код класу `ScholarshipStatusConverter` представлений у додатку Б.7.

Реалізовані переліки дозволяють легко оперувати константами, які відповідають типам фінансування або статусам стипендії, зручні для відображення і зберігання даних.

Конвертери спрощують збереження та зчитування цих констант з бази даних у вигляді рядків, зберігаючи при цьому типізацію на рівні коду для безпечного використання.

Інтерфейс GenericDAO. Клас `GenericDAO` є інтерфейсом для виконання основних операцій взаємодії з базою даних з використанням `Hibernate`, реалізуючи `CRUD` (`Create`, `Read`, `Update`, `Delete`) операції для будь-якого типу сутності. Він забезпечує загальні методи для обробки сутностей, таких як отримання списків записів із можливістю фільтрації та сортування, пошук за ключовими полями, перевірка наявності дублікатів та валідація даних.

Інтерфейс має наступні аспекти реалізації описані нижче.

1 CRUD операції. Підтримується вставка, оновлення та видалення записів з використанням транзакцій Hibernate. Кожен метод працює з SessionFactory для доступу до бази даних.

2 Фільтрація та сортування. Реалізовані можливості отримання відфільтрованих і відсортованих списків з використанням критеріїв (Criteria API) та використання предикатів для складних пошукових запитів.

3 Валідація та обробка дублікатів. Перед виконанням операцій вставки або оновлення виконується валідація даних із застосуванням вбудованого валідатора (Validator) і перевіряється наявність дублікатів.

4 Логічне видалення. Підтримується механізм логічного видалення записів через зміну статусу поля isDeleted, що дозволяє фільтрувати видалені записи з подальших запитів.

Цей клас є базовим для роботи з будь-якими сутностями в базі даних, оскільки дозволяє розширювати функціональність і переуспадкокувати методи для конкретних випадків використання.

Код інтерфейсу GenericDAO представлений у додатку Б.8.

Клас EntrantDAO. EntrantDAO – це клас, який реалізує патерн Data Access Object (DAO) для роботи з сутністю Entrant в базі даних. Він відповідає за взаємодію з даними абітурієнтів (Entrant) та надає методи для здійснення основних операцій CRUD (Create, Read, Update, Delete), а також фільтрації, сортування та перевірки на наявність дублікатів.

Основні методи даного класу реалізації інтерфейсу включають наступні методи описані нижче.

1 Пошук за різними критеріями. Методи findByCaseNumber(), findBySurname(), findByBirthday(), findByGender(), findByRatingScore() – це методи для пошуку абітурієнтів за конкретними полями, такими як номер справи, прізвище, дата народження, стать та рейтинг.

2 Отримання всього списку абітурієнтів. Метод getAllList() – повертає список всіх абітурієнтів у базі даних, з можливістю виключити логічно видалені записи.

3 Фільтрація та сортування. Метод `getFilteredAndSortedList()` – дозволяє отримати відфільтрований і відсортований список абітурієнтів на основі переданих критеріїв.

4 Перевірка на дублікацію. Метод `hasDuplicate()` – перевіряє, чи існує вже запис з тими самими унікальними полями, що і у вказаного об'єкта абітурієнта.

Цей клас використовує реалізацію інтерфейсу `GenericDAO`, що дозволяє виконувати загальні операції бази даних для сутності `Entrant`.

Код класу `EntrantDAO` представлений у додатку Б.9.

Для наповнення таблиці “entrants” тестовими даними було створено наступний клас.

Тестовий набір `EntrantTest`. `EntrantTest` – це клас тестування, створений для перевірки функціональності класу `Entrant` з використанням `JUnit 5`. Клас містить різноманітні тестові кейси, які перевіряють коректність валідації даних, форматування та основної бізнес-логіки, що використовується у класі `Entrant`.

Основні тестові сценарії даного класу включають:

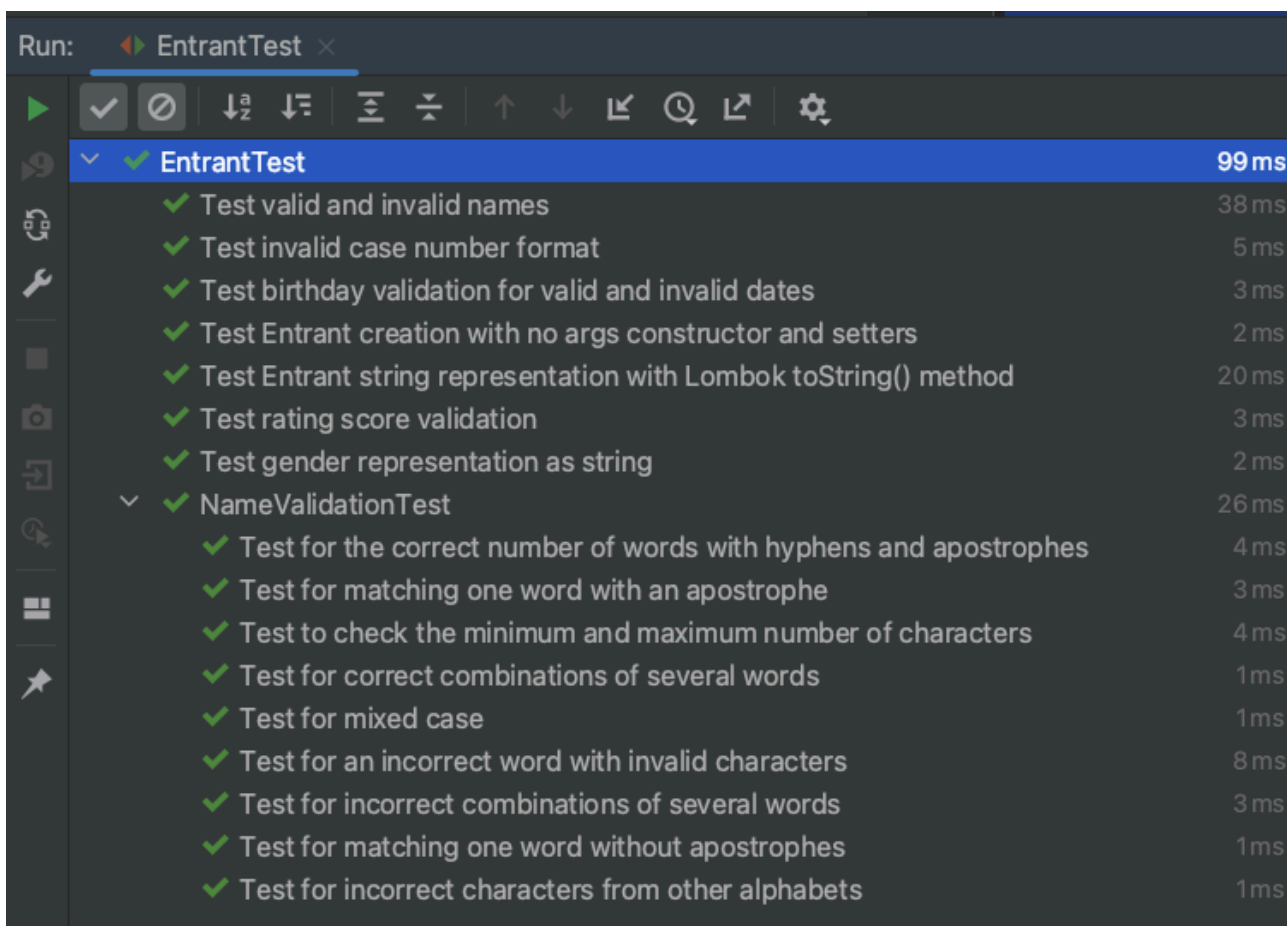
- 1) тестування створення об'єкта `Entrant` з використанням конструктора без аргументів та методів “setter”;
- 2) перевірка рядкового представлення об'єкта `Entrant`, яка використовує метод `toString()` (створений з використанням `Lombok`);
- 3) перевірка валідації номеру справи (case number) на відповідність формату і невалідний формат викликає виняток;
- 4) тестування валідації імені: тести перевіряють, чи відповідає ім'я правилам форматування, включаючи валідні та невалідні формати;
- 5) тестування рядкового представлення статі: перевіряє, чи правильно представлена стаття (“male” або “female”);
- 6) тестування валідації дати народження: перевіряється формат дати та допустимі вікові межі для абітурієнта;
- 7) тестування валідації рейтингового балу: перевіряє, чи знаходиться бал у допустимому діапазоні від 120.01 до 200.00.

Додатково клас містить вкладений клас `NameValidationTest`, який спеціально тестує правила форматування імені з використанням регулярних виразів.

Цей клас тестів забезпечує повну перевірку функціональності та валідації класу `Entrant`, дозволяючи виявити та виправити потенційні помилки у бізнес-логіці або форматуванні даних.

Код тестового набору `EntrantTest` представлений у додатку Б.10.

Результати виконання тестових випадків даного набору тестів для модульного тестування представлені на рис. 7.



Test Case	Duration
EntrantTest	99 ms
✓ Test valid and invalid names	38 ms
✓ Test invalid case number format	5 ms
✓ Test birthday validation for valid and invalid dates	3 ms
✓ Test Entrant creation with no args constructor and setters	2 ms
✓ Test Entrant string representation with Lombok toString() method	20 ms
✓ Test rating score validation	3 ms
✓ Test gender representation as string	2 ms
NameValidationTest	26 ms
✓ Test for the correct number of words with hyphens and apostrophes	4 ms
✓ Test for matching one word with an apostrophe	3 ms
✓ Test to check the minimum and maximum number of characters	4 ms
✓ Test for correct combinations of several words	1 ms
✓ Test for mixed case	1 ms
✓ Test for an incorrect word with invalid characters	8 ms
✓ Test for incorrect combinations of several words	3 ms
✓ Test for matching one word without apostrophes	1 ms
✓ Test for incorrect characters from other alphabets	1 ms

Рисунок 7 – Результати виконання тестового набору `EntrantTest`

Тестовий набір `EntrantListTest`. `EntrantListTest` – це клас тестування, створений з використанням JUnit 5 для перевірки функціональності класу `EntrantList`. Основна мета цього класу – перевірити правильність створення

списку абітурієнтів, валідацію даних та відповідність певним умовам, таким як унікальність номерів справ, правильне значення статі, формат дати народження та діапазон рейтингового балу.

Основні тестові випадки даного класу включають наступні кейси описані нижче.

1 Тестування створення списку абітурієнтів. Перевіряється, чи правильно створений та заповнений список.

2 Перевірка даних абітурієнтів у списку. Перевіряється, чи відповідають дані першого абітурієнта очікуваним значенням.

3 Валідація гендеру. Перевіряється, що всі абітурієнти мають валідні значення статі (true або false).

4 Перевірка діапазону рейтингового балу. Перевіряється, що рейтинговий бал кожного абітурієнта знаходиться в межах 120.01 - 200.00.

5 Унікальність номерів справ. Перевіряється, що всі абітурієнти мають унікальні номери справ.

6 Валідація формату дати народження. Перевіряється, що формат дати народження для всіх абітурієнтів відповідає “dd.ММ.уууу”.

Клас забезпечує повну перевірку логіки роботи зі списком абітурієнтів, а також коректність форматування та збереження даних у цьому списку.

Код тестового набору EntrantListTest представлений у додатку Б.11.

Результати виконання тестових випадків даного набору тестів для модульного тестування представлені на рис. 8 далі в тексті роботи.

Тестовий набір HibernateAndDAOTest. HibernateAndDAOTest – це клас тестування, написаний з використанням JUnit 5 для перевірки взаємодії з Hibernate і операцій CRUD з використанням EntrantDAO і сутності Entrant. Основна мета класу – протестувати різні операції бази даних, включаючи вставку, оновлення, видалення та пошук записів абітурієнтів. Крім того, він перевіряє можливості фільтрації та сортування даних з використанням DAO методів.

Основні тестові випадки даного класу включають наступні кейси описані нижче.

1 Тестування запису всіх абітурієнтів. Всі записи з класу EntrantList додаються до бази даних і перевіряється, чи всі вони успішно збережені.

Run: EntrantListTest	
✓ EntrantListTest	43 ms
✓ Validate Gender Data in Entrant List	13 ms
✓ Check birthday date format in Entrant List	2 ms
✓ Check uniqueness of case numbers in Entrant List	2 ms
✓ Create Entrant List	5 ms
✓ Get Entrant List string representation	19 ms
✓ Check Entrant Data in the List	1 ms
✓ Validate Rating Score Range in Entrant List	1 ms

Рисунок 8 – Результати виконання тестового набору EntrantListTest

2 Операції CRUD. Тестування повного циклу операцій (створення, читання, оновлення, видалення) для одного запису.

3 Пошук абітурієнтів за різними критеріями. Пошук за ID, номером справи, прізвищем, ім'ям, по-батькові, датою народження, гендером та рейтинговим балом.

Run: HibernateAndDAOTest	
✓ HibernateAndDAOTest	337 ms
✓ Find an Entrant in the database by gender	29 ms
✓ Find an Entrant in the database by rating score	4 ms
✓ Find an Entrant in the database by surname & name & patronymic	4 ms
✓ Find an Entrant in the database by id	7 ms
✓ Test CRUD operations and get all Entrants from the database including deleted re	55 ms
✓ Find an Entrant in the database by case number	5 ms
✓ Find an Entrant in the database by surname	5 ms
✓ Get all Entrants from the database	6 ms
✓ Find an Entrant in the database by birthday	4 ms
✓ Get a list of records with filtering and sorting	218 ms

Рисунок 9 – Результати виконання тестового набору HibernateAndDAOTest

4 Отримання відфільтрованих та відсортованих списків. Перевіряється можливість отримати список записів на основі фільтрації за конкретними полями та сортування за прізвищем.

Клас також включає методи для порівняння отриманих результатів із очікуваними значеннями для забезпечення коректності виконання CRUD операцій.

Код тестового набору HibernateAndDAOTest представлений у додатку Б.12.

Результати виконання тестових випадків даного набору тестів для модульного тестування представлені на рис. 9 вище в тексті роботи.

3 Створення JSP сторінки таблиці з пошуком, сортуванням та фільтрацією даних та сервлета до неї

У даній лабораторній роботі було використано такі JSP сторінки та сервлети до них з попередньої лабораторної роботи:

- 1) сторінка авторизації;
- 2) сторінка помилки авторизації (401 Unauthorized);
- 3) домашня сторінка;
- 4) компоненти для сторінок вебзастосунку:
 - а) хедер;
 - б) футер.

Результатом виконання поточного та наступного завдання даної лабораторної роботи є реалізовані наступні JSP сторінки:

- 1) таблиця з даними абітурієнтів;
- 2) сторінка для додавання, оновлення та видалення даних з таблиці;
- 3) сторінка заборони доступу до ресурсу (403 Forbidden);
- 4) сторінка помилки доступу до неіснуючого ресурсу (404 Not Found);
- 5) сторінка помилки сервера (500 Internal Server Error).

Всі сторінки вебзастосунку використовують фреймворк Bootstrap для кращого форматування контенту та адаптивного дизайну. Використання Bootstrap у цьому проєкті надало потужні можливості для швидкої та ефективної розробки інтерфейсів з використанням готових компонентів і стилів. Контент сторінок поділений на секції з використанням CSS-класів Bootstrap, що робить його зручним для перегляду на різних пристроях. Використання готових елементів, таких як кнопки, форми, навігаційні панелі, таблиці, модальні вікна та інші інтерфейсні компоненти значно скоротило час розробки. Інтеграція цих компонентів дозволило створити сучасні, інтерактивні та естетично привабливі вебсторінки, зберігаючи консистенцію стилів у всьому проєкті.

Сторінка EntrantTable. Цей JSP-файл відповідає за відображення таблиці абітурієнтів (Entrant Table) з використанням фреймворку JSP, бібліотек JSTL та функціональних можливостей JavaScript для фільтрації, сортування та пошуку. Основні аспекти реалізації даної сторінки EntrantTable наведено нижче.

1 Використані технології та бібліотеки:

- а) JSTL для відображення даних та керування логікою на стороні сервера;
- б) Bootstrap для стилізації інтерфейсу та забезпечення адаптивного дизайну;
- в) JSP теги: `c:forEach` для циклічного відображення полів і записів таблиці абітурієнтів.

2 Пошук та фільтрація:

- а) пошук за номером справи та прізвищем реалізований через текстове поле пошуку;
- б) фільтри за датою народження (birthday) та рейтинговим балом (ratingScore) дозволяють користувачу вибирати інтервали для фільтрації результатів у таблиці.

3 Сортування: сортування записів по стовпцях таблиці реалізоване через гіперпосилання на кожному стовпці, які змінюють порядок сортування при натисканні.

4 Додавання та редагування записів:

- а) кнопка для додавання нового абітурієнта відправляє користувача на відповідну сторінку для створення нового запису;
- б) кожен запис у таблиці має опції для редагування та видалення (кнопки з іконками).

5 Логічна структура сторінки з таблицею даних про вступників: NavBar та Footer включаються до даної сторінки, забезпечуючи єдиний стиль навігації по всіх сторінках додатку.

6 JavaScript для інтерактивності:

- а) функції JavaScript забезпечують динамічну обробку дій користувача, таких як фільтрація, пошук та сортування;
- б) використовуються функції для встановлення мінімальних та максимальних дат народження відповідно до дозволених вікових меж.

Сторінка EntrantTable забезпечує зручний інтерфейс для користувачів з можливістю перегляду, пошуку, фільтрації та сортування записів абітурієнтів.

Код сторінки EntrantTable представлений у додатку B.1.

Сторінки помилок Forbidden, PageNotFound, InternalServer. Ці JSP-файли призначені для обробки різних помилок, таких як “404 Not Found”, “500 Internal Server Error”, і “403 Forbidden”. Вони надають прості та зрозумілі повідомлення для користувача у випадку виникнення помилок на сервері або при доступі до ресурсів, які не знайдені або заборонені. Кожна сторінка має однакову структуру з невеликими відмінностями у заголовках і повідомленнях про помилки.

Візуальні елементи сторінок включають використання карток (card), рамок (border), тіней (shadow) та інших компонентів для покращення зовнішнього вигляду сторінки.

Для кожної помилки є свій унікальний заголовок і повідомлення, що пояснюють суть проблеми:

- 404 Page Not Found: повідомляє, що сторінка не знайдена;
- 500 Internal Server Error: повідомляє про внутрішню помилку на сервері;

– 403 Forbidden Error: повідомляє про заборону доступу до ресурсу.

Всі сторінки мають кнопку Go Back, яка повертає користувача на попередню сторінку. Для сторінок 500 та 403 є додаткова кнопка Go Home, яка перенаправляє користувача на головну сторінку додатку.

Ці сторінки використовуються для обробки стандартних HTTP помилок. з використанням вказання відповідних властивостей у файлі web.xml застосунок налаштовує сервер так, щоб він виконував перенаправлення клієнта на відповідну сторінку у випадку, коли виникає одна з цих помилок. Даний код відображений на рис. 10.

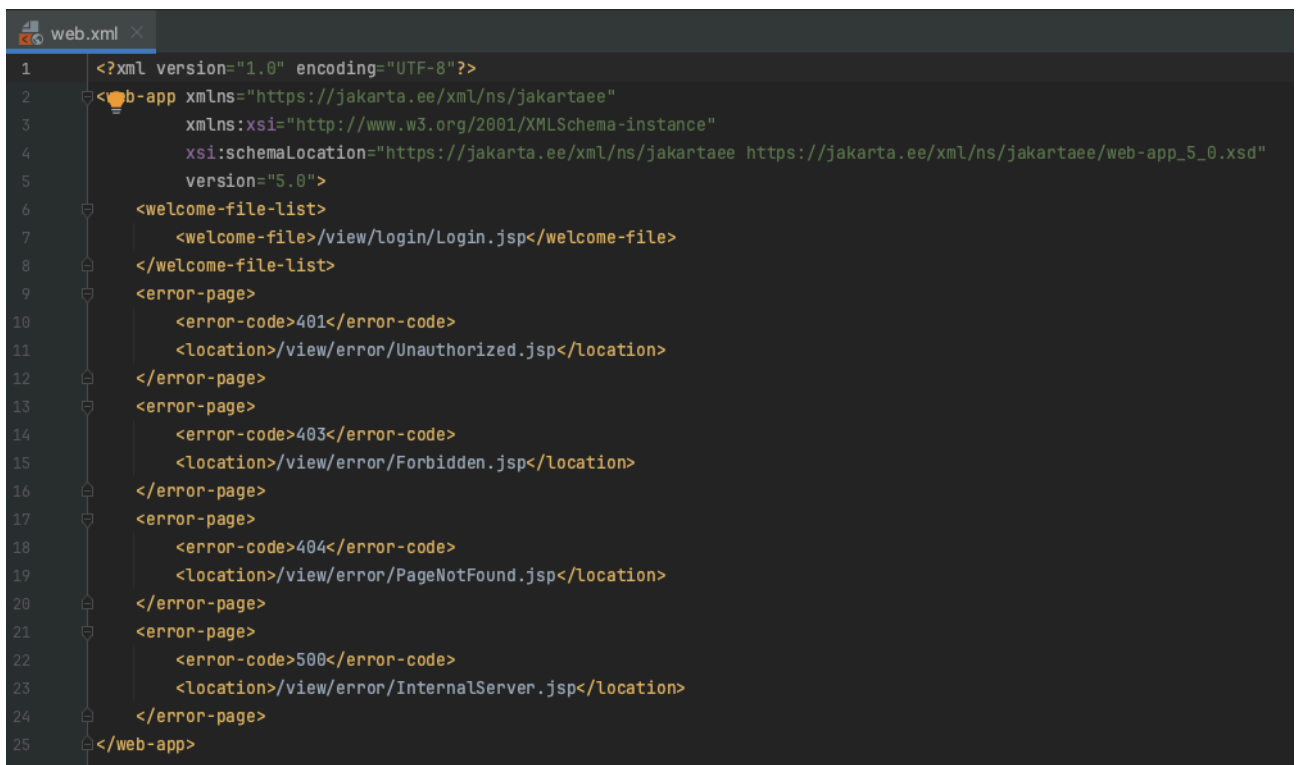


Рисунок 10 – Налаштування властивостей у файлі web.xml

Код сторінки PageNotFound представлений у додатку B.2. Код сторінок Forbidden, InternalServer аналогічний данному файлу, лиш змінюється повідомлення про помилку.

Сервлет EntrantTableServlet. Цей сервлет EntrantTableServlet відповідає за обробку запитів для відображення таблиці з даними про абітурієнтів, забезпечуючи функціональність фільтрації, сортування та пошуку записів на

основі параметрів, переданих у запиті. Основні характеристики даного сервлета представлені далі.

1 Ініціалізація сервлета. У методі `init()` виконується початкова завантаження списку абітурієнтів (`entrant`) у базу даних при ініціалізації сервлета. Використовується метод `insertAll()` для збереження всіх записів у базу даних.

2 Метод `doGet()`. Основна логіка обробки HTTP GET-запитів наступна:

а) якщо присутні параметри в запиті, то система намагається витягнути фільтри, сортування та критерії пошуку з `get`-параметрів запиту з використанням класу `EntrantFiltersAndSorts`;

б) якщо жодні параметри не передані, то витягується весь список абітурієнтів з бази даних з використанням методу `getAllList()`;

в) дані зберігаються в атрибутах запиту (`entrantList` і `fieldNames`) для подальшого відображення на JSP-сторінці.

3 Фільтрація, сортування та пошук. Інформація ці методи роботи з даними обробляються через допоміжний клас `EntrantFiltersAndSorts`, який витягує параметри з HTTP-запиту і генерує відповідні критерії для пошуку в базі даних. Якщо при обробці параметрів запиту виникає помилка, вона виводиться у стандартний потік помилок.

4 Перенаправлення на JSP. Після обробки даних запиту і отримання списку абітурієнтів сервлет передає ці дані в JSP-сторінку `EntrantTable.jsp` з використанням механізму `RequestDispatcher` методу `forward()`.

Можливі сценарії використання даного сервлета `EntrantTableServlet` наступні:

1 Без `get`-параметрів у запиті. При такому запиті відображається весь список абітурієнтів.

2 З `get`-параметрами у запиті. При такому запиті використовуються фільтри, сортування та пошук для відображення специфічних результатів.

Ключові компоненти сервлета `EntrantTableServlet`:

1) EntrantDAO – використовується для взаємодії з базою даних і виконання CRUD операцій;

2) EntrantFiltersAndSorts – допоміжний клас для отримання фільтрів, сортувань та пошукових параметрів з запиту;

3) EntrantList – клас для початкового створення списку абітурієнтів, який може бути використаний для тестування чи демонстрації.

Цей сервлет реалізує динамічне управління даними та надає користувачам можливість взаємодіяти з базою даних абітурієнтів через зручний інтерфейс з функціоналом сортування та фільтрації.

Код сервлета EntrantTableServlet представлений у додатку В.3.

Клас EntrantFiltersAndSorts. У цьому класі EntrantFiltersAndSorts реалізована логіка для обробки фільтрації, сортування і пошуку по полях сутності Entrant із get-параметрів HTTP-запиту. Основний функціонал класу описаний далі.

1 Фільтри. Методи для додавання фільтрів по полях, таких як номер справи, ім'я, прізвище, по-батькові, дата народження, стать, та рейтинг.

2 Сортування. Сортування підтримується за всіма полями (окрім унікального ідентифікатора) з напрямками за зростанням (ASC) або за зменшенням (DESC).

3 Пошук. Поля “номер справи” і “прізвище” можуть бути використані для пошуку.

4 Формат дати. Формат дати встановлюється за міжнародним стандартом ISO 8601 що відповідає масці “уууу-ММ-дд”.

Методи фільтрування та сортування отримують значення get-параметрів із HTTP-запиту та зберігають їх у відповідних структурах Map (filters, sortOrder, search), які потім можна використовувати у запитах до бази даних.

Код класу EntrantFiltersAndSorts представлений у додатку В.4.

Сервлет UnauthorizedErrorServlet. Це сервлет, який обробляє запити до URL “/unauthorized-error”. Його основне призначення – перенаправляти

користувача на сторінку з повідомленням про помилку доступу, інформуючи про те, що користувач не має дозволу на доступ до певного ресурсу.

Сервлет обробляє виключно GET-запити, що надходять на адресу “/unauthorized-error”. Виконує перенаправлення користувача на спеціально підготовлену JSP-сторінку Unauthorized.jsp, де відображається повідомлення про помилку авторизації.

Основні функції сервлета:

- 1) інформування користувачів про відмову у доступі до ресурсу без відповідних прав;
- 2) використання централізованого механізму для обробки помилок 403 у вебзастосунку;
- 3) створення зручного інтерфейсу для відображення повідомлення про помилку.

Код сервлета UnauthorizedErrorServlet представлений у додатку B.5.

Сервлет PageNotFoundServlet. Це сервлет, який обробляє запити до URL “/page-not-found-error”. Його основне призначення – перенаправляти користувача на сторінку з повідомленням про помилку 404, коли запитувана сторінка не була знайдена.

Сервлет обробляє виключно GET-запити, що надходять на адресу “/page-not-found-error”. Виконує перенаправлення користувача на спеціально підготовлену JSP-сторінку PageNotFound.jsp, де відображається повідомлення про помилку отримання ресурсу.

Основні функції даного сервлета аналогічні функціям сервлета UnauthorizedErrorServlet описаним вище в тексті роботи.

Код сервлета PageNotFoundServlet представлений у додатку B.6.

Далі в тексті роботи на рис. 10–17 представлені результати роботи функціоналу вебзастосунку реалізованого до завдання 3.

The screenshot shows a web browser window with the URL `localhost:8080/WebAppsRIVLab2/`. The page has a light blue header with a database icon and the title "Student Database Viewer". Below the header is a "Login" button. The main content area contains a login form with two input fields: "Login" (containing the text "irepeshko") and "Password" (containing seven dots). A "Login" button is positioned below the password field. The footer is a light blue bar with three columns of text: "Powered by Inessa Repeshko CS-222a CSIT SEMIT NTU 'KhPI'", "Address National Technical University 'Kharkiv Polytechnic Institute' 2 Kyrpychova 61002 Kharkiv, Ukraine", and "Contact Us +38-(057)-707-66-34 omsroot@kpi.kharkov.ua".

Рисунок 10 – Сторінка авторизації на URL
“`http://localhost:8080/WebAppsRIVLab2/`”

The screenshot shows a web browser window with the URL `localhost:8080/WebAppsRIVLab2/unauthorized-error`. The page displays a red error message box with the text "401 Unauthorized Error" and "Authorisation failed due to entering invalid credentials. Please, try again!". Below the error box is a "Go Back" button. The footer is a light blue bar with three columns of text: "Powered by Inessa Repeshko CS-222a CSIT SEMIT NTU 'KhPI'", "Address National Technical University 'Kharkiv Polytechnic Institute' 2 Kyrpychova 61002 Kharkiv, Ukraine", and "Contact Us +38-(057)-707-66-34 omsroot@kpi.kharkov.ua". A copyright notice "Copyright © 2024 All Rights Reserved" is also present at the bottom center.

Рисунок 11 – Сторінка помилки авторизації на URL
“`http://localhost:8080/WebAppsRIVLab2/unauthorized-error`”

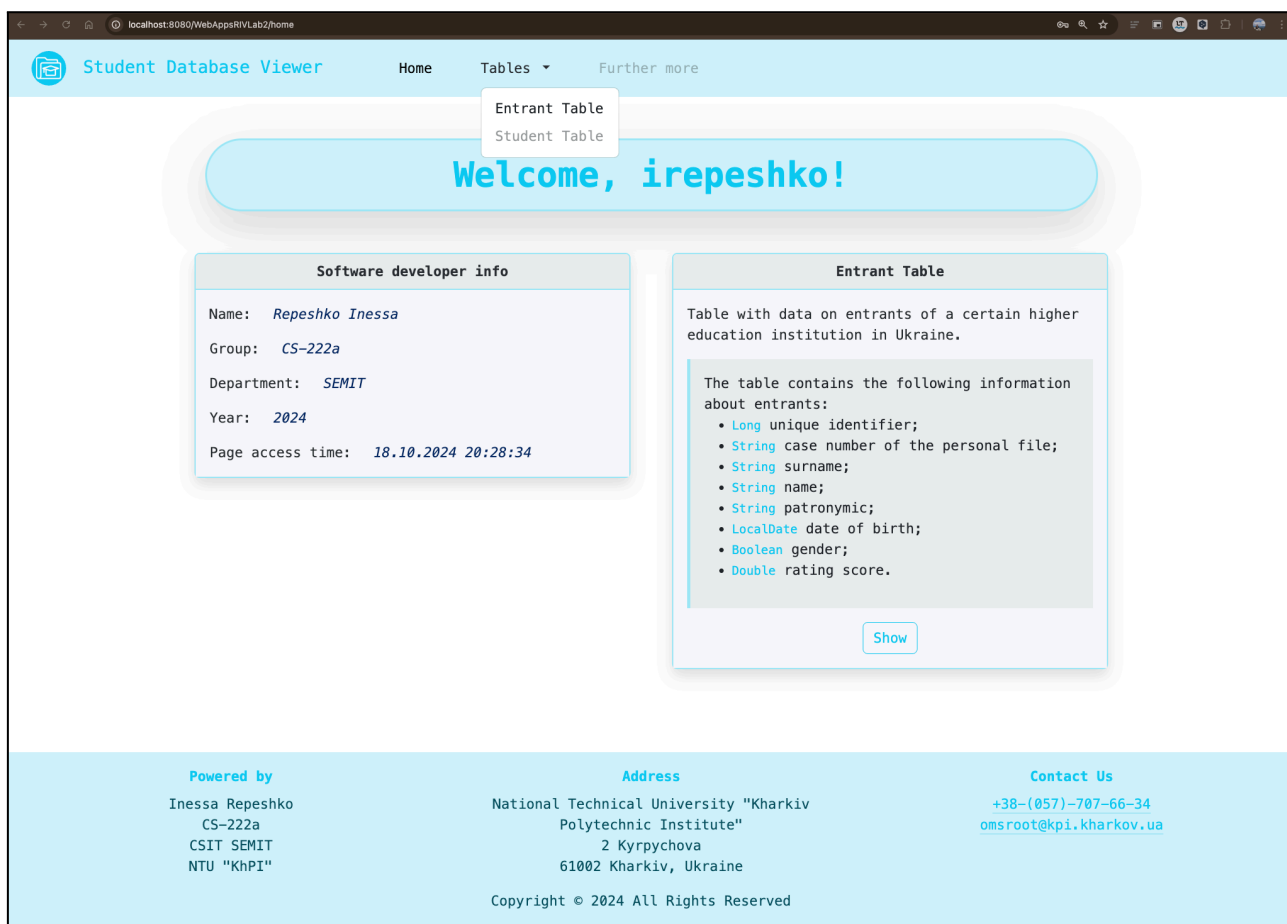


Рисунок 12 – Домашня сторінка вебзастосунку на URL
“http://localhost:8080/WebAppsRIVLab2/home”

JSP-сторінка `EntrantTable.jsp` відповідає за відображення таблиці абітурієнтів із можливістю пошуку, сортування, та фільтрації. Вона інтегрує сучасний інтерфейс на основі Bootstrap, що забезпечує адаптивний дизайн для різних розмірів екранів. Дана сторінка має наступні технічні аспекти реалізації.

1 Пошук і фільтрація. Пошук здійснюється за номером справи або прізвищем абітурієнта. Для цього в формі пошуку є поле `inputSearch`, а відповідний обробник подій на JavaScript дозволяє застосувати або скинути пошук через методи `applySearch()` і `resetSearch()`.

Фільтрація підтримує вибір абітурієнтів за датою народження та рейтингом. Є два окремі блоки для фільтрації:

- дата народження – фільтр дозволяє вибрати інтервал дат, починаючи з мінімуму 110 років і максимуму 16 років від поточної дати.

– рейтинговий бал – фільтр з використанням полів введення чисел з інтервалом від 120.001 до 200.000.

Student Database Viewer Home Tables Further more

Entrant Table

Search
Search by case number, surname

Type to search...

🔍 ↻

Filters

Birthday
Filtering by birthdate interval

Start
dd.mm.yyyy

End
dd.mm.yyyy

🔍 ↻

RatingScore
Filtering by rating score interval

Start

End

🔍 ↻

Add row
Add new row to the end of the table

➕

#	Case Number	Surname	Name	Patronymic	Birthday	Gender	Rating Score	Operations
1	KN22-4814	Бабічев	Олексій	Вікторович	22.05.2005	male	168.119	✏️ 🗑️
2	KN22-5835	Волков	Дмитро	Олександрович	16.07.2005	male	151.200	✏️ 🗑️
3	KN22-5511	Геєнко	Анна	Олексіївна	03.12.2005	female	150.010	✏️ 🗑️
4	KN22-9299	Голоп'яров	Микита	Сергійович	07.11.2005	male	161.480	✏️ 🗑️
5	KN22-9012	Гриценко	Максим	Георгійович	04.12.2005	male	191.941	✏️ 🗑️
6	KN22-6636	Жарий	Вадим	Віталійович	25.10.2005	male	128.349	✏️ 🗑️
7	KN22-5140	Жупанов	Андрій	Володимирович	22.06.2005	male	183.626	✏️ 🗑️
8	KN22-4896	Кльоз	Михайло	Романович	15.01.2005	male	127.286	✏️ 🗑️
9	KN22-4694	Коваль	Микита	Артемович	11.10.2005	male	139.736	✏️ 🗑️
10	KN22-5529	Ковтун	Єгор	Вячеславович	10.04.2005	male	187.794	✏️ 🗑️
11	KN22-6874	Колесниченко	Денис	Євгенович	26.03.2005	male	165.604	✏️ 🗑️
12	KN22-2947	Ласкевич	Андрій	Сергійович	05.09.2005	male	175.399	✏️ 🗑️
13	KN22-3856	Лукаш	Антон	Олександрович	24.11.2005	male	184.473	✏️ 🗑️
14	KN22-7534	Палій	Нікіта	Андрійович	05.05.2005	male	168.000	✏️ 🗑️
15	KN22-3843	Прокопов	Владислав	Сергійович	12.12.2004	male	164.660	✏️ 🗑️
16	KN22-7819	Репешко	Інесса	Віталіївна	06.05.2005	female	185.300	✏️ 🗑️
17	KN22-9115	Скиба	Андрій	Григорович	26.02.2004	male	125.909	✏️ 🗑️
18	KN22-3812	Цибань	Ольга	Юріївна	19.05.2004	female	190.298	✏️ 🗑️
19	KN22-1828	Шевченко	Данило	Валерійович	25.10.2004	male	166.847	✏️ 🗑️
20	KN22-9514	Антипенко	Катерина	Віталіївна	06.01.2005	female	168.467	✏️ 🗑️
21	KN22-2929	Багрянцева	Марина	Русланівна	01.10.2005	female	157.631	✏️ 🗑️
22	KN22-1637	Бондаренко	Ярослав	Юрійович	24.04.2005	male	199.530	✏️ 🗑️
23	KN22-6681	Булгаков	Віктор	Олександрович	17.05.2005	male	153.680	✏️ 🗑️
24	KN22-7557	Васильєв	Михайло	Олександрович	25.01.2005	male	170.867	✏️ 🗑️
25	KN22-8759	Деркач	Дарина	Євгенівна	03.03.2005	female	171.711	✏️ 🗑️
26	KN22-7309	Ілюхін	Тимур	Ігорович	24.01.2005	male	125.771	✏️ 🗑️
27	KN22-8254	Криженко	Аліна	Степанівна	29.08.2005	female	138.746	✏️ 🗑️
28	KN22-3825	Посмашний	Ярослав	Дмитрович	25.02.2000	male	160.715	✏️ 🗑️
29	KN22-8153	Рожкова	Вікторія	Андріївна	04.10.1993	female	188.304	✏️ 🗑️
30	KN22-4788	Сизоненко	Єлизавета	Олександрівна	17.02.1997	female	124.241	✏️ 🗑️
31	АБВГГІІ99-0000	Микол-Іванович Петренко-О'браєн	Іван Петро	Петрович Іванович-О'браєн	06.06.1996	female	200.000	✏️ 🗑️

Go Home

Powered by

Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address

National Technical University "Kharkiv Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок 13 – Сторінка таблиці вступників на URL

“http://localhost:8080/WebAppsRIVLab2/entrants”

Student Database Viewer Home Tables Further more

Entrant Table

Search
Search by case number, surname

Type to search...

Filters

Birthday
Filtering by birthdate interval

Start
dd.mm.yyyy

End
dd.mm.yyyy

RatingScore
Filtering by rating score interval

Start

End

Add row
Add new row to the end of the table



#	Case Number	Surname	Name	Patronymic	Birthday	Gender	Rating Score	Operations
1	KN22-9514	Антипенко	Катерина	Віталіївна	06.01.2005	female	168.467	 
2	KN22-4814	Бабічев	Олексій	Вікторович	22.05.2005	male	168.119	 
3	KN22-2929	Багрянцева	Марина	Русланівна	01.10.2005	female	157.631	 
4	KN22-1637	Бондаренко	Ярослав	Юрійович	24.04.2005	male	199.530	 
5	KN22-6681	Булгаков	Віктор	Олександрович	17.05.2005	male	153.680	 
6	KN22-7557	Васильєв	Михайло	Олександрович	25.01.2005	male	170.867	 
7	KN22-5835	Волков	Дмитро	Олександрович	16.07.2005	male	151.200	 
8	KN22-5511	Геєнко	Анна	Олексіївна	03.12.2005	female	150.010	 
9	KN22-9299	Голоп'яров	Микита	Сергійович	07.11.2005	male	161.480	 
10	KN22-9012	Гриценко	Максим	Георгійович	04.12.2005	male	191.941	 
11	KN22-8759	Деркач	Дарина	Євгенівна	03.03.2005	female	171.711	 
12	KN22-6636	Жарий	Вадим	Віталійович	25.10.2005	male	128.349	 
13	KN22-5140	Жупанов	Андрій	Володимирович	22.06.2005	male	183.626	 
14	KN22-7309	Ілюхін	Тимур	Ігорович	24.01.2005	male	125.771	 
15	KN22-4896	Кльоз	Михайло	Романович	15.01.2005	male	127.286	 
16	KN22-4694	Коваль	Микита	Артемович	11.10.2005	male	139.736	 
17	KN22-5529	Ковтун	Єгор	Вячеславович	10.04.2005	male	187.794	 
18	KN22-6874	Колесниченко	Денис	Євгенович	26.03.2005	male	165.604	 
19	KN22-8254	Криженко	Аліна	Степанівна	29.08.2005	female	138.746	 
20	KN22-2947	Ласкевич	Андрій	Сергійович	05.09.2005	male	175.399	 
21	KN22-3856	Лукаш	Антон	Олександрович	24.11.2005	male	184.473	 
22	АБВГГІІ99-0000	Микол-Іванович Петренко-О'браєн	Іван	Петрович Іванович-О'браєн	06.06.1996	female	200.000	 
23	KN22-7534	Палій	Нікіта	Андрійович	05.05.2005	male	168.000	 
24	KN22-3825	Посмашний	Ярослав	Дмитрович	25.02.2000	male	160.715	 
25	KN22-3843	Прокопов	Владислав	Сергійович	12.12.2004	male	164.660	 
26	KN22-7819	Репешко	Інесса	Віталіївна	06.05.2005	female	185.300	 
27	KN22-8153	Рожкова	Вікторія	Андріївна	04.10.1993	female	188.304	 
28	KN22-4788	Сизоненко	Єлизавета	Олександрівна	17.02.1997	female	124.241	 
29	KN22-9115	Скиба	Андрій	Григорович	26.02.2004	male	125.909	 
30	KN22-3812	Цибань	Ольга	Юріївна	19.05.2004	female	190.298	 
31	KN22-1828	Шевченко	Данило	Валерійович	25.10.2004	male	166.847	 

[Go Home](#)

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv
Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine
Copyright © 2024 All Rights Reserved

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Рисунок 14 – Сторінка таблиці вступників з відсортованими даними по
прізвищу на URL

“http://localhost:8080/WebAppsRIVLab2/entrants?sort=surname-asc”

2 Сортування. Реалізоване сортування за будь-яким полем таблиці (номер справи, прізвище, ім'я, по-батькові тощо). При кліку на заголовок поля сортування відбувається у висхідному або спадному порядку.

Сортування обробляється з використанням функції `applySort()`, яка додає параметр сортування до URL у форматі “fieldName-order” (де “order” може бути “asc” або “desc”).

The screenshot displays the 'Student Database Viewer' application. The main section is titled 'Entrant Table' and shows a table with 3 rows of student data. The table is sorted by surname in ascending order. The search bar on the left contains the text 'pe'. The filters section shows the birthday filter set to 'dd.mm.yyyy' and the rating score filter set to 'interval'. The 'Add row' button is also visible.

#	Case Number	Surname	Name	Patronymic	Birthday	Gender	Rating Score	Operations
1	KH22-1637	Бондаренко	Ярослав	Юрійович	24.04.2005	male	199.530	
2	АБВГГІІ99-0000	Микол-Іванович Петренко-О'браєн	Іван Петро	Петрович Іванович-О'браєн	06.06.1996	female	200.000	
3	KH22-7819	Репешко	Інесса	Віталіївна	06.05.2005	female	185.300	

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок 15 – Сторінка таблиці вступників з відсортованими даними по прізвищу та пошуком значення “pe” на URL “http://localhost:8080/WebAppsRIVLab2/entrants?sort=surname-asc&search=%D1%80%D0%B5”

The screenshot displays a web application titled "Student Database Viewer". The main content area features a table titled "Entrant Table" with the following data:

#	Case Number	Surname	Name	Patronymic	Birthday	Gender	Rating Score	Operations
1	KH22-1637	Бондаренко	Ярослав	Юрійович	24.04.2005	male	199.530	
2	KH22-7819	Репешко	Інесса	Віталіївна	06.05.2005	female	185.300	

On the left side, there are three panels:

- Search:** A search bar with the text "pe" and a search button.
- Filters:**
 - Birthday:** Filtering by birthdate interval. Start: 01.01.2000, End: dd.mm.yyyy.
 - RatingScore:** Filtering by rating score interval. Start and End fields are empty.
- Add row:** A button to add a new row to the end of the table.

At the bottom, there is a "Go Home" button and a footer section with contact information and copyright notice.

Рисунок 16 – Сторінка таблиці вступників з відсортованими даними по прізвищу, пошуком значення “pe” та фільтрацією по даті народження від “01.01.2000” на URL “http://localhost:8080/WebAppsRIVLab2/entrants?sort=surname-asc&search=%D1%80%D0%B5&birthdayStart=2000-01-01”

The screenshot displays a web application titled "Student Database Viewer". The main content area shows a table titled "Entrant Table" with the following data:

#	Case Number	Surname	Name	Patronymic	Birthday	Gender	Rating Score	Operations
1	KH22-1637	Бондаренко	Ярослав	Юрійович	24.04.2005	male	199.530	

On the left side, there are three panels:

- Search:** A search bar with the text "pe" and a search button.
- Filters:**
 - Birthday:** Filtering by birthdate interval. Start: 01.01.2000, End: dd.mm.yyyy.
 - RatingScore:** Filtering by rating score interval. Start: 190, End: (empty).
- Add row:** A button to add a new row to the end of the table.

At the bottom, there is a "Go Home" button and a footer section with contact information:

Powered by: Inessa Repeshko, CS-222a, CSIT SEMIT, NTU "KhPI"

Address: National Technical University "Kharkiv Polytechnic Institute", 2 Kyrpychova, 61002 Kharkiv, Ukraine

Contact Us: +38-(057)-707-66-34, omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок 17 – Сторінка таблиці вступників з відсортованими даними по прізвищу, пошуком значення “pe”, фільтрацією по даті народження від “01.01.2000” та рейтинговому балу від “190” на URL “http://localhost:8080/WebAppsRIVLab2/entrants?sort=surname-asc&search=%D1%80%D0%B5&birthdayStart=2000-01-01&ratingScoreMin=190”

3 Додавання запису. Є кнопка для додавання нового запису про абітурієнта, яка перенаправляє користувача на сторінку створення нового абітурієнта.

4 Таблиця абітурієнтів. Таблиця реалізована з використанням Bootstrap класів для стилізації, і містить:

- стовпець з нумерацією рядків таблиці;
- стовпці з інформацією про абітурієнта (номер справи, прізвище, ім'я, по-батькові, дата народження, стать, рейтинговий бал);
- стовпець операцій, що містить кнопки для оновлення або видалення запису.

Стать абітурієнта відображається кольорами (синій для чоловіків та рожевий для жінок).

5 Функціональність кнопок та взаємодія з сервером. Кнопки для видалення та оновлення запису інтегруються з динамічними URL, які дозволяють обробляти відповідні операції.

Використовуються зображення для іконок (пошук, скидання, сортування), які завантажуються через змінну `buttonPath`.

6 JavaScript для клієнтської логіки:

- `setBirthdayLimit()` – задає мінімальні та максимальні дати для полів вибору дати народження;
- `setInputValueFromAttr()` – заповнює поля форми значеннями з URL-параметрів;
- `applySearch()`, `resetSearch()`, `applyFilterBirthday()`, `resetFilterBirthday()` – обробляють логіку пошуку і фільтрації для абітурієнтів;
- `applySort()` – забезпечує сортування записів за вказаними полями.

Ця сторінка забезпечує зручний інтерфейс для управління записами абітурієнтів, дозволяючи виконувати пошук, фільтрацію, сортування та основні операції CRUD.

4 Створення JSP сторінки форми додавання та редагування записів і сервлетів до них

Сторінка EntrantForm. Цей JSP-файл забезпечує форму для створення, оновлення або видалення запису абітурієнта (Entrant). Основні компоненти та функції цього файлу описані нижче.

Використаний функціонал для сторінки EntrantForm наступний:

1 Теги JSP та JSTL. Використання тегів JSTL для динамічного встановлення змінних, таких як `requestType`, що визначає тип запиту: створення, оновлення або видалення. Використання умовних тегів JSTL (`c:set` і `${}`) для керування заголовками сторінок і кнопками.

2 Заголовок сторінки. Динамічно змінюється в залежності від типу запиту (“Create Entrant”, “Update Entrant”, “Delete Entrant”).

3 Форма для створення, оновлення або видалення запису. Форма містить поля для введення даних абітурієнта (номер справи, прізвище, ім’я, по-батькові, дата народження, стать та рейтинговий бал). У випадку видалення поля стають тільки для читання.

Використовуються динамічні атрибути для обробки поля, яке має змінювати своє значення в залежності від типу запиту: `readonly` для полів під час видалення.

4 Обробка даних та валідація. Поля введення, такі як номер справи, ім’я, прізвище, по-батькові та дата народження, мають шаблони для валідації на стороні клієнта з використанням JavaScript.

Дата народження має обмеження на діапазон допустимих дат (від 16 до 110 років назад від дати отримання доступу до сторінки).

5 Кнопки дій. Кнопки для підтвердження та скидання форми, а також для видалення запису, які динамічно відображаються в залежності від типу запиту. Підтвердження дій передбачено через функцію JavaScript `submitConfirmation()`.

6 JavaScript для динамічної валідації. Встановлення атрибутів шаблонів для полів форми (`pattern`), динамічна валідація введення для кожного поля. Функція `setAllInputsValidity()` встановлює правила валідації для всіх полів

форми та відображення повідомлень про помилки. Присутня обробка обмежень на дату народження та рейтинговий бал із повідомленнями про помилки.

7 Скрипт для підтвердження дій. Функція `submitConfirmation()` викликає підтвердження з боку користувача перед виконанням оновлення чи видалення запису.

Динамічні аспекти сторінки `EntrantForm` наступні:

1) заголовок сторінки і форма змінюються залежно від типу запиту (`requestType`) – створення, оновлення або видалення абітурієнта;

2) кнопки відображаються чи приховуються залежно від контексту (видалення приховує кнопки “Зберегти” та “Скинути”, залишаючи тільки “Видалити”);

3) валідація забезпечує, що поля відповідають вимогам з використанням шаблонів і регулярних виразів.

Файл `EntrantForm JSP` використовується для універсальної форми абітурієнта, що дозволяє користувачу створювати, оновлювати або видаляти записи у базі даних в залежності від типу запиту.

Код сторінки `EntrantForm` представлений у додатку Г.1.

Сервлет `CreateEntrantServlet`. Даний файл реалізує функціональність створення нового запису про абітурієнта в базі даних через вебінтерфейс. Цей клас є сервлетом, що обробляє HTTP-запити для URL-адреси “/entrants/create”. Ключові аспекти реалізації сервлета `CreateEntrantServlet` наступні.

1 Обробка запитів GET. При отриманні GET-запиту цей сервлет відповідає за відображення форми для введення даних нового абітурієнта.

Обробляє GET-запити, передаючи атрибут `requestType` зі значенням “CREATE”.

Запит перенаправляється до JSP-сторінки `EntrantForm.jsp` через механізм `RequestDispatcher` для подальшого відображення.

2 Обробка запитів POST. При отриманні POST-запиту сервлет отримує дані форми через методи `request.getParameter()`, введені користувачем у формі, і створює об’єкт класу `Entrant`.

Після створення нового об'єкта Entrant на основі даних, наданих користувачем, сервлет використовує метод insert() з DAO-класу для збереження об'єкта в базі даних.

Після успішного збереження, клієнт отримує перенаправлення до сторінки оновлення даних абітурієнта через sendRedirect, де відображається запис, який щойно був створений.

3 Взаємодія з базою даних. Для взаємодії з базою даних використовується об'єкт EntrantDAO, який забезпечує операції зі створенням нового запису абітурієнта.

Метод insert() DAO-класу вставляє новий запис у базу, ідентифікуючи його за унікальним номером справи.

4 Технічні деталі. Використовується анотація @WebServlet для вказання шляху до сервлета – “/entrants/create”.

Код забезпечує належну обробку винятків і коректне перенаправлення після виконання запиту.

Використовується метод RequestDispatcher.forward() для передачі запиту і відповіді на JSP-сторінку для відображення форми.

Основні функції сервлета:

- 1) відображення форми для створення абітурієнта;
- 2) прийом і обробка введених даних через форми;
- 3) створення об'єкта абітурієнта та запис його до бази даних;
- 4) перенаправлення користувача на сторінку оновлення абітурієнта після успішного створення запису.

Код сервлета CreateEntrantServlet представлений у додатку Г.2.

Сервлет UpdateAndDeleteEntrantServlet. Даний сервлет обробляє запити на оновлення або видалення запису абітурієнта з бази даних. Він працює з обома типами запитів: GET та POST, і відповідає за обробку URL-адреси із шаблоном “/entrants/{id}/update” та “/entrants/{id}/delete”. Ключові аспекти реалізації сервлета CreateEntrantServlet наступні.

1 Обробка GET-запитів. GET-запит використовується для завантаження форми з даними абітурієнта, що підлягають редагуванню або видаленню.

Сервлет отримує ідентифікатор абітурієнта з URL-адреси, після чого викликає DAO-метод `findById()` для отримання запису абітурієнта з бази даних.

Якщо абітурієнт знайдений, сервлет передає його дані на сторінку `EntrantForm.jsp`, передаючи на сторінку з формою атрибут з інформацією про тип поточного запиту – “UPDATE” або “DELETE” залежно від URL.

Для передачі запиту та відповіді на JSP-сторінку використовується `RequestDispatcher.forward()`.

2 Обробка POST-запитів. POST-запит обробляє дані, які надходять із форми. Сервлет визначає, чи запит стосується оновлення або видалення на основі URL-адреси.

Якщо запит відповідає шаблону “/{id}/update”, сервлет створює об'єкт класу `Entrant` з оновленими даними, після чого викликає метод `update()` DAO для оновлення запису в базі даних.

Якщо запит відповідає шаблону “/{id}/delete”, сервлет видаляє запис абітурієнта з бази даних з використанням методу `delete()` DAO.

Після успішного оновлення або видалення сервлет виконує перенаправлення на відповідні сторінки з використанням `sendRedirect()`.

3 Взаємодія з базою даних. Сервлет використовує об'єкт `EntrantDAO` для виконання основних CRUD-операцій (створення, читання, оновлення, видалення) у базі даних.

- `findById()` викликається для пошуку абітурієнта за ID;
- `update()` викликається для оновлення існуючого запису;
- `delete()` використовується для видалення абітурієнта з бази.

4 Технічні деталі. Використовується анотація `@WebServlet` для вказання шляху “/entrants/*” для обробки запитів на оновлення та видалення.

У разі виникнення помилок або якщо абітурієнт не знайдений, сервлет перенаправляє на сторінку помилки 404 з використанням `sendRedirect()`.

Метод `split()` та регулярні вирази використовуються для аналізу URL-адреси та отримання ID абітурієнта.

Основні функції сервлета:

- 1) завантаження та відображення форми редагування або видалення абітурієнта на основі GET-запиту;
- 2) оновлення або видалення запису абітурієнта на основі POST-запиту;
- 3) обробка помилок та перенаправлення на відповідні сторінки після виконання операцій.

Цей сервлет забезпечує повну функціональність для оновлення та видалення даних абітурієнтів у системі, включаючи роботу з базою даних, а також підтримку перевірки ідентифікаторів і обробку помилок.

Код сервлета `UpdateAndDeleteEntrantServlet` представлений у додатку Г.3.

Далі в тексті роботи на рис. 18-25 представлені результати роботи функціоналу вебзастосунку реалізованого до завдання 4.

Засоби Java для створення динамічних вебсторінок надають можливість створювати інтерактивні вебдодатки, які можуть змінювати свій вміст на основі запитів користувачів, взаємодії з базою даних та іншими сервісами.

JSP (JavaServer Pages) – це технологія для впровадження Java-коду в HTML-сторінки, що дозволяє динамічно генерувати вміст на сервері. Вона надає можливість писати Java-код безпосередньо всередині HTML з використанням JSP тегів. А також підтримує JSTL (JavaServer Pages Standard Tag Library), яка спрощує роботу з циклами, умовами, обробкою даних та іншими завданнями на JSP-сторінках. Ще має можливість створення динамічного вмісту такого як таблиць та форм.

Приклад використання у даній роботі: JSP використовується для динамічного виведення результатів пошуку або фільтрації даних, таких як список абітурієнтів.

The screenshot shows a web browser window with the URL `localhost:8080/WebAppsRIVLab2/entrants/create`. The page title is "Student Database Viewer". The main content is a "Create Entrant" form with the following fields and values:

- Case Number: A24-4
- Surname: Репешко
- Name: Інесса
- Patronymic: Віталіївна
- Birthday: 01.01.2000
- Gender: Male (selected), Female
- Rating score: 180

Below the form are two buttons: a save icon and a refresh icon. Below these is a "Go to Table" button. The footer contains the following information:

- Powered by:** Inessa Repeshko, CS-222a, CSIT SEMIT, NTU "KhPI"
- Address:** National Technical University "Kharkiv Polytechnic Institute", 2 Kyrpychova, 61002 Kharkiv, Ukraine
- Contact Us:** +38-(057)-707-66-34, omsroot@kpi.kharkov.ua
- Copyright © 2024 All Rights Reserved

Рисунок 18 – Сторінка форми додавання запису до таблиці після натискання кнопки додавання запису на сторінці з таблицею на заповнення даних нового запису на URL “`http://localhost:8080/WebAppsRIVLab2/entrants/create`”

Сторінка `EntrantForm.jsp` для створення, оновлення або видалення запису абітурієнта – це сторінка, яка надає інтерфейс для взаємодії з даними абітурієнта. Ця сторінка дозволяє користувачеві створювати нові записи, редагувати існуючі або видаляти їх із системи. У залежності від типу запиту (CREATE, UPDATE або DELETE), інтерфейс змінюється відповідно. Дана сторінка має наступні технічні аспекти реалізації.

1 Універсальний інтерфейс для трьох операцій. Заголовок сторінки та форма змінюються в залежності від типу операції (створення, оновлення або видалення), який визначається через змінну `requestType`. Це дозволяє використовувати одну сторінку для різних дій без дублювання коду.

Student Database Viewer

Home Tables Further more

Update Entrant

Case Number: A24-4

Surname: Репешко

Name: Інесса

Patronymic: Віталіївна

Birthday: 01.01.2000

Gender: Male Female

Rating score: 180,0

Save Refresh

Go to Table

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv
Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine
Copyright © 2024 All Rights Reserved

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Рисунок 19 – Сторінка форми редагування запису після його збереження до бази даних зі сторінки додавання запису на URL
“http://localhost:8080/WebAppsRIVLab2/entrants/33/update”

Використовується JSTL для динамічної зміни елементів сторінки, таких як заголовки, кнопки та дії форми залежно від типу операції (requestTypeIsUpdate, requestTypeIsDelete).

2 Форма для вводу даних. У формі представлені поля для вводу таких даних абітурієнта, як:

- ID (внутрішній ідентифікатор запису).
- номер справи (case number).
- прізвище, ім'я та по батькові (surname, name, patronymic).
- дата народження (birthday).
- стать (gender).

– рейтинговый бал (rating score).

Student Database Viewer

Home

Tables

Further more

Entrant Table

Search

Search by case number,
surname

Type to search...

🔍

↻️

Filters

Birthday

Filtering by birthdate
interval

Start

dd.mm.yyyy

End

dd.mm.yyyy

🔍

↻️

RatingScore

Filtering by rating score
interval

Start

End

🔍

↻️

Add row

Add new row to the end of
the table

➕

#	Case Number	Surname	Name	Patronymic	Birthday	Gender	Rating Score	Operations
1	KN22-4814	Бабічев	Олексій	Вікторович	22.05.2005	male	168.119	✏️ 🗑️
2	KN22-5835	Волков	Дмитро	Олександрович	16.07.2005	male	151.200	✏️ 🗑️
3	KN22-5511	Гєєнко	Анна	Олексіївна	03.12.2005	female	150.010	✏️ 🗑️
4	KN22-9299	Голоп'яров	Микита	Сергійович	07.11.2005	male	161.480	✏️ 🗑️
5	KN22-9012	Гриценко	Максим	Георгійович	04.12.2005	male	191.941	✏️ 🗑️
6	KN22-6636	Жарий	Вадим	Віталійович	25.10.2005	male	128.349	✏️ 🗑️
7	KN22-5140	Жупанов	Андрій	Володимирович	22.06.2005	male	183.626	✏️ 🗑️
8	KN22-4896	Кльоз	Михайло	Романович	15.01.2005	male	127.286	✏️ 🗑️
9	KN22-4694	Коваль	Микита	Артемович	11.10.2005	male	139.736	✏️ 🗑️
10	KN22-5529	Ковтун	Єгор	Вячеславович	10.04.2005	male	187.794	✏️ 🗑️
11	KN22-6874	Колесниченко	Денис	Євгенович	26.03.2005	male	165.604	✏️ 🗑️
12	KN22-2947	Ласкевич	Андрій	Сергійович	05.09.2005	male	175.399	✏️ 🗑️
13	KN22-3856	Лукаш	Антон	Олександрович	24.11.2005	male	184.473	✏️ 🗑️
14	KN22-7534	Палій	Нікіта	Андрійович	05.05.2005	male	168.000	✏️ 🗑️
15	KN22-3843	Прокопов	Владислав	Сергійович	12.12.2004	male	164.660	✏️ 🗑️
16	KN22-7819	Репешко	Інесса	Віталіївна	06.05.2005	female	185.300	✏️ 🗑️
17	KN22-9115	Скиба	Андрій	Григорович	26.02.2004	male	125.909	✏️ 🗑️
18	KN22-3812	Цибань	Ольга	Юріївна	19.05.2004	female	190.298	✏️ 🗑️
19	KN22-1828	Шевченко	Данило	Валерійович	25.10.2004	male	166.847	✏️ 🗑️
20	KN22-9514	Антипенко	Катерина	Віталіївна	06.01.2005	female	168.467	✏️ 🗑️
21	KN22-2929	Багрятцева	Марина	Русланівна	01.10.2005	female	157.631	✏️ 🗑️
22	KN22-1637	Бондаренко	Ярослав	Юрійович	24.04.2005	male	199.530	✏️ 🗑️
23	KN22-6681	Булгаков	Віктор	Олександрович	17.05.2005	male	153.680	✏️ 🗑️
24	KN22-7557	Васильєв	Михайло	Олександрович	25.01.2005	male	170.867	✏️ 🗑️
25	KN22-8759	Держач	Дарина	Євгенівна	03.03.2005	female	171.711	✏️ 🗑️
26	KN22-7309	Ілюхін	Тимур	Ігорович	24.01.2005	male	125.771	✏️ 🗑️
27	KN22-8254	Криженко	Аліна	Степанівна	29.08.2005	female	138.746	✏️ 🗑️
28	KN22-3825	Посмашний	Ярослав	Дмитрович	25.02.2000	male	160.715	✏️ 🗑️
29	KN22-8153	Рожкова	Вікторія	Андріївна	04.10.1993	female	188.304	✏️ 🗑️
30	KN22-4788	Сизоненко	Елизавета	Олександрівна	17.02.1997	female	124.241	✏️ 🗑️
31	ABBGII99-0000	Микол-Іванович Петренко-О'браєн	Іван Петро	Петрович Іванович-О'браєн	06.06.1996	female	200.000	✏️ 🗑️
32	A24-4	Репешко	Інесса	Віталіївна	01.01.2000	female	180.000	✏️ 🗑️

Go Home

Powered by

Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address

National Technical University "Kharkiv
Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок 20 – Сторінка таблиці вступників з доданим записом у останньому рядку

The screenshot shows a web browser window with the URL `localhost:8080/WebAppsRIVLab2/entrants/1/update`. The page title is "Student Database Viewer". The main content area displays the "Update Entrant" form. The form has the following fields and values:

- Case Number: KH22-4814
- Surname: Бабічев
- Name: Олексій
- Patronymic: Вікторович
- Birthday: 22.05.2005
- Gender: Male (selected)
- Rating score: 168,119

Below the form are two buttons: a save icon and a refresh icon. Below these is a "Go to Table" button. The footer of the page contains the following information:

- Powered by:** Inessa Repeshko, CS-222a, CSIT SEMIT, NTU "KhPI"
- Address:** National Technical University "Kharkiv Polytechnic Institute", 2 Kyrpychova, 61002 Kharkiv, Ukraine
- Contact Us:** +38-(057)-707-66-34, omsroot@kpi.kharkov.ua
- Copyright:** © 2024 All Rights Reserved

Рисунок 21 – Сторінка форми редагування запису після натискання на сторінці з таблицею відповідної кнопки редагування даного запису на URL “`http://localhost:8080/WebAppsRIVLab2/entrants/1/update`”

Поля мають валідацію з використанням атрибутів `pattern`, `min`, `max`, а також спеціальних скриптів для перевірки правильності введених даних перед відправкою.

3 Скрипти для валідації та взаємодії з формою. Використовуються клієнтські скрипти для встановлення обмежень на введені значення, таких як шаблон для номера справи та імені, діапазони для дати народження та рейтингового балу.

Також присутня перевірка валідності введених даних перед відправкою форми. Якщо дані не відповідають вимогам, вони не будуть відправлені.

The screenshot displays a web browser window with the address bar showing 'localhost:8080/WebAppsRIVLab2/entrants/1/update'. The page title is 'Student Database Viewer'. The main content area features a light blue box titled 'Update Entrant' containing the following form elements:

- Case Number:** Text input with value 'KH22-9999'.
- Surname:** Text input with value 'Бабічева'.
- Name:** Text input with value 'Олександра'.
- Patronymic:** Text input with value 'Вікторівна'.
- Birthday:** Text input with value '10.10.2005' and a calendar icon.
- Gender:** Radio buttons for 'Male' and 'Female', with 'Female' selected.
- Rating score:** Text input with value '150'.

Below the form are two buttons: a 'Save' button (floppy disk icon) and a 'Reset' button (circular arrow icon). At the bottom of the form area is a 'Go to Table' button.

The footer of the page is divided into three sections:

- Powered by:** Inessa Repeshko, CS-222a, CSIT SEMIT, NTU "KhPI".
- Address:** National Technical University "Kharkiv Polytechnic Institute", 2 Kyrpychova, 61002 Kharkiv, Ukraine.
- Contact Us:** +38-(057)-707-66-34, omsroot@kpi.kharkov.ua.

Copyright © 2024 All Rights Reserved.

Рисунок 22 – Сторінка форми редагування запису після введення оновлених даних на URL “http://localhost:8080/WebAppsRIVLab2/entrants/1/update”

3.3 Динамічне встановлення мінімальних і максимальних значень для дати народження: від 110 до 16 років від дати звернення на сторінку.

4 Візуалізація інтерфейсу. Сторінка використовує Bootstrap 5 для стилізації інтерфейсу. Компоненти, такі як кнопки, текстові поля та форма, оформлені відповідно до сучасних стандартів UI/UX.

Використовуються стандартні Bootstrap кнопки для різних дій (збереження, скидання, видалення), що додає інтуїтивності в користувацький інтерфейс.

5 Динамічні дії. Кнопки для дій змінюються в залежності від типу операції. Кнопки Save і Reset приховані, якщо це запит на видалення, тоді як кнопка Delete активна тільки для видалення.

Student Database Viewer Home Tables Further more

Entrant Table

Search
Search by case number, surname
Type to search...
[Search] [Refresh]

Filters
Birthday
Filtering by birthdate interval
Start: dd.mm.yyyy [Calendar]
End: dd.mm.yyyy [Calendar]
[Filter] [Refresh]

RatingScore
Filtering by rating score interval
Start: [Input]
End: [Input]
[Filter] [Refresh]

Add row
Add new row to the end of the table
[Add]

#	Case Number	Surname	Name	Patronymic	Birthday	Gender	Rating Score	Operations
1	KN22-9999	Бабічева	Олександра	Вікторівна	10.10.2005	female	150.000	[Edit] [Delete]
2	KN22-5835	Волков	Дмитро	Олександрович	16.07.2005	male	151.200	[Edit] [Delete]
3	KN22-5511	Геєнко	Анна	Олексіївна	03.12.2005	female	150.010	[Edit] [Delete]
4	KN22-9299	Голоп'яров	Микита	Сергійович	07.11.2005	male	161.480	[Edit] [Delete]
5	KN22-9012	Гриценко	Максим	Георгійович	04.12.2005	male	191.941	[Edit] [Delete]
6	KN22-6636	Жарий	Вадим	Віталійович	25.10.2005	male	128.349	[Edit] [Delete]
7	KN22-5140	Жупанов	Андрій	Володимирович	22.06.2005	male	183.626	[Edit] [Delete]
8	KN22-4896	Кльоз	Михайло	Романович	15.01.2005	male	127.286	[Edit] [Delete]
9	KN22-4694	Коваль	Микита	Артемович	11.10.2005	male	139.736	[Edit] [Delete]
10	KN22-5529	Ковтун	Егор	Вячеславович	10.04.2005	male	187.794	[Edit] [Delete]
11	KN22-6874	Колесниченко	Денис	Євгенович	26.03.2005	male	165.604	[Edit] [Delete]
12	KN22-2947	Ласкевич	Андрій	Сергійович	05.09.2005	male	175.399	[Edit] [Delete]
13	KN22-3856	Лукаш	Антон	Олександрович	24.11.2005	male	184.473	[Edit] [Delete]
14	KN22-7534	Палій	Нікіта	Андрійович	05.05.2005	male	168.000	[Edit] [Delete]
15	KN22-3843	Прокопов	Владислав	Сергійович	12.12.2004	male	164.660	[Edit] [Delete]
16	KN22-7819	Репешко	Інесса	Віталіївна	06.05.2005	female	185.300	[Edit] [Delete]
17	KN22-9115	Скиба	Андрій	Григорович	26.02.2004	male	125.909	[Edit] [Delete]
18	KN22-3812	Цибань	Ольга	Юріївна	19.05.2004	female	190.298	[Edit] [Delete]
19	KN22-1828	Шевченко	Данило	Валерійович	25.10.2004	male	166.847	[Edit] [Delete]
20	KN22-9514	Антипенко	Катерина	Віталіївна	06.01.2005	female	168.467	[Edit] [Delete]
21	KN22-2929	Багрянцева	Марина	Русланівна	01.10.2005	female	157.631	[Edit] [Delete]
22	KN22-1637	Бондаренко	Ярослав	Юрійович	24.04.2005	male	199.530	[Edit] [Delete]
23	KN22-6681	Булгаков	Віктор	Олександрович	17.05.2005	male	153.680	[Edit] [Delete]
24	KN22-7557	Васильєв	Михайло	Олександрович	25.01.2005	male	170.867	[Edit] [Delete]
25	KN22-8759	Деркач	Дарина	Євгенівна	03.03.2005	female	171.711	[Edit] [Delete]
26	KN22-7309	Ілюхін	Тимур	Ігорович	24.01.2005	male	125.771	[Edit] [Delete]
27	KN22-8254	Криженко	Аліна	Степанівна	29.08.2005	female	138.746	[Edit] [Delete]
28	KN22-3825	Посмашний	Ярослав	Дмитрович	25.02.2000	male	160.715	[Edit] [Delete]
29	KN22-8153	Рожкова	Вікторія	Андріївна	04.10.1993	female	188.304	[Edit] [Delete]
30	KN22-4788	Сизоненко	Єлизавета	Олександрівна	17.02.1997	female	124.241	[Edit] [Delete]
31	АБВГГІІ99-0000	Микол-Іванович Петренко-О'брасн	Іван Петро	Петрович Іванович-О'брасн	06.06.1996	female	200.000	[Edit] [Delete]
32	A24-4	Репешко	Інесса	Віталіївна	01.01.2000	female	180.000	[Edit] [Delete]

[Go Home](#)

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv
Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine
Copyright © 2024 All Rights Reserved

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Рисунок 23 – Сторінка таблиці вступників із оновленими даними у першому рядку після збереження оновленого запису до бази даних

6 Функція підтвердження дії. Перед виконанням дії (оновлення або видалення) користувачеві пропонується підтвердити свій намір з використанням спливаючого вікна confirm.

The screenshot shows a web browser window with the URL `localhost:8080/WebAppsRIVLab2/entrants/2/delete`. The page title is "Student Database Viewer". The main content area displays a "Delete Entrant" form with the following fields:

- Case Number: KH22-5835
- Surname: Волков
- Name: Дмитро
- Patronymic: Олександрович
- Birthday: 16.07.2005
- Gender: Male (selected), Female
- Rating score: 151,2

Below the form is a trash icon button. At the bottom of the page, there is a footer with the following information:

- Powered by:** Inessa Repeshko, CS-222a, CSIT SEMIT, NTU "KhPI"
- Address:** National Technical University "Kharkiv Polytechnic Institute", 2 Kyrpychova, 61002 Kharkiv, Ukraine
- Contact Us:** +38-(057)-707-66-34, omsroot@kpi.kharkov.ua
- Copyright © 2024 All Rights Reserved

Рисунок 24 – Сторінка форми для видалення даних про запис у таблиці на URL “`http://localhost:8080/WebAppsRIVLab2/entrants/2/delete`”

7 Використання трафаретних зображень. Кнопки збереження, скидання та видалення мають відповідні зображення, які завантажуються динамічно з використанням змінної `buttonPath`.

Основні функції сторінки:

- 1) створення, оновлення або видалення запису абітурієнта в залежності від типу запиту;
- 2) валідація введених користувачем даних на стороні клієнта;
- 3) забезпечення динамічного інтерфейсу для різних CRUD операцій.

Ця сторінка є центральною точкою для CRUD-операцій над абітурієнтами і забезпечує динамічний інтерфейс для взаємодії з даними користувача, що відповідає сучасним стандартам веброзробки.

Student Database Viewer

Home Tables Further more

Entrant Table

Search
Search by case number, surname

Type to search...

Filters

Birthday
Filtering by birthdate interval

Start
dd.mm.yyyy

End
dd.mm.yyyy

RatingScore
Filtering by rating score interval

Start

End

Add row
Add new row to the end of the table



#	Case Number	Surname	Name	Patronymic	Birthday	Gender	Rating Score	Operations
1	KN22-9999	Бабічева	Олександра	Вікторівна	10.10.2005	female	150.000	 
2	KN22-5511	Геєнко	Анна	Олексіївна	03.12.2005	female	150.010	 
3	KN22-9299	Голоп'яров	Микита	Сергійович	07.11.2005	male	161.480	 
4	KN22-9012	Гриценко	Максим	Георгійович	04.12.2005	male	191.941	 
5	KN22-6636	Жарий	Вадим	Віталійович	25.10.2005	male	128.349	 
6	KN22-5140	Жупанов	Андрій	Володимирович	22.06.2005	male	183.626	 
7	KN22-4896	Кльоз	Михайло	Романович	15.01.2005	male	127.286	 
8	KN22-4694	Коваль	Микита	Артемович	11.10.2005	male	139.736	 
9	KN22-5529	Ковтун	Єгор	Вячеславович	10.04.2005	male	187.794	 
10	KN22-6874	Колесниченко	Денис	Євгенович	26.03.2005	male	165.604	 
11	KN22-2947	Ласкевич	Андрій	Сергійович	05.09.2005	male	175.399	 
12	KN22-3856	Лукаш	Антон	Олександрович	24.11.2005	male	184.473	 
13	KN22-7534	Палій	Нікіта	Андрійович	05.05.2005	male	168.000	 
14	KN22-3843	Прокопов	Владислав	Сергійович	12.12.2004	male	164.660	 
15	KN22-7819	Репешко	Інесса	Віталіївна	06.05.2005	female	185.300	 
16	KN22-9115	Скиба	Андрій	Григорович	26.02.2004	male	125.909	 
17	KN22-3812	Цибань	Ольга	Юріївна	19.05.2004	female	190.298	 
18	KN22-1828	Шевченко	Данило	Валерійович	25.10.2004	male	166.847	 
19	KN22-9514	Антипенко	Катерина	Віталіївна	06.01.2005	female	168.467	 
20	KN22-2929	Багрянцева	Марина	Русланівна	01.10.2005	female	157.631	 
21	KN22-1637	Бондаренко	Ярослав	Юрійович	24.04.2005	male	199.530	 
22	KN22-6681	Булгаков	Віктор	Олександрович	17.05.2005	male	153.680	 
23	KN22-7557	Васильєв	Михайло	Олександрович	25.01.2005	male	170.867	 
24	KN22-8759	Деркач	Дарина	Євгенівна	03.03.2005	female	171.711	 
25	KN22-7309	Ілюхін	Тимур	Ігорович	24.01.2005	male	125.771	 
26	KN22-8254	Криженко	Аліна	Степанівна	29.08.2005	female	138.746	 
27	KN22-3825	Посмашний	Ярослав	Дмитрович	25.02.2000	male	160.715	 
28	KN22-8153	Рожкова	Вікторія	Андріївна	04.10.1993	female	188.304	 
29	KN22-4788	Сизоненко	Єлизавета	Олександрівна	17.02.1997	female	124.241	 
30	АБВГГІІ99-0000	Микол-Іванович Петренко-О'брасн	Іван Петро	Петрович Іванович-О'брасн	06.06.1996	female	200.000	 
31	A24-4	Репешко	Інесса	Віталіївна	01.01.2000	female	180.000	 

[Go Home](#)

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv
Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine
Copyright © 2024 All Rights Reserved

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Рисунок 25 – Сторінка таблиці вступників після видалення другого запису з таблиці з прізвищем “Волков”

Висновки

Отримані результати підтверджують важливість вивчення технологій Java для розробки вебзастосунків з реляційними базами даних з використанням ORM Hibernate. Hibernate є одним із найпоширеніших інструментів для

об'єктно-реляційного відображення в Java-додатках. У ході виконання завдань лабораторної роботи були продемонстровані основні переваги та можливості, які Hibernate надає розробникам для ефективної взаємодії з базами даних.

Переваги використання Hibernate для Java-розробників описані нижче.

1 Автоматизація SQL-запитів. Hibernate значно спрощує процес створення SQL-запитів. Розробникам більше не потрібно писати складні SQL-запити вручну. Hibernate автоматично генерує SQL на основі об'єктної моделі, що дозволяє зосередитися на логіці додатку, а не на низькорівневих деталях взаємодії з базою даних.

2 Зручний інтерфейс для побудови динамічних SQL-запитів. Criteria API доповнює функціонал ORM Hibernate і надає розробникам зручний інтерфейс для побудови динамічних SQL-запитів. Інтерфейс дозволяє будувати запити динамічно під час виконання програми, має об'єктно-орієнтований підхід, забезпечує кращу читабельність і підтримку складних запитів, а також зменшує ризик застосування SQL-ін'єкцій.

3 Зменшення кількості коду. Завдяки використанню Hibernate, об'єктно-реляційне відображення реалізується значно простіше та ефективніше. Це зменшує кількість шаблонного коду, оскільки інструмент бере на себе рутинні завдання, такі як виконання запитів, керування транзакціями, обробка помилок тощо.

4 Кешування та оптимізація продуктивності. Hibernate підтримує двохрівневе кешування, що дозволяє зменшити кількість запитів до бази даних та значно підвищити продуктивність програми. Перший рівень кешування працює на рівні сесії, тоді як другий рівень кешу зберігає дані на рівні всього додатку, що дозволяє знизити навантаження на базу даних.

5 Автоматичне відстеження стану об'єктів. Hibernate дозволяє автоматично синхронізувати стан об'єктів у пам'яті з даними в базі. Завдяки цьому, розробники не повинні слідкувати за тим, коли і як зберігати або оновлювати дані – Hibernate відстежує зміни і самостійно виконує необхідні операції.

6 Підтримка транзакцій. Hibernate надає розширену підтримку транзакцій, що дозволяє забезпечити цілісність та атомарність операцій з базою даних. Це особливо важливо для великих додатків, де робота з базою даних має бути надійною і стійкою до збоїв.

7 Підтримка відносин між об'єктами. Hibernate надає зручні засоби для роботи з реляційними структурами бази даних, дозволяючи встановлювати зв'язки між об'єктами (один-до-одного, один-до-багатьох, багато-до-багатьох) і легко керувати ними.

Описані вище переваги використання Hibernate досягаються за рахунок наступного.

1 Мапування об'єктів Java на реляційні таблиці. Hibernate дозволяє розробникам мапувати класи Java на таблиці бази даних за допомогою анотацій або XML-конфігурацій. Це означає, що для взаємодії з базою даних використовуються об'єкти Java, що полегшує процес роботи з даними.

2 HQL (Hibernate Query Language). Hibernate надає можливість використовувати HQL, який є об'єктно-орієнтованим аналогом SQL. Це мова запитів, яка дозволяє писати запити на основі об'єктної моделі, не прив'язуючись до специфічної реалізації бази даних.

3 Criteria API має type-safe запити завдяки використанню Java Generics і має підтримку всіх основних операцій баз даних. Також він легко інтегрується з існуючими Hibernate-сесіями і має можливість побудови умов “на льоту”.

4 Кешування на першому та другому рівнях. Hibernate кешує об'єкти на рівні сесії (перший рівень) та додатку (другий рівень), що значно підвищує продуктивність за рахунок зменшення кількості запитів до бази даних.

5 Механізми lazy та eager loading. Hibernate дозволяє контролювати завантаження даних за допомогою механізмів lazy (відкладене завантаження) і eager loading (жадібне завантаження), що допомагає оптимізувати використання ресурсів та уникнути зайвого завантаження даних, які не використовуються.

6 Автоматичне управління зв'язками. Hibernate автоматично керує зв'язками між об'єктами, що дозволяє зберігати складні об'єктні структури без необхідності вручну писати SQL-запити для підтримки зв'язків між таблицями.

Звідси можемо дійти висновку, що вивчення Hibernate є корисним для розробника, оскільки цей ORM-фреймворк спрощує роботу з реляційними базами даних, дозволяючи зосередитися на об'єктно-орієнтованому програмуванні без необхідності написання SQL-запитів. Hibernate автоматизує CRUD операції, забезпечує підтримку транзакцій, а також надає засоби для роботи з кешуванням та оптимізацією запитів. Крім того, використання Criteria API дозволяє динамічно будувати запити у безпечний для типів спосіб, підвищуючи гнучкість і безпеку коду, що знижує ризик помилок та полегшує підтримку застосунків.

Практичні навички, набуті під час роботи з ORM Hibernate, є критичними для фахівців у галузі програмної інженерії, особливо при розробці динамічних вебзастосунків. Важливість вивчення цих технологій полягає у здатності створювати масштабовані та надійні вебсервіси, що відповідають сучасним стандартам індустрії. Отримані знання створюють міцний фундамент для подальшого вивчення більш складних фреймворків та підходів до розробки, що дозволяє реалізовувати масштабні та продуктивні вебрішення.

ДОДАТОК А

Налаштування проєкту з Hibernate та реалізація відображення «застосунок - база даних»

А.1 Код файлу pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>ntukhpi.csit.semit.riv</groupId>
    <artifactId>WebAppsRIVLab2</artifactId>
    <version>1.0-SNAPSHOT</version>
    <name>WebAppsRIVLab2</name>
    <packaging>war</packaging>

    <properties>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.target>17</maven.compiler.target>
        <maven.compiler.source>17</maven.compiler.source>
        <junit.version>5.10.3</junit.version>
    </properties>

    <dependencies>
        <!-- Tomcat 10.1 requires: Servlet API 6, JPA 3.1, JSTL, EL
5-->

        <dependency>
            <groupId>jakarta.servlet</groupId>
            <artifactId>jakarta.servlet-api</artifactId>
```

```

        <version>6.1.0</version>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>org.glassfish.web</groupId>
        <artifactId>jakarta.servlet.jsp.jstl</artifactId>
        <version>3.0.1</version>
    </dependency>
    <dependency>
        <groupId>jakarta.servlet.jsp.jstl</groupId>
        <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
        <version>3.0.0</version>
    </dependency>
    <dependency>
        <groupId>jakarta.el</groupId>
        <artifactId>jakarta.el-api</artifactId>
        <version>6.0.1</version>
    </dependency>

    <!-- DBMS -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.30</version>
    </dependency>

    <!-- Hibernate -->
    <dependency>
        <groupId>org.hibernate.orm</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>6.5.2.Final</version>
    </dependency>
    <dependency>
        <groupId>org.hibernate.validator</groupId>
        <artifactId>hibernate-validator</artifactId>
        <version>8.0.1.Final</version>
    </dependency>

```

```

</dependency>
<dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>4.0.5</version>
</dependency>

<!-- Java Persistence API -->
<dependency>
    <groupId>jakarta.persistence</groupId>
    <artifactId>jakarta.persistence-api</artifactId>
    <version>3.2.0</version>
</dependency>
<dependency>
    <groupId>jakarta.transaction</groupId>
    <artifactId>jakarta.transaction-api</artifactId>
    <version>2.0.1</version>
</dependency>

<!-- Unit Testing -->
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
</dependency>

<!-- Lombok -->
<dependency>
    <groupId>org.projectlombok</groupId>

```

```

        <artifactId>lombok</artifactId>
        <version>1.18.34</version>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>org.jetbrains</groupId>
        <artifactId>annotations</artifactId>
        <version>24.1.0</version>
        <scope>compile</scope>
    </dependency>

    <!-- Log4j 2 core -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>2.23.1</version>
    </dependency>
    <!-- Log4j 2 API -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-api</artifactId>
        <version>2.23.1</version>
    </dependency>
    <!-- Log4j 2 binding для SLF4J -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-slf4j-impl</artifactId>
        <version>2.23.1</version>
    </dependency>
    <dependency>
        <groupId>org.jboss.logging</groupId>
        <artifactId>jboss-logging</artifactId>
        <version>3.5.3.Final</version>
    </dependency>
    <dependency>

```

```

        <groupId>org.apache.commons</groupId>
        <artifactId>commons-lang3</artifactId>
        <version>3.12.0</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-war-plugin</artifactId>
            <version>3.3.2</version>
        </plugin>
    </plugins>
</build>
</project>

```

A.2 Код файла hibernate.cfg.xml

```

<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- JDBC Database connection settings -->
        <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver<
/property>
        <property
name="connection.url">jdbc:mysql://localhost:3307/webappsrivlab2?u
seSSL=false&serverTimezone=Europe/Kiev</property>
        <property name="connection.username">root</property>

```

```

        <property
name="connection.password">rootpassword</property>

        <!-- <property name="connection.pool_size">3</property>-->

        <!--Set UTF-8 charset-->

        <property
name="hibernate.connection.CharSet">utf8</property>

        <property
name="hibernate.connection.characterEncoding">utf8</property>

        <property
name="hibernate.connection.useUnicode">true</property>

        <!-- Hibernate properties -->

        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</prope
rty>

        <property name="hibernate.show_sql">true</property>
        <property name="show_sql">true</property>
        <property name="hibernate.format_sql">true</property>
        <property name="hibernate.highlight_sql">true</property>
        <property
name="hibernate.use_sql_comments">false</property>

        <!-- Enable automatic schema generation -->
        <property name="hibernate.hbm2ddl.auto">create</property>

        <!-- HBM mappings or annotated classes -->
        <mapping
class="ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.
Entrant"/>

        <mapping
class="ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.student.
Student"/>

        <mapping
class="ntukhpi.csit.semit.riv.webappsrivlab2.model.util.hibernate.
HibernateUtil"/>

```

```

    </session-factory>
</hibernate-configuration>

```

A.3 Код класу HibernateUtil.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.model.util.hibernate;

import jakarta.validation.Validation;
import jakarta.validation.Validator;
import jakarta.validation.ValidatorFactory;
import org.hibernate.SessionFactory;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistry;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;
import
org.hibernate.validator.messageinterpolation.ParameterMessageInter
polator;

/**
 * HibernateUtil is a utility class that provides methods for
managing the Hibernate {@link SessionFactory}
 * and the {@link Validator} for entity validation. This class is
responsible for initializing,
 * configuring, and shutting down these resources.
 *
 * It supports:
 * - SessionFactory initialization and management for Hibernate
sessions.
 * - Validator initialization and management for validating
entities using JSR-303/JSR-380 Bean Validation.
 * - Graceful shutdown of Hibernate resources and the Validator
factory.

```

```

*
* This class is designed to be thread-safe and ensures that the
resources are initialized lazily when needed.
*
* @author Inessa Repeshko CS-222a
*/
public class HibernateUtil {
    private static StandardServiceRegistry registry;
    private static SessionFactory sessionFactory;
    private static ValidatorFactory validatorFactory;
    private static Validator validator;

    static {
        buildSessionFactory();
        buildValidator();
    }

    private static void buildSessionFactory() {
        registry = new StandardServiceRegistryBuilder()
            .configure("hibernate.cfg.xml")
            .build();

        try {
            MetadataSources sources = new
MetadataSources(registry);
            Metadata metadata =
sources.getMetadataBuilder().build();
            sessionFactory =
metadata.getSessionFactoryBuilder().build();
        } catch (Exception e) {
            System.err.println("Error during sessionFactory
initialization: " + e.getMessage());
            e.printStackTrace(System.out);
            StandardServiceRegistryBuilder.destroy(registry);
        }
    }
}

```



```

private static void buildValidator() {
    try {
        validatorFactory = Validation.byDefaultProvider()
            .configure()
            .messageInterpolator(new
ParameterMessageInterpolator())
            .buildValidatorFactory();
        validator = validatorFactory.getValidator();
    } catch (Exception e) {
        throw new RuntimeException("Failed to initialize
Validator.", e);
    }
}

public static SessionFactory getSessionFactory() {
    if (sessionFactory == null) {
        buildSessionFactory();
    }

    return sessionFactory;
}

public static Validator getValidator() {
    if (validator == null) {
        buildValidator();
    }

    return validator;
}

public static void shutdown() {
    if (sessionFactory != null) {
        sessionFactory.close();
    }
}

```

```
    if (registry != null) {  
        StandardServiceRegistryBuilder.destroy(registry);  
    }  
  
    if (validatorFactory != null) {  
        validatorFactory.close();  
    }  
}  
}
```

ДОДАТОК Б

Реалізація Entity-класів, таблиць у базі даних та CRUD операцій

Б.1 Код класу Entrant.java

```
package
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant;

import jakarta.persistence.Table;
import jakarta.validation.constraints.*;
import lombok.*;

import jakarta.persistence.*;
import org.hibernate.annotations.*;
import org.hibernate.type.descriptor.java.BooleanJavaType;

import java.lang.reflect.Field;
import java.lang.reflect.Modifier;
import java.time.LocalDate;
import java.time.Period;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.*;

/**
 * Entrant is an entity class representing a record in the
 * "entrants" table.
 * It stores detailed information about an entrant, including
 * personal data and academic rating.
 * The class also includes validation constraints and custom
 * methods for data formatting.
 *
 * Key features:
```

```

* - Validation for fields such as case number, name, surname,
patronymic, birthday, gender, and rating score.
* - Logical deletion using the `isDeleted` field.
* - Filters for database queries, such as filtering out logically
deleted entrants.
*
* The class includes constructors for creating Entrant objects and
methods to manipulate or validate data fields.
* It also contains utility methods for converting dates and
generating formatted output.
*
* Validation rules:
* - The case number must follow a specific format.
* - The name, surname, and patronymic must only contain letters
from the Ukrainian alphabet.
* - The birthday must be in the past and the entrant must be
between 16 and 110 years old.
* - The rating score must be between 120.01 and 200.00.
*
* This class uses Hibernate annotations for ORM mapping and
Jakarta validation for input validation.
*
* @see Table
* @see Filter
* @see NotBlank
* @see Size
* @see Pattern
* @see Past
* @see DecimalMin
* @see DecimalMax
* @see FilterDef
* @see Data
* @see AllArgsConstructor
* @see NoArgsConstructor
*
* @author Inessa Repeshko CS-222a

```

```

*/

@Entity
@Table(name = "entrants")
@FilterDef(name = "deletedEntrantFilter", parameters =
@ParamDef(name = "isDeleted", type = BooleanJavaType.class))
@Filters({
    @Filter(name = "deletedEntrantFilter", condition =
"is_deleted = :isDeleted")
})
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Entrant {
    private static final String REGEX_CASE_NUMBER =
"^((?=.{5,20}$) [A-ЩЮЯҐЄІİ]{1,10} \\d{2} [\\-] \\d{1,4}$)";
    private static final String REGEX_NAMES =
"^((?=.{1,50}$) [A-ЩЮЯҐЄІİ] (?:[A-ЯҐЄІİă-яґєіі']*[A-ЯҐЄІİă-яґєіі])?(?:[
\\-] [A-ЩЮЯҐЄІİ] (?:[A-ЯҐЄІİă-яґєіі']*[A-ЯҐЄІİă-яґєіі])?)*$";
    private static final DateTimeFormatter dateFormatter =
DateTimeFormatter.ofPattern("dd.MM.yyyy");

    @Column(name = "id")
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "case_number", nullable = false, unique = true,
length = 20)
    @Check(constraints = "REGEXP_LIKE(case_number,
'^[A-ЩЮЯҐЄІİ]{1,10} [0-9]{2}-[0-9]{1,4}$', 'c') = 1")
    @NotBlank(message = "The case number must not be blank.")
    @Pattern(regexp = REGEX_CASE_NUMBER, message = "The case number
should be in the format 'LLLLLLLLLLLLNN-NNNN'.")

```

```

private String caseNumber;

@Column(name = "surname", nullable = false, length = 50)
@Check(constraints = "REGEXP_LIKE(surname, '" + REGEX_NAMES +
"', 'c') = 1")
@NotBlank(message = "The surname must not be blank.")
@Size(min = 1, max = 50, message = "The surname size should be
between 1 to 50.")
@Pattern(regexp = REGEX_NAMES, message = "The surname can
contain only letters of the Ukrainian alphabet and can be complex
using a space or dash.")
private String surname;

@Column(name = "name", nullable = false, length = 50)
@Check(constraints = "REGEXP_LIKE(name, '" + REGEX_NAMES + "',
'c') = 1")
@NotBlank(message = "The name must not be blank.")
@Size(min = 1, max = 50, message = "The name size should be
between 1 to 50.")
@Pattern(regexp = REGEX_NAMES, message = "The name can contain
only letters of the Ukrainian alphabet and can be complex using a
space or dash.")
private String name;

@Column(name = "patronymic", nullable = false, length = 50)
@Check(constraints = "REGEXP_LIKE(patronymic, '" + REGEX_NAMES
+ "', 'c') = 1")
@NotBlank(message = "The patronymic must not be blank.")
@Size(min = 1, max = 50, message = "The patronymic size should
be between 1 to 50.")
@Pattern(regexp = REGEX_NAMES, message = "The patronymic can
contain only letters of the Ukrainian alphabet and can be complex
using a space or dash.")

```

```

private String patronymic;

@Column(name = "birthday", nullable = false)
@Check(constraints = "birthday BETWEEN '1914-01-01' AND
'2008-01-01'")
@NotNull(message = "The birthday must not be null.")
@Past(message = "The birthday must be in the past.")
private LocalDate birthday;

@Column(name = "gender", nullable = false)
@ColumnDefault(value = "1")
@NotNull(message = "The gender must not be null.")
private Boolean gender = Boolean.FALSE;

@Column(name = "rating_score", nullable = false)
@Check(constraints = "rating_score > 120.00 AND rating_score <=
200.00")
@NotNull(message = "The rating score must not be null.")
@DecimalMin(value = "120.001", inclusive = true)
@DecimalMax(value = "200.000", inclusive = true)
private Double ratingScore;

@Column(name = "is_deleted", nullable = false)
@ColumnDefault(value = "0")
@NotNull(message = "The deleted state must not be null.")
private Boolean isDeleted = Boolean.FALSE;

public Entrant(String caseNumber, String surname, String name,
String patronymic, String birthday, boolean gender, double
ratingScore) {
    setCaseNumber(caseNumber);

```

```

        setSurname(surname);
        setName(name);
        setPatronymic(patronymic);
        setBirthday(birthday);
        setGender(gender);
        setRatingScore(ratingScore);
    }

    public Entrant(String id, String caseNumber, String surname,
String name, String patronymic, String birthday, String gender,
String ratingScore) {
        try {
            setId(Long.parseLong(id));
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("The id should not
be empty and should be long.");
        }

        setStringFields(caseNumber, surname, name, patronymic,
birthday, gender, ratingScore);
    }

    public Entrant(String caseNumber, String surname, String name,
String patronymic, String birthday, String gender, String
ratingScore) {
        setStringFields(caseNumber, surname, name, patronymic,
birthday, gender, ratingScore);
    }

    public void setId(Long id) {
        this.id = id;
    }

    public void setCaseNumber(String caseNumber) {
        this.caseNumber = validateCaseNumber(caseNumber);
    }

```



```

    public void setSurname(String surname) {
        this.surname = validateName(surname);
    }

    public void setName(String name) {
        this.name = validateName(name);
    }

    public void setPatronymic(String patronymic) {
        this.patronymic = patronymic.trim().equals("-") ?
patronymic.trim() : validateName(patronymic);
    }

    public void setBirthday(String birthday) {
        this.birthday = validateBirthday(birthday);
    }

    public void setGender(boolean gender) {
        this.gender = Boolean.valueOf(gender);
    }

    public void setRatingScore(double ratingScore) {
        this.ratingScore = validateRatingScore(ratingScore);
    }

    private void setStringFields(String caseNumber, String surname,
String name, String patronymic, String birthday, String gender,
String ratingScore) {
        setCaseNumber(caseNumber);
        setSurname(surname);
        setName(name);
        setPatronymic(patronymic);
        setBirthday(changeDateFormatFromDefaulttoUA(birthday));

        if (gender != null && !gender.isBlank()) {

```

```

        setGender(Boolean.parseBoolean(gender));
    } else {
        throw new IllegalArgumentException("The gender should
not be empty and should be boolean.");
    }

    try {
        setRatingScore(Double.parseDouble(ratingScore));
    } catch (NumberFormatException | NullPointerException e) {
        throw new IllegalArgumentException("The rating score
should not be empty and should be double.");
    }
}

private String validateCaseNumber(String caseNumber) {
    if (caseNumber == null) {
        throw new IllegalArgumentException("The case number
cannot be empty.");
    }

    caseNumber = caseNumber.trim();

    if (caseNumber.isEmpty() ||
!caseNumber.matches(Regex.CASE_NUMBER)) {
        throw new IllegalArgumentException("The case number
should be in the format 'LLLLLLLLLLNN-NNNN'.");
    }

    return caseNumber;
}

private String validateName(String name) {
    if (name == null || name.trim().isEmpty()) {
        throw new IllegalArgumentException("Parts of the name
cannot be empty.");
    }
}

```

```

        name = name.trim();

        String[] words = name.split("\\s+");
        StringBuilder formattedName = new StringBuilder();

        for (String word : words) {
            String[] subWords = word.split("-");

            for (String subWord : subWords) {
                if
(!subWord.matches("^[A-Щa-щЮюЯяҐґЄєІіЇї](?:[A-Яa-яҐґЄєІіЇї']+)?$"))
{
                    throw new IllegalArgumentException("Parts of
the name can contain only letters of the Ukrainian alphabet and
can be complex using a space or dash.");
                }

                if (!subWord.isEmpty()) {
formattedName.append(Character.toUpperCase(subWord.charAt(0)))

.append(subWord.substring(1).toLowerCase())

                .append("-");
                }
            }

            formattedName.setLength(formattedName.length() - 1);
            formattedName.append(" ");
        }

        return formattedName.toString().trim();
    }

    private LocalDate validateBirthday(String birthday) {
        if (birthday == null) {

```

```

        throw new IllegalArgumentException("The birthday cannot
be empty.");
    }

    LocalDate birthDate;

    try {
        birthDate = LocalDate.parse(birthday, dateFormatter);
    } catch (DateTimeParseException e) {
        throw new IllegalArgumentException("The birthday should
be in the format 'dd.MM.yyyy'.");
    }

    int age = Period.between(birthDate,
LocalDate.now()).getYears();

    if (age < 16 || age > 110) {
        throw new IllegalArgumentException("The age should be
between 16 and 110 years old.");
    }

    return birthDate;
}

private Double validateRatingScore(double ratingScore) {
    if (ratingScore < 120.001 || ratingScore > 200.000) {
        throw new IllegalArgumentException("The rating score
should be between 120.01 and 200.00.");
    }

    return ratingScore;
}

private static String changeDateFormatFromDefaulttoUA(String
inputDate) {
    if (inputDate == null) {

```

```

        throw new IllegalArgumentException("The birthday cannot
be empty.");
    }

    DateTimeFormatter inputFormatter =
DateTimeFormatter.ofPattern("yyyy-MM-dd");
    DateTimeFormatter outputFormatter =
DateTimeFormatter.ofPattern("dd.MM.yyyy");

    LocalDate date = LocalDate.parse(inputDate,
inputFormatter);
    return date.format(outputFormatter);
}

public String getGenderAsString() {
    return getGender() ? "male" : "female";
}

public String getBirthdayAsUAString() {
    return getBirthday().format(DateTimeFormatter.ofPattern("dd
MMM yyyy", new Locale("uk")));
}

public String getBirthdayAsUADate() {
    return
getBirthday().format(DateTimeFormatter.ofPattern("dd.MM.yyyy"));
}

public String getBirthdayAsDefaultDate() {
    return
getBirthday().format(DateTimeFormatter.ofPattern("yyyy-MM-dd"));
}

public String getRatingScoreAsString() {
    return String.format("%.3f", getRatingScore());
}

```

```

    public Map<String, Object> getAllFields() {
        Map<String, Object> fields = new HashMap<>();
        Field[] declaredFields =
this.getClass().getDeclaredFields();

        for (Field field : declaredFields) {
            if (!Modifier.isStatic(field.getModifiers()) &&
!Modifier.isFinal(field.getModifiers())) {
                field.setAccessible(true);

                try {
                    fields.put(field.getName(), field.get(this));
                } catch (IllegalAccessException e) {
                    e.printStackTrace();
                }
            }
        }

        return fields;
    }

    public Map<String, Object> getUniqueFields() {
        Map<String, Object> fields = new HashMap<>();

        for (Map.Entry<String, Object> entry :
getAllFields().entrySet()) {
            if (entry.getKey().equals("id")
                || entry.getKey().equals("caseNumber")
                || entry.getKey().equals("isDeleted")) {
                fields.put(entry.getKey(), entry.getValue());
            }
        }

        return fields;
    }

```

```

public List<String[]> getFieldNamesAsFormattedString() {
    List<String[]> fieldNames = new ArrayList<>();
    fieldNames.add(new String[]{"caseNumber", "Case Number"});
    fieldNames.add(new String[]{"surname", "Surname"});
    fieldNames.add(new String[]{"name", "Name"});
    fieldNames.add(new String[]{"patronymic", "Patronymic"});
    fieldNames.add(new String[]{"birthday", "Birthday"});
    fieldNames.add(new String[]{"gender", "Gender"});
    fieldNames.add(new String[]{"ratingScore", "Rating
Score"});

    return fieldNames.stream().toList();
}
}

```

Б.2 Код класу EntrantList.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant;

import lombok.NoArgsConstructor;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * EntrantList is a singleton class that extends {@link ArrayList}
 * to store a predefined list of {@link Entrant} objects.
 * It provides a method for initializing the list with sample data
 * and retrieving the list of entrants.
 *
 * Key features:

```

```

* - Singleton pattern implementation to ensure only one instance
of EntrantList exists.
* - Predefined list of entrants that can be used for testing or
demo purposes.
* - The list includes both valid and logically deleted entrants,
as well as examples with complex name formats.
*
* This class is primarily used to simulate a database or
repository of entrants.
*
* @see Entrant
* @see ArrayList
* @author Inessa Repeshko CS-222a
*/

@NoArgsConstructor
public class EntrantList extends ArrayList<Entrant> {
    private static final long serialVersionUID = 1L;
    private static EntrantList instance;

    public static EntrantList getInstance() {
        if (instance == null) {
            instance = new EntrantList();

            List<String[]> entrantData = Arrays.asList(
                new String[]{"KH22-4814", "Бабічев", "Олексій",
"Вікторович", "22.05.2005", "TRUE", "168.119"},
                new String[]{"KH22-5835", "Волков", "Дмитро",
"Олександрович", "16.07.2005", "TRUE", "151.200"},
                new String[]{"KH22-5511", "Геєнко", "Анна",
"Олексіївна", "03.12.2005", "FALSE", "150.010"},
                new String[]{"KH22-9299", "Голоп'яров",
"Микита", "Сергійович", "07.11.2005", "TRUE", "161.480"},
                new String[]{"KH22-9012", "Гриценко", "Максим",
"Георгійович", "04.12.2005", "TRUE", "191.941"},

```



```

        new String[]{"KH22-6636", "Жарий", "Вадим",
"Віталійович", "25.10.2005", "TRUE", "128.349"},

        new String[]{"KH22-5140", "Жупанов", "Андрій",
"Володимирович", "22.06.2005", "TRUE", "183.626"},

        new String[]{"KH22-4896", "Кльоз", "Михайло",
"Романович", "15.01.2005", "TRUE", "127.286"},

        new String[]{"KH22-4694", "Коваль", "Микита",
"Артемович", "11.10.2005", "TRUE", "139.736"},

        new String[]{"KH22-5529", "Ковтун", "Єгор",
"Вячеславович", "10.04.2005", "TRUE", "187.794"},

        new String[]{"KH22-6874", "Колесниченко",
"Денис", "Євгенович", "26.03.2005", "TRUE", "165.604"},

        new String[]{"KH22-2947", "Ласкевич", "Андрій",
"Сергійович", "05.09.2005", "TRUE", "175.399"},

        new String[]{"KH22-3856", "Лукаш", "Антон",
"Олександрович", "24.11.2005", "TRUE", "184.473"},

        new String[]{"KH22-7534", "Палій", "Нікіта",
"Андрійович", "05.05.2005", "TRUE", "168.000"},

        new String[]{"KH22-3843", "Прокопов",
"Владислав", "Сергійович", "12.12.2004", "TRUE", "164.660"},

        new String[]{"KH22-7819", "Репешко", "Інесса",
"Віталіївна", "06.05.2005", "FALSE", "185.300"},

        new String[]{"KH22-9115", "Скиба", "Андрій",
"Григорович", "26.02.2004", "TRUE", "125.909"},

        new String[]{"KH22-3812", "Цибань", "Ольга",
"Юріївна", "19.05.2004", "FALSE", "190.298"},

        new String[]{"KH22-1828", "Шевченко", "Данило",
"Валерійович", "25.10.2004", "TRUE", "166.847"},

        new String[]{"KH22-9514", "Антипенко",
"Катерина", "Віталіївна", "06.01.2005", "FALSE", "168.467"},

        new String[]{"KH22-2929", "Багрянцева",
"Марина", "Русланівна", "01.10.2005", "FALSE", "157.631"},

        new String[]{"KH22-1637", "Бондаренко",
"Ярослав", "Юрійович", "24.04.2005", "TRUE", "199.530"},

        new String[]{"KH22-6681", "Булгаков", "Віктор",
"Олександрович", "17.05.2005", "TRUE", "153.680"},

```

```

        new String[]{"KH22-7557", "Васильєв",
"Михайло", "Олександрович", "25.01.2005", "TRUE", "170.867"},
        new String[]{"KH22-8759", "Деркач", "Дарина",
"Євгенівна", "03.03.2005", "FALSE", "171.711"},
        new String[]{"KH22-7309", "Ілюхін", "Тимур",
"Ігорович", "24.01.2005", "TRUE", "125.771"},
        new String[]{"KH22-8254", "Криженко", "Аліна",
"Степанівна", "29.08.2005", "FALSE", "138.746"},
        new String[]{"KH22-3825", "Посмашний",
"Ярослав", "Дмитрович", "25.02.2000", "TRUE", "160.715"},
        new String[]{"KH22-8153", "Рожкова",
"Вікторія", "Андріївна", "04.10.1993", "FALSE", "188.304"},
        new String[]{"KH22-4788", "Сизоненко",
"Єлизавета", "Олександрівна", "17.02.1997", "FALSE", "124.241"}
    );

    entrantData.forEach(data -> instance.add(new
Entrant(data[0], data[1], data[2], data[3], data[4],
Boolean.parseBoolean(data[5]), Double.parseDouble(data[6]))));
//
    entrantData.forEach(data -> instance.add(new
Entrant(data[0], data[1], data[2], data[3], data[4], data[5],
data[6])));

    Entrant entrant1 = new Entrant("KH22-9998", "Марчук",
"Регіна", "Віталіївна", "06.06.1996",
Boolean.parseBoolean("FALSE"), Double.parseDouble("200.000"));
    entrant1.setIsDeleted(true);
    instance.add(entrant1);

    Entrant entrant2 = new Entrant("АБВГГІІ99-0000",
"Микол-Іванович Петренко-О'Браєн", "Іван Петро", "Петрович
Іванович-О'Браєн", "06.06.1996", Boolean.parseBoolean("FALSE"),
Double.parseDouble("200.000"));
    instance.add(entrant2);
}

```

```

        return instance;
    }

    public List<Entrant> getEntrants() {
        return new ArrayList<>(getInstance());
    }
}

```

Б.3 Код класу Student.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.student;

import jakarta.persistence.ForeignKey;
import jakarta.persistence.Table;
import lombok.*;

import jakarta.persistence.*;
import jakarta.validation.constraints.Pattern;

import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
;
import org.hibernate.annotations.*;
import org.hibernate.type.descriptor.java.BooleanJavaType;

/**
 * The Student class represents an entity of a student who has
 * transitioned from an entrant.
 * It includes details such as the funding type, scholarship
 * status, and corporate email.
 *

```

```

* The class is mapped to the "students" table in the database and
uses various JPA and Hibernate annotations for
* defining relationships, constraints, and filters.
*
* Key attributes:
* - {@link #entrant}: The one-to-one relationship with the {@link
Entrant} entity, representing the entrant information.
* - {@link #fundingType}: The funding type (contract or budget),
stored as a string in the database.
* - {@link #scholarshipStatus}: The scholarship status (ordinary,
increased, or none), stored as a string in the database.
* - {@link #corporateEmail}: The corporate email with a validation
pattern ensuring it follows a specific format.
* - {@link #deleted}: A flag for logical deletion of the student
record.
*
* Filters:
* - A filter is applied to allow the exclusion of logically
deleted students from queries.
*
* Relationships:
* - One-to-one relationship with the {@link Entrant} entity.
* - A foreign key constraint is defined for the entrant ID, which
sets the foreign key to null on deletion or update.
*
* @see Entrant
* @see FundingType
* @see ScholarshipStatus
* @see FundingTypeConverter
* @see ScholarshipStatusConverter
*
* @author Inessa Repeshko CS-222a
*/
@Entity
@Table(name = "students")

```

```

@FilterDef(name = "deletedStudentFilter", parameters =
@ParamDef(name = "deletedStudentFilter", type =
BooleanJavaType.class))
@Filters({
    @Filter(name = "deletedStudentFilter", condition =
":deleted = 0")
})
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;

    @OneToOne
    @JoinColumn(name = "entrant_id",
        referencedColumnName = "id",
        foreignKey = @ForeignKey(
            foreignKeyDefinition = "FOREIGN KEY
(entrant_id) REFERENCES entrant(id) ON DELETE SET NULL ON UPDATE
SET NULL"))
    private Entrant entrant;

    @Column(name = "funding_type",
        nullable = false)
    @Enumerated(EnumType.STRING)
    @Convert(converter = FundingTypeConverter.class)
    private FundingType fundingType;

    @Column(name = "scholarship_status",
        nullable = false)
    @Enumerated(EnumType.STRING)
    @Convert(converter = ScholarshipStatusConverter.class)
    @ColumnDefault(value = "'NONE'")

```

```

private ScholarshipStatus scholarshipStatus;

@Column(name = "corporate_email",
        nullable = false,
        unique = true,
        length = 123)
@Pattern(regex =
"^[a-z]{1,100}@[a-z]{1,10}\\\\.khpi\\.edu\\.ua$", message = "The
corporate email should match the format
'name.surname@faculty.khpi.edu.ua'")
private String corporateEmail;

@Column(name = "deleted", nullable = false)
@ColumnDefault(value = "0")
private Boolean deleted = Boolean.FALSE;
}

```

Б.4 Код переліку FundingType.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.student;

/**
 * Enum FundingType represents the type of funding for a student.
 * It can either be "contract" (контракт) or "budget" (бюджет).
 *
 * This enum provides a mapping between the enum constants and
 * their corresponding Ukrainian string values.
 * It also includes a method to retrieve an enum constant based on
 * the string value.
 *
 * Available types:
 * - CONTRACT: Students who are funded on a contract basis.
 * - BUDGET: Students who are funded by the state.

```

```

*
* Key methods:
* - {@link #getValue()} returns the Ukrainian string
representation of the funding type.
* - {@link #fromValue(String)} retrieves the corresponding enum
constant based on the provided string value.
*
* @author Inessa Repeshko CS-222a
*/
public enum FundingType {
    CONTRACT("контракт"),
    BUDGET("бюджет");

    private final String value;

    FundingType(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }

    public static FundingType fromValue(String value) {
        for (FundingType type : FundingType.values()) {
            if (type.getValue().equals(value)) {
                return type;
            }
        }

        throw new IllegalArgumentException("Unknown FundingType
value: " + value);
    }
}

```

Б.5 Код класу FundingTypeConverter.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.student;

import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;

/**
 * FundingTypeConverter is a JPA attribute converter that converts
 * between the {@link FundingType} enum
 * and its string representation in the database.
 *
 * This converter automatically applies to any entity field of type
 * {@link FundingType} and ensures
 * that the enum is stored in the database as a string, and
 * converted back to the enum when reading
 * from the database.
 *
 * Key methods:
 * - {@link #convertToDatabaseColumn(FundingType)}: Converts a
 * {@link FundingType} enum to its string value for storage in the
 * database.
 * - {@link #convertToEntityAttribute(String)}: Converts a string
 * value from the database back to the corresponding {@link
 * FundingType} enum.
 *
 * This converter is marked with {@link Converter} annotation and
 * `autoApply=true`, which means it will automatically apply
 * to all entity fields of type {@link FundingType}.
 *
 * @see FundingType
 * @see AttributeConverter
 *
 * @author Inessa Repeshko CS-222a

```



```

*/

@Converter(autoApply = true)
public class FundingTypeConverter implements
AttributeConverter<FundingType, String> {

    @Override
    public String convertToDatabaseColumn(FundingType fundingType)
{
        if (fundingType == null) {
            return null;
        }

        return fundingType.getValue();
    }

    @Override
    public FundingType convertToEntityAttribute(String dbData) {
        if (dbData == null || dbData.isEmpty()) {
            return null;
        }

        for (FundingType type : FundingType.values()) {
            if (type.getValue().equals(dbData)) {
                return type;
            }
        }

        throw new IllegalArgumentException("Unknown FundingType
value: " + dbData);
    }
}

```

Б.6 Код переліку ScholarshipStatus.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.student;

/**
 * Enum ScholarshipStatus represents the scholarship status of a
 * student.
 * It can be one of the following:
 * - ORDINARY: Represents a student receiving a standard
 * scholarship ("звичайна").
 * - INCREASED: Represents a student receiving an increased
 * scholarship ("підвищена").
 * - NONE: Represents a student who does not receive a scholarship
 * ("відсутня").
 *
 * This enum provides a mapping between the enum constants and
 * their corresponding Ukrainian string values.
 * It also includes a method to retrieve an enum constant based on
 * the string value.
 *
 * Key methods:
 * - {@link #getValue()} returns the Ukrainian string
 * representation of the scholarship status.
 * - {@link #fromValue(String)} retrieves the corresponding enum
 * constant based on the provided string value.
 *
 * @author Inessa Repeshko CS-222a
 */
public enum ScholarshipStatus {
    ORDINARY("звичайна"),
    INCREASED("підвищена"),
    NONE("відсутня");

    private final String value;

```

```

    ScholarshipStatus(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }

    public static ScholarshipStatus fromValue(String value) {
        for (ScholarshipStatus status : ScholarshipStatus.values())
        {
            if (status.getValue().equals(value)) {
                return status;
            }
        }

        throw new IllegalArgumentException("Unknown
ScholarshipStatus value: " + value);
    }
}

```

Б.7 Код класу ScholarshipStatusConverter.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.student;

import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;

/**
 * ScholarshipStatusConverter is a JPA attribute converter that
 * converts between the {@link ScholarshipStatus} enum
 * and its string representation in the database.

```

```

*
* This converter automatically applies to any entity field of type
* {@link ScholarshipStatus} and ensures
* that the enum is stored in the database as a string, and
* converted back to the enum when reading
* from the database.
*
* Key methods:
* - {@link #convertToDatabaseColumn(ScholarshipStatus)}: Converts
* a {@link ScholarshipStatus} enum to its string value for storage
* in the database.
* - {@link #convertToEntityAttribute(String)}: Converts a string
* value from the database back to the corresponding {@link
* ScholarshipStatus} enum.
*
* This converter is marked with the {@link Converter} annotation
* and `autoApply=true`, which means it will automatically apply
* to all entity fields of type {@link ScholarshipStatus}.
*
* @see ScholarshipStatus
* @see AttributeConverter
*
* @author Inessa Repeshko CS-222a
*/
@Converter(autoApply = true)
public class ScholarshipStatusConverter implements
AttributeConverter<ScholarshipStatus, String> {

    @Override
    public String convertToDatabaseColumn(ScholarshipStatus
scholarshipStatus) {
        if (scholarshipStatus == null) {
            return null;
        }

        return scholarshipStatus.getValue();
    }
}

```

```

    }

    @Override
    public ScholarshipStatus convertToEntityAttribute(String
dbData) {
        if (dbData == null || dbData.isEmpty()) {
            return null;
        }

        for (ScholarshipStatus status : ScholarshipStatus.values())
{
            if (status.getValue().equals(dbData)) {
                return status;
            }
        }

        throw new IllegalArgumentException("Unknown
ScholarshipStatus value: " + dbData);
    }
}

```

Б.8 Код інтерфейсу GenericDAO.java

```

package ntukhpi.csit.semit.riv.webappsrivlab2.model.DAO;

import jakarta.persistence.TypedQuery;
import jakarta.persistence.criteria.*;
import jakarta.validation.ConstraintViolation;
import jakarta.validation.Validator;
import
ntukhpi.csit.semit.riv.webappsrivlab2.model.util.hibernate.Hiberna
teUtil;
import org.hibernate.Session;
import org.hibernate.SessionFactory;

```

```

import org.hibernate.Transaction;

import java.time.LocalDate;
import java.util.*;
import org.apache.commons.lang3.Range;

/**
 * GenericDAO is a generic interface for performing common database
 * operations using Hibernate.
 * It defines methods for CRUD (Create, Read, Update, Delete)
 * operations, filtering, sorting, and validation.
 * This interface provides default implementations for these
 * operations, allowing reuse across multiple entity types.
 *
 * @param <E> the entity type that this DAO will manage
 *
 * The following operations are provided:
 * - Fetching all entities with optional filtering by the
 *   `isDeleted` flag.
 * - Fetching filtered and sorted lists of entities based on
 *   specified criteria.
 * - Searching for entities by key or by multiple keys.
 * - Inserting, updating, and deleting entities.
 * - Handling duplicates and validation of entity constraints.
 * - Truncating a table.
 *
 * The class relies on a {@link SessionFactory} and a {@link
 * Validator} for Hibernate and validation operations.
 *
 * @author Inessa Repeshko CS-222a
 */

public interface GenericDAO<E> {
    final SessionFactory sessionFactory =
HibernateUtil.getSessionFactory();
    final Validator validator = HibernateUtil.getValidator();

```



```

        try (Session session = sessionFactory.openSession()) {
            if (!includeDeleted) {

session.enableFilter("deletedEntrantFilter").setParameter("isDelet
ed", false);

            }

            CriteriaBuilder cb = session.getCriteriaBuilder();
            CriteriaQuery<E> cq = cb.createQuery(entityClass);
            Root<E> rootEntry = cq.from(entityClass);

            List<Predicate> predicates = new ArrayList<>();

            if (filters != null && !filters.isEmpty()) {
                for (Map.Entry<String, Object> filter :
filters.entrySet()) {

                    String fieldName = filter.getKey();
                    Object fieldValue = filter.getValue();

                    /* String filters */
                    if (fieldValue instanceof String) {
                        String stringFilter = (String) fieldValue;

                        /* Check if there are wildcards for LIKE */
                        if (stringFilter.contains("%")) {

predicates.add(cb.like(cb.upper(rootEntry.get(fieldName)),
cb.upper(cb.literal(stringFilter))));

                        } else {

predicates.add(cb.like(cb.upper(rootEntry.get(fieldName)),
cb.upper(cb.literal("%" + stringFilter + "%"))));

                        }
                    }
                }
            }
        }

```



```

        /* Date filters */
        else if (fieldValue instanceof Range &&
fieldName.equals("birthday")) {
            Range<LocalDate> dateRange =
(Range<LocalDate>) fieldValue;
            LocalDate startDate =
dateRange.getMinimum();
            LocalDate endDate = dateRange.getMaximum();

predicates.add(cb.between(rootEntry.get(fieldName), startDate,
endDate));
        }

        /* Numeric filters */
        else if (fieldValue instanceof Range &&
fieldName.equals("ratingScore")) {
            Range<Double> range = (Range<Double>)
fieldValue;
            Double min = range.getMinimum();
            Double max = range.getMaximum();

predicates.add(cb.between(rootEntry.get(fieldName), min, max));
        }

        /* Boolean filters */
        else if (fieldValue instanceof Boolean) {
            boolean value = (Boolean) fieldValue;

predicates.add(cb.equal(rootEntry.get(fieldName), value));
        }
    }

    if (search != null && !search.isEmpty()) {

```

```

        List<Predicate> searchPredicates = new
ArrayList<>();

        for (Map.Entry<String, String> searchEntry :
search.entrySet()) {
            String fieldName = searchEntry.getKey();
            String searchValue = searchEntry.getValue();

            if (searchValue != null &&
!searchValue.isEmpty()) {

searchPredicates.add(cb.like(cb.upper(rootEntry.get(fieldName)),
cb.upper(cb.literal("%" + searchValue + "%"))));
            }
        }

        /* Combine all search predicates with OR condition
*/

        if (!searchPredicates.isEmpty()) {

predicates.add(cb.or(searchPredicates.toArray(Predicate[]::new)));
        }

        if (!predicates.isEmpty()) {
            cq.where(predicates.toArray(Predicate[]::new));
        }

        List<Order> orderList = new ArrayList<>();

        if (sortOrders != null && !sortOrders.isEmpty()) {
            for (Map.Entry<String, String> sortRule :
sortOrders.entrySet()) {
                String fieldName = sortRule.getKey();
                String direction =
sortRule.getValue().toUpperCase();

```

```

        if ("ASC".equals(direction)) {

orderList.add(cb.asc(rootEntry.get(fieldName)));

        } else if ("DESC".equals(direction)) {

orderList.add(cb.desc(rootEntry.get(fieldName)));

        }

    }

    if (!orderList.isEmpty()) {
        cq.orderBy(orderList);
    }

    TypedQuery<E> allQuery = session.createQuery(cq);
    return allQuery.getResultList();
}

}

default E findById(Class<E> entityClass, Long id) {
    if (id == null || id <= 0) {
        throw new IllegalArgumentException("Unable to find a
record by an invalid identifier.");
    }

    try (Session session = sessionFactory.openSession()) {

session.enableFilter("deletedEntrantFilter").setParameter("isDelet
ed", false);

        E entity = session.get(entityClass, id);

        if (entity == null) {
            throw new IllegalArgumentException("Could not find
a record by the passed ID.");
        }
    }
}

```

```

        return entity;
    }
}

default List<E> findByKey(Class<E> entityClass, String
fieldName, Object fieldValue) {
    try (Session session = sessionFactory.openSession()) {
session.enableFilter("deletedEntrantFilter").setParameter("isDelet
ed", false);

        CriteriaBuilder cb = session.getCriteriaBuilder();
        CriteriaQuery<E> cq = cb.createQuery(entityClass);
        Root<E> root = cq.from(entityClass);
        cq.select(root).where(cb.equal(root.get(fieldName),
fieldName));

        TypedQuery<E> query = session.createQuery(cq);
        List<E> results = query.getResultList();
        return results;
    }
}

default List<E> findByKeyWhereBetween(Class<E> entityClass,
String columnName, Object fromValue, Object toValue) {
    try (Session session = sessionFactory.openSession()) {
        CriteriaBuilder cb = session.getCriteriaBuilder();
        CriteriaQuery<E> cq = cb.createQuery(entityClass);
        Root<E> root = cq.from(entityClass);

        cq.select(root).where(cb.between(root.get(columnName),
(Comparable) fromValue, (Comparable) toValue));

        TypedQuery<E> query = session.createQuery(cq);
        List<E> results = query.getResultList();
        return results;
    }
}

```

```

    }

    }

    default List<E> findByMultipleKeys(Class<E> entityClass,
Map<String, Object> parameters, boolean includeDeleted) {
        try (Session session = sessionFactory.openSession()) {
            if (!includeDeleted) {

session.enableFilter("deletedEntrantFilter").setParameter("isDelet
ed", false);

            }

            CriteriaBuilder cb = session.getCriteriaBuilder();
            CriteriaQuery<E> cq = cb.createQuery(entityClass);
            Root<E> root = cq.from(entityClass);

            List<Predicate> predicatesList = new ArrayList<>();

            for (Map.Entry<String, Object> entry :
parameters.entrySet()) {

predicatesList.add(cb.equal(root.get(entry.getKey()),
entry.getValue()));

            }

            Predicate[] predicatesArray =
predicatesList.toArray(Predicate[]::new);
            cq.select(root).where(predicatesArray);

            TypedQuery<E> query = session.createQuery(cq);
            List<E> results = query.getResultList();
            return results;

        }

    }

```

```

        default void validateEntity(E entity) throws
IllegalArgumentException {
            Set<ConstraintViolation<E>> violationSet =
validator.validate(entity);

            if (!violationSet.isEmpty()) {
                StringBuilder violationMessages = new
StringBuilder("Invalid entity values:");

                for (ConstraintViolation<E> violation : violationSet) {
violationMessages.append("\n").append(violation.getMessage());
                }

                throw new
IllegalArgumentException(violationMessages.toString());
            }
        }

        default E insert(E entityToSave) {
            try {
                validateEntity(entityToSave);
                hasDuplicate(entityToSave);
            } catch (IllegalArgumentException e) {
                System.err.println(e.getMessage());
                return null;
            }

            Transaction transaction = null;

            try (Session session = sessionFactory.openSession()) {
                transaction = session.beginTransaction();
                E entity = session.merge(entityToSave);
                transaction.commit();
                return entity;
            } catch (Exception e) {

```

```

        if (transaction != null && transaction.isActive()) {
            transaction.rollback();
        }

        e.printStackTrace();
        return null;
    }
}

default List<E> insertAll(List<E> entitiesToSave) {
    List<E> duplicates = new ArrayList<>();

    for (E entity : entitiesToSave) {
        try {
            validateEntity(entity);
        } catch (IllegalArgumentException e) {
            System.err.println(e.getMessage());
            return null;
        }

        try {
            hasDuplicate(entity);
        } catch (IllegalArgumentException e) {
            duplicates.add(entity);
        }
    }

    duplicates.forEach(entitiesToSave::remove);

    if (entitiesToSave.isEmpty()) {
        return null;
    }

    Transaction transaction = null;
    Set<E> savedEntities = new HashSet<>();

```

```

    try (Session session = sessionFactory.openSession()) {
        transaction = session.beginTransaction();

        for (E entity : entitiesToSave) {
            E savedEntity = session.merge(entity);
            savedEntities.add(savedEntity);
        }

        if (savedEntities.size() == entitiesToSave.size()) {
            transaction.commit();
            return savedEntities.stream().toList();
        } else {
            if (transaction != null && transaction.isActive())
            {
                transaction.rollback();
            }

            return null;
        }
    } catch (Exception e) {
        e.printStackTrace();

        if (transaction != null && transaction.isActive()) {
            transaction.rollback();
        }

        return null;
    }
}

default E update(E entityToUpdate) {
    try {
        validateEntity(entityToUpdate);

        var getIdMethod = getEntityClass().getMethod("getId");
    }
}

```



```

        Long id = (Long) getIdMethod.invoke(entityToUpdate);

        if (id == null) {
            throw new IllegalArgumentException("Cannot update a
record without an id.");
        }

        if (findById(getEntityClass(), id) == null) {
            throw new IllegalArgumentException("No record
exists to update.");
        }
    } catch (IllegalArgumentException e) {
        System.err.println(e.getMessage());
        return null;
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }

    Transaction transaction = null;

    try (Session session = sessionFactory.openSession()) {
        transaction = session.beginTransaction();
        E entity = session.merge(entityToUpdate);
        transaction.commit();
        return entity;
    } catch (Exception e) {
        if (transaction != null && transaction.isActive()) {
            transaction.rollback();
        }

        e.printStackTrace();
        return null;
    }
}

```

```

    default boolean delete(E entityToDelete, Long id) {
        try {
            if (findById(getEntityClass(), id) == null) {
                throw new IllegalArgumentException("No record
exists to delete.");
            }
        } catch (IllegalArgumentException e) {
            e.printStackTrace();
            return false;
        }

        Transaction transaction = null;

        try (Session session = sessionFactory.openSession()) {
            transaction = session.beginTransaction();
            session.remove(entityToDelete);
            transaction.commit();
            return true;
        } catch (Exception e) {
            e.printStackTrace();

            if (transaction != null && transaction.isActive()) {
                transaction.rollback();
            }

            return false;
        }
    }

    default boolean delete(E entityToDelete) {
        try {
            var getIdMethod = getEntityClass().getMethod("getId");
            Long idToDelete = (Long)
getIdMethod.invoke(entityToDelete);

```

```

        if (idToDelete == null) {
            throw new IllegalArgumentException("Cannot delete a
record without an id.");
        }

        E entrantExists = findById(getEntityClass(),
idToDelete);

        if (entrantExists == null) {
            throw new IllegalArgumentException("No record
exists for deletion.");
        }

        var setIsDeletedMethod =
getEntityClass().getMethod("setIsDeleted", Boolean.class);
        setIsDeletedMethod.invoke(entityToDelete,
Boolean.TRUE);

        return update(entityToDelete) != null;
    } catch (IllegalArgumentException e) {
        System.err.println(e.getMessage());
        return false;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}

default boolean truncateTable(String tableName) {
    Transaction transaction = null;

    try (Session session = sessionFactory.openSession()) {
        transaction = session.beginTransaction();
        String sql = "TRUNCATE TABLE " + tableName;
        session.createNativeQuery(sql,
getEntityClass()).executeUpdate();
    }
}

```

```

        transaction.commit();
        return true;
    } catch (Exception e) {
        if (transaction != null && transaction.isActive()) {
            transaction.rollback();
        }

        e.printStackTrace();
        return false;
    }
}
}

```

Б.9 Код класу EntrantDAO.java

```

package ntukhpi.csit.semit.riv.webappsrivlab2.model.DAO;

import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
;

import java.time.LocalDate;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * EntrantDAO is a data access object (DAO) that provides methods
 * for interacting with the
 * `Entrant` entity in the database. It implements the {@link
 * GenericDAO} interface and

```

```

* provides specific methods for finding, filtering, sorting, and
validating entrant.
*
* This class includes methods to:
* - Find entrant by various fields (e.g., case number, surname,
birthday, gender, etc.).
* - Retrieve all entrant or a filtered and sorted list of entrant.
* - Check for duplicate entrant records based on unique fields.
*
* @see GenericDAO
* @see Entrant
*
* @author Inessa Repeshko CS-222a
*/
public class EntrantDAO implements GenericDAO<Entrant> {
    public Entrant findById(Long id) {
        return findById(getEntityClass(), id);
    }

    public List<Entrant> findByCaseNumber(String caseNumber) {
        return findByKey(Entrant.class, "caseNumber", caseNumber);
    }

    public List<Entrant> findBySurname(String surname) {
        return findByKey(Entrant.class, "surname", surname);
    }

    public List<Entrant> findBySurnameNamePatronymic(String
surname, String name, String patronymic) {
        Map<String, Object> parameters = new HashMap<>();
        parameters.put("surname", surname);
        parameters.put("name", name);
        parameters.put("patronymic", patronymic);
        return findByMultipleKeys(Entrant.class, parameters,
false);
    }
}

```

```

public List<Entrant> findByBirthday(LocalDate birthday) {
    return findByKey(Entrant.class, "birthday", birthday);
}

public List<Entrant> findByGender(Boolean gender) {
    return findByKey(Entrant.class, "gender", gender);
}

public List<Entrant> findByRatingScore(Double ratingScore) {
    return findByKey(Entrant.class, "ratingScore",
ratingScore);
}

public List<Entrant> getAllList() {
    return getAllList(Entrant.class, false);
}

public List<Entrant> getFilteredAndSortedList(Map<String,
Object> filters,
                                                Map<String,
String> sortOrders,
                                                Map<String,
String> search) {
    return getFilteredAndSortedEntrantList(Entrant.class,
false, filters, sortOrders, search);
}

@Override
public Class<Entrant> getEntityClass() {
    return Entrant.class;
}

@Override
public void hasDuplicate(Entrant entityToCheck) throws
IllegalArgumentException {

```

```

        Map<String, Object> uniqueFields =
entityToCheck.getUniqueFields();

        for (Map.Entry<String, Object> field :
uniqueFields.entrySet()) {
            if (field.getValue() == null) {
                uniqueFields.remove(field.getKey());
            }
        }

        List<Entrant> results = findByMultipleKeys(Entrant.class,
uniqueFields, true);

        if (!results.isEmpty()) {
            StringBuilder message = new StringBuilder("Duplicate
record found.");

            for (Map.Entry<String, Object> entry :
uniqueFields.entrySet()) {
                String fieldName = entry.getKey();
                Object fieldValue = entry.getValue();

                for (Entrant duplicate : results) {
                    Map<String, Object> duplicateFields =
duplicate.getUniqueFields();
                    Object duplicateFieldValue =
duplicateFields.get(fieldName);

                    if (fieldValue != null &&
fieldValue.equals(duplicateFieldValue)) {
                        message.append("\nThe record with the field
'").append(fieldName)
                                .append("' having value
'").append(fieldValue).append("' already exists.");
                    }
                }
            }
        }
    }
}

```

```

    }

    throw new IllegalArgumentException(message.toString());
}
}
}

```

Б.10 Код тесту EntrantTest.java

```

import static org.junit.jupiter.api.Assertions.*;

import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;

/**
 * EntrantTest is a JUnit 5 test class designed to test the
 * functionality of the {@link Entrant} class.
 * It verifies the correctness of data validation, formatting, and
 * business logic within the Entrant class.
 *
 * Key test cases include:
 * - {@link #testEntrantCreation()}: Verifies the creation of an
 * entrant using a no-arguments constructor and setters.

```



```

* - {@link #testEntrantStringRepresentation()}: Ensures the string
representation of the entrant object is formatted correctly using
Lombok's `toString()` method.
* - {@link #testInvalidCaseNumber()}: Tests that an invalid case
number throws an appropriate exception.
* - {@link #testNameValidation()}: Tests name validation rules,
including valid and invalid name formats.
* - {@link #testGenderAsString()}: Ensures the correct string
representation for gender ("male" or "female").
* - {@link #testBirthdayValidation()}: Verifies the birthday
validation logic, ensuring correct date format and valid age
ranges.
* - {@link #testRatingScoreValidation()}: Tests rating score
validation, ensuring it falls within the valid range.
*
* It also contains a nested test class, {@link
NameValidationTest}, which focuses on testing name format
validation based on specific rules.
*
* @see Entrant
* @see Test
* @see Nested
* @see BeforeEach
* @see DisplayName
*
* @author Inessa Repeshko CS-222a
*/
public class EntrantTest {
    private Entrant entrant;

    @BeforeEach
    public void setUp() {
        entrant = new Entrant();
        entrant.setCaseNumber("KH22-1234");
        entrant.setSurname("Іванов о-конел а");
        entrant.setName("Іван петро");
    }
}

```

```

        entrant.setPatronymic("ІВАНОВИЧ");
        entrant.setBirthday("01.01.2005");
        entrant.setGender(true);
        entrant.setRatingScore(187.98);
    }

    @Test
    @DisplayName("Test Entrant creation with no args constructor and setters")
    public void testEntrantCreation() {
        assertEquals("KH22-1234", entrant.getCaseNumber());
        assertEquals("ІВАНОВ О-КОНЕЛ А", entrant.getSurname());
        assertEquals("ІВАН ПЕТРО", entrant.getName());
        assertEquals("ІВАНОВИЧ", entrant.getPatronymic());
        assertEquals(LocalDate.of(2005, 1, 1),
entrant.getBirthday());
        assertEquals(Boolean.TRUE, entrant.getGender());
        assertEquals(187.98, entrant.getRatingScore());
    }

    @Test
    @DisplayName("Test Entrant string representation with Lombok toString() method")
    public void testEntrantStringRepresentation() {
        assertEquals("Entrant(id=null, caseNumber=KH22-1234, surname=ІВАНОВ О-КОНЕЛ А, name=ІВАН ПЕТРО, patronymic=ІВАНОВИЧ, birthday=2005-01-01, gender=true, ratingScore=187.98, isDeleted=false)",
            entrant.toString());
    }

    @Test
    @DisplayName("Test invalid case number format")
    public void testInvalidCaseNumber() {
        IllegalArgumentException exception =
assertThrows(IllegalArgumentException.class, () -> {

```

```

        entrant.setCaseNumber("НевірнийНомер");
    });

    assertEquals("The case number should be in the format
'LLLLLLLLLLNN-NNNN'.", exception.getMessage());
}

@Test
@DisplayName("Test valid and invalid names")
public void testNameValidation() {
    // Valid names
    entrant.setName("Марія");
    assertEquals("Марія", entrant.getName());

    entrant.setName("Олексій-Петро");
    assertEquals("Олексій-Петро", entrant.getName());

    entrant.setName("анна марія");
    assertEquals("Анна Марія", entrant.getName());

    // Invalid name with digits
    IllegalArgumentException exception =
assertThrows(IllegalArgumentException.class, () ->
entrant.setName("Марія123"));

    assertEquals("Parts of the name can contain only letters of
the Ukrainian alphabet and can be complex using a space or dash.",
exception.getMessage());

    // Invalid name with special symbols
    exception = assertThrows(IllegalArgumentException.class, ()
-> entrant.setName("%Марія!"));

    assertEquals("Parts of the name can contain only letters of
the Ukrainian alphabet and can be complex using a space or dash.",
exception.getMessage());
}

@Nested

```

```

public class NameValidationTest {
    private static final String REGEX_NAMES =
"^(?=.{1,50}$)[А-ЩЮЯҐЄІЇ](?:[А-ЯҐЄІЇа-яґєії']*[А-ЯҐЄІЇа-яґєії])?(?:[
- ] [А-ЩЮЯҐЄІЇ](?:[А-ЯҐЄІЇа-яґєії']*[А-ЯҐЄІЇа-яґєії])?)*$";

    private boolean isValidName(String name) {
        return name.matches(REGEX_NAMES);
    }

    @Test
    @DisplayName("Test for matching one word without
apostrophes")
    public void testSingleWordValid() {
        assertTrue(isValidName("Іван"));
        assertTrue(isValidName("Олександр"));
        assertTrue(isValidName("Коцур-Щербина"));
        assertTrue(isValidName("Анна Марія"));
    }

    @Test
    @DisplayName("Test for an incorrect word with invalid
characters")
    public void testSingleWordInvalid() {
        assertFalse(isValidName("М'яч"));
        assertFalse(isValidName("Бом"));
    }

    @Test
    @DisplayName("Test for matching one word with an
apostrophe")
    public void testSingleWordWithApostrophe() {
        assertTrue(isValidName("О'Браєн"));
        assertFalse(isValidName("'О'Браєн"));
        assertFalse(isValidName("О'Браєн'"));
    }
}

```

```

@Test
@DisplayName("Test for correct combinations of several
words")
public void testMultipleWordsValid() {
    assertTrue(isValidName("Іван-Петро"));
    assertTrue(isValidName("Петров'ян Іван-О'Браєн"));
}

@Test
@DisplayName("Test for incorrect combinations of several
words")
public void testMultipleWordsInvalid() {
    assertFalse(isValidName("Іван--Петро"));
    assertFalse(isValidName("Іван Петро-"));
    assertFalse(isValidName("'Петро Іван"));
}

@Test
@DisplayName("Test for mixed case")
public void testMixedCaseValid() {
    assertTrue(isValidName("ІВАН"));
    assertTrue(isValidName("ОлЕкСАНДР"));
    assertFalse(isValidName("анна"));
}

@Test
@DisplayName("Test for incorrect characters from other
alphabets")
public void testInvalidAlphabets() {
    assertFalse(isValidName("John"));
    assertFalse(isValidName("ЫВАН"));
}

@Test
@DisplayName("Test for the correct number of words with
hyphens and apostrophes")

```

```

        public void testValidMultipleWordsWithHyphenAndApostrophe()
        {
            assertTrue(isValidName("Микола-Іванович
Петренко-О'Браєн"));
            assertTrue(isValidName("Анна-Марія Іван-О'Браєн"));
        }

        @Test
        @DisplayName("Test to check the minimum and maximum number
of characters")
        public void testMinMaxCharacters() {
            assertTrue(isValidName("A"));
            assertTrue(isValidName("Олександр-Миколайович"));

            assertFalse(isValidName("Олександр-Миколайович-Петро-Іванович-Сидо
ров-Олександр-Миколайович-Петро-Іванович-Сидоров
Олександр-Миколайович-Петро-Іванович-Сидоров-Олександр-Миколайович
-Петро-Іванович-Сидоров"));
        }
    }

    @Test
    @DisplayName("Test birthday validation for valid and invalid
dates")
    public void testBirthdayValidation() {
        // Valid birthday
        entrant.setBirthday("01.01.1990");
        assertEquals(LocalDate.of(1990, 1, 1),
entrant.getBirthday());

        // Invalid birthday (too young)
        IllegalArgumentException exception =
assertThrows(IllegalArgumentException.class, () -> {
            entrant.setBirthday("01.01.2010");
        });
    }

```

```

        assertEquals("The age should be between 16 and 110 years
old.", exception.getMessage());

        // Invalid birthday (too old)
        exception = assertThrows(IllegalArgumentException.class, ()
-> entrant.setBirthday("01.01.1900"));
        assertEquals("The age should be between 16 and 110 years
old.", exception.getMessage());

        // Invalid format
        exception = assertThrows(IllegalArgumentException.class, ()
-> entrant.setBirthday("1990/01/01"));
        assertEquals("The birthday should be in the format
'dd.MM.yyyy'.", exception.getMessage());
    }

    @Test
    @DisplayName("Test gender representation as string")
    public void testGenderAsString() {
        assertEquals("male", entrant.getGenderAsString());

        entrant.setGender(false);
        assertEquals("female", entrant.getGenderAsString());
    }

    @Test
    @DisplayName("Test rating score validation")
    public void testRatingScoreValidation() {
        // Valid rating score
        entrant.setRatingScore(150.50);
        assertEquals(150.50, entrant.getRatingScore());

        // Invalid rating score (too low)
        IllegalArgumentException exception =
assertThrows(IllegalArgumentException.class, () ->
entrant.setRatingScore(119.99));

```

```

        assertEquals("The rating score should be between 120.01 and
200.00.", exception.getMessage());

        // Invalid rating score (too high)
        exception = assertThrows(IllegalArgumentException.class, ()
-> entrant.setRatingScore(200.01));
        assertEquals("The rating score should be between 120.01 and
200.00.", exception.getMessage());
    }
}

```

Б.11 Код тесту EntrantListTest.java

```

import static org.junit.jupiter.api.Assertions.*;

import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
;
import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
List;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;

/**
 * EntrantListTest is a JUnit 5 test class for testing the
functionality of the {@link EntrantList} class.
 * It verifies that the list of entrants is created correctly, the
data is valid, and specific fields

```



```

* meet the defined constraints such as unique case numbers, valid
gender, and birthday format.
*
* Key test cases:
* - {@link #testEntrantListCreation()}: Ensures the entrant list
is created and populated correctly.
* - {@link #testEntrantDataInList()}: Verifies the first entrant's
data matches the expected values.
* - {@link #testGenderValidationInList()}: Ensures all entrants
have a valid gender value.
* - {@link #testRatingScoreRange()}: Validates that the rating
score for each entrant is within the allowed range.
* - {@link #testEntrantListStringRepresentation()}: Confirms the
string representation of the entrant list matches the expected
format.
* - {@link #testUniqueCaseNumbers()}: Ensures all entrants have
unique case numbers.
* - {@link #testBirthdayDateFormat()}: Verifies that the birthday
format of all entrants is correct.
*
* This class ensures the integrity of the entrant data and the
correct behavior of the {@link EntrantList} class.
*
* @see EntrantList
* @see Entrant
* @see Test
* @see BeforeEach
* @see DisplayName
*
* @author Inessa Repeshko CS-222a
*/
public class EntrantListTest {
    private List<Entrant> entrantList;
    private static final int listLength =
EntrantList.getInstance().size();

```

```

@BeforeEach
public void setUp() {
    entrantList = EntrantList.getInstance().getEntrants();
}

@Test
@DisplayName("Create Entrant List")
public void testEntrantListCreation() {
    assertFalse(entrantList.isEmpty(), "The list should not be empty.");
    assertEquals(listLength, entrantList.size(), "The list must contain the " + listLength + " of the elements.");
}

@Test
@DisplayName("Check Entrant Data in the List")
public void testEntrantDataInList() {
    Entrant firstEntrant = entrantList.get(0);
    assertEquals("KH22-4814", firstEntrant.getCaseNumber());
    assertEquals("Бабічев", firstEntrant.getSurname());
    assertEquals("Олексій", firstEntrant.getName());
    assertEquals("Вікторович", firstEntrant.getPatronymic());
    assertEquals(LocalDate.of(2005, 5, 22),
firstEntrant.getBirthDay());
    assertEquals(true, firstEntrant.getGender());
    assertEquals(168.11, firstEntrant.getRatingScore(), 0.01);
}

@Test
@DisplayName("Validate Gender Data in Entrant List")
public void testGenderValidationInList() {
    entrantList.forEach(entrant -> {
        assertNotNull(entrant.getGender(), "Gender cannot be null.");
    });
}

```

```

        assertTrue(entrant.getGender() == Boolean.TRUE ||
entrant.getGender() == Boolean.FALSE, "Gender must be either
'true' or 'false'.");
    });
}

@Test
@DisplayName("Validate Rating Score Range in Entrant List")
public void testRatingScoreRange() {
    entrantList.forEach(entrant -> {
        assertTrue(entrant.getRatingScore() >= 120.01 &&
entrant.getRatingScore() <= 200.00,
            "The rating score should be between 120.01 and
200.00.");
    });
}

@Test
@DisplayName("Get Entrant List string representation")
public void testEntrantListStringRepresentation() {
    String expected = ""

        Entrant(id=null, caseNumber=KH22-4814,
surname=Бабічев, name=Олексій, patronymic=Вікторович,
birthday=2005-05-22, gender=true, ratingScore=168.119,
isDeleted=false)

        Entrant(id=null, caseNumber=KH22-5835,
surname=Волков, name=Дмитро, patronymic=Олександрович,
birthday=2005-07-16, gender=true, ratingScore=151.2,
isDeleted=false)

        Entrant(id=null, caseNumber=KH22-5511,
surname=Геєнко, name=Анна, patronymic=Олексіївна,
birthday=2005-12-03, gender=false, ratingScore=150.01,
isDeleted=false)

        Entrant(id=null, caseNumber=KH22-9299,
surname=Голоп'яров, name=Микита, patronymic=Сергійович,

```

```
birthday=2005-11-07, gender=true, ratingScore=161.48,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-9012,  
surname=Гриценко, name=Максим, patronymic=Георгійович,  
birthday=2005-12-04, gender=true, ratingScore=191.941,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-6636,  
surname=Жарий, name=Вадим, patronymic=Віталійович,  
birthday=2005-10-25, gender=true, ratingScore=128.349,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-5140,  
surname=Жупанов, name=Андрій, patronymic=Володимирович,  
birthday=2005-06-22, gender=true, ratingScore=183.626,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-4896,  
surname=Кльоз, name=Михайло, patronymic=Романович,  
birthday=2005-01-15, gender=true, ratingScore=127.286,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-4694,  
surname=Коваль, name=Микита, patronymic=Артемович,  
birthday=2005-10-11, gender=true, ratingScore=139.736,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-5529,  
surname=Ковтун, name=Єгор, patronymic=Вячеславович,  
birthday=2005-04-10, gender=true, ratingScore=187.794,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-6874,  
surname=Колесниченко, name=Денис, patronymic=Євгенович,  
birthday=2005-03-26, gender=true, ratingScore=165.604,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-2947,  
surname=Ласкевич, name=Андрій, patronymic=Сергійович,  
birthday=2005-09-05, gender=true, ratingScore=175.399,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-3856,  
surname=Лукаш, name=Антон, patronymic=Олександрович,
```

```
birthday=2005-11-24, gender=true, ratingScore=184.473,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-7534,  
surname=Палій, name=Нікіта, patronymic=Андрійович,  
birthday=2005-05-05, gender=true, ratingScore=168.0,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-3843,  
surname=Прокопов, name=Владислав, patronymic=Сергійович,  
birthday=2004-12-12, gender=true, ratingScore=164.66,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-7819,  
surname=Репешко, name=Інесса, patronymic=Віталіївна,  
birthday=2005-05-06, gender=false, ratingScore=185.3,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-9115,  
surname=Скиба, name=Андрій, patronymic=Григорович,  
birthday=2004-02-26, gender=true, ratingScore=125.909,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-3812,  
surname=Цибань, name=Ольга, patronymic=Юріївна,  
birthday=2004-05-19, gender=false, ratingScore=190.298,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-1828,  
surname=Шевченко, name=Данило, patronymic=Валерійович,  
birthday=2004-10-25, gender=true, ratingScore=166.847,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-9514,  
surname=Антипенко, name=Катерина, patronymic=Віталіївна,  
birthday=2005-01-06, gender=false, ratingScore=168.467,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-2929,  
surname=Багрянцева, name=Марина, patronymic=Русланівна,  
birthday=2005-10-01, gender=false, ratingScore=157.631,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-1637,  
surname=Бондаренко, name=Ярослав, patronymic=Юрійович,
```

```
birthday=2005-04-24, gender=true, ratingScore=199.53,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-6681,  
surname=Булгаков, name=Віктор, patronymic=Олександрович,  
birthday=2005-05-17, gender=true, ratingScore=153.68,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-7557,  
surname=Васильєв, name=Михайло, patronymic=Олександрович,  
birthday=2005-01-25, gender=true, ratingScore=170.867,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-8759,  
surname=Деркач, name=Дарина, patronymic=Євгенівна,  
birthday=2005-03-03, gender=false, ratingScore=171.711,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-7309,  
surname=Ілюхін, name=Тимур, patronymic=Ігорович,  
birthday=2005-01-24, gender=true, ratingScore=125.771,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-8254,  
surname=Криженко, name=Аліна, patronymic=Степанівна,  
birthday=2005-08-29, gender=false, ratingScore=138.746,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-3825,  
surname=Посмашний, name=Ярослав, patronymic=Дмитрович,  
birthday=2000-02-25, gender=true, ratingScore=160.715,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-8153,  
surname=Рожкова, name=Вікторія, patronymic=Андріївна,  
birthday=1993-10-04, gender=false, ratingScore=188.304,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-4788,  
surname=Сизоненко, name=Єлизавета, patronymic=Олександрівна,  
birthday=1997-02-17, gender=false, ratingScore=124.241,  
isDeleted=false)  
    Entrant(id=null, caseNumber=KH22-9998,  
surname=Марчук, name=Регіна, patronymic=Віталіївна,
```

```

birthday=1996-06-06, gender=false, ratingScore=200.0,
isDeleted=true)

        Entrant(id=null, caseNumber=АБВГТІІ99-0000,
surname=Микол-Іванович Петренко-О'браєн, name=Іван Петро,
patronymic=Петрович Іванович-О'браєн, birthday=1996-06-06,
gender=false, ratingScore=200.0, isDeleted=false)
        """;

        StringBuilder actual = new StringBuilder();
        entrantList.forEach(e -> actual.append(e).append("\n"));
        assertEquals(expected, actual.toString());
    }

    @Test
    @DisplayName("Check uniqueness of case numbers in Entrant
List")
    public void testUniqueCaseNumbers() {
        long uniqueCaseNumbersCount = entrantList.stream()
            .map(Entrant::getCaseNumber)
            .distinct()
            .count();

        assertEquals(entrantList.size(), uniqueCaseNumbersCount,
"Case numbers must be unique..");
    }

    @Test
    @DisplayName("Check birthday date format in Entrant List")
    public void testBirthdayDateFormat() {
        entrantList.forEach(entrant -> {
            assertDoesNotThrow(() ->
LocalDate.parse(entrant.getBirthdayAsUADate(),
DateTimeFormatter.ofPattern("dd.MM.yyyy")),
"The birthday should be in the format
'dd.MM.yyyy'."));
        });
    }
}

```

```
}
```

Б.12 Код тесту HibernateAndDAOTest.java

```
import lombok.extern.log4j.Log4j2;
import ntukhpi.csit.semit.riv.webappsrivlab2.model.DAO.EntrantDAO;
import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
;
import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
List;
import org.apache.commons.lang3.Range;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.junit.jupiter.api.*;

import static org.junit.jupiter.api.Assertions.*;

import
ntukhpi.csit.semit.riv.webappsrivlab2.model.util.hibernate.Hiberna
teUtil;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * HibernateAndDAOTest is a JUnit 5 test class designed to test
 * Hibernate interactions and CRUD operations
```



```

* with the {@link EntrantDAO} and {@link Entrant} entity. It
focuses on:
* - Initializing and managing Hibernate sessions.
* - Performing database operations, such as inserting, updating,
and deleting records.
* - Verifying data retrieval with filtering and sorting using DAO
methods.
*
* Key methods include:
* - {@link #testWriteAll()}: Inserts all entrants from {@link
EntrantList} into the database.
* - {@link #testCRUDOperations()}: Tests the full cycle of CRUD
operations (create, read, update, delete).
* - {@link #testGetAll()}: Retrieves all entrants from the
database.
* - {@link #testGetFilteredAndUnsortedList()}: Retrieves a
filtered and sorted list of entrants based on specified criteria.
*
* The class also includes comparison methods for verifying that
the retrieved and expected data are equal.
*
* @see EntrantDAO
* @see Entrant
* @see EntrantList
* @see HibernateUtil
* @see SessionFactory
* @see Test
* @see BeforeAll
* @see AfterAll
* @see DisplayName
*
* @author Inessa Repeshko CS-222a
*/
@Log4j2
public class HibernateAndDAOTest {
    private static SessionFactory sessionFactory;

```

```

private static Session session;

@BeforeAll
public static void testsSetUp() {
    sessionSetUp();
    fieldsSetUp();
    testWriteAll();
}

static void sessionSetUp() {
    log.info("Initialize Hibernate SessionFactory");
    sessionFactory = HibernateUtil.getSessionFactory();

    log.info("Hibernate Open session");
    session = sessionFactory.openSession();
    try (Session session = sessionFactory.openSession()) {
        assertNotNull(session, "Session should not be null");
    }
}

@AfterAll
public static void sessionShutdown() {
    log.info("Hibernate Shutdown SessionFactory");

    if (sessionFactory != null) {
        sessionFactory.close();
    }

    HibernateUtil.shutdown();
}

private static final EntrantDAO operations = new EntrantDAO();
private static List<Entrant> entrantList;
private static final int listLength =
EntrantList.getInstance().size();
private final Entrant testEntrant = new Entrant(

```

```

        16L,
        "KH22-7819",
        "Репешко",
        "Інесса",
        "Віталіївна",
        LocalDate.parse("2005-05-06"),
        false,
        185.3,
        false
    );

    static void fieldsSetUp() {
        entrantList = null;
    }

    public static void testTruncateTable() {
        assertTrue(operations.truncateTable("entrant"), "Table
truncation failed");
    }

    public static void testWriteAll() {
        entrantList = EntrantList.getInstance().getEntrants();
        assertFalse(entrantList.isEmpty());
        assertEquals(listLength, entrantList.size());

        assertEquals(entrantList.size(),
operations.insertAll(entrantList).size());
    }

    @Test
    @DisplayName("Get all Entrants from the database")
    void testGetAll() {
        entrantList = operations.getAllList(Entrant.class, false);
        assertFalse(entrantList.isEmpty(), "The list of query
results is empty");
    }

```

```

        assertEquals(listLength - 1, entrantList.size(), "The size
of list of query results is not expected");

        List<Entrant> expectedList =
EntrantList.getInstance().getEntrants();
        compareEntrantLists(expectedList);
        entrantList.forEach(System.out::println);
    }

    @Test
    @DisplayName("Test CRUD operations and get all Entrants from
the database including deleted records")
    void testCRUDOperations() {
        log.info("Insert test Entrant");
        Entrant entrantToInsert = new Entrant("KH22-9995",
"Марчук", "Регіна", "Віталіївна", "06.06.1996",
Boolean.parseBoolean("FALSE"), Double.parseDouble("200.00"));
        Entrant insertedEntrant =
operations.insert(entrantToInsert);
        assertEquals(listLength + 1, insertedEntrant == null ? 0 :
insertedEntrant.getId(), "Id невалідні");

        log.info("Update test Entrant");
        Entrant entrantToUpdate = insertedEntrant;
        entrantToUpdate.setName("Інецца");
        Entrant updateResult = operations.update(entrantToUpdate);
        assertTrue(updateResult.getId() != null);
        compareEntrantsByFields(entrantToUpdate, updateResult,
false);

        log.info("Delete test Entrant");
        boolean deleteResult = operations.delete(entrantToUpdate);
        assertTrue(deleteResult);

        log.info("Get All records including deleted");
        entrantList = operations.getAllList(Entrant.class, true);
    }

```

```

        assertFalse(entrantList.isEmpty(), "The list of query
results is empty");

        assertEquals(listLength + 1, entrantList.size(), "The size
of list of query results is not expected");

        List<Entrant> expectedList =
EntrantList.getInstance().getEntrants();
        expectedList.add(entrantToUpdate);
        compareEntrantLists(expectedList);
        entrantList.forEach(System.out::println);
    }

    private void compareEntrantLists(List<Entrant> expected) {
        assertFalse(entrantList.isEmpty(), "The list of query
results is empty");

        boolean includingId = expected.get(0).getId() != null;

        for (int i = 0; i < entrantList.size(); i++) {
            Entrant exp = expected.get(i);
            Entrant act = entrantList.get(i);

            if (includingId) {
                assertEquals(exp, act, "Entrants do not match");
            }

            compareEntrantsByFields(exp, act, includingId);
        }
    }

    private void compareEntrantObjects() {
        assertFalse(entrantList.isEmpty(), "The list of query
results is empty");

        assertEquals(testEntrant, entrantList.get(0), "Entrants do
not match");

        for (Entrant actual : entrantList) {
            compareEntrantsByFields(testEntrant, actual, true);
        }
    }

```

```

    }

}

private void compareEntrantObjects(Entrant expected) {
    assertFalse(entrantList.isEmpty(), "The list of query
results is empty");
    assertEquals(expected, entrantList.get(0), "Entrants do not
match");

    for (Entrant actual : entrantList) {
        compareEntrantsByFields(expected, actual, true);
    }
}

private void compareEntrantObjects(Entrant expected, Entrant
actual) {
    assertEquals(expected, actual, "Entrants do not match");
    compareEntrantsByFields(expected, actual, true);
}

private void compareEntrantsByFields(Entrant expected, Entrant
actual, boolean includingId) {
    Map<String, Object> fieldsExpected =
expected.getAllFields();
    Map<String, Object> fieldsActual = actual.getAllFields();

    assertEquals(fieldsExpected.size(), fieldsActual.size(),
        "The size of fieldset of Entrants do not match: " +
fieldsExpected.size() + " != " + fieldsActual.size());

    for (Map.Entry<String, Object> entry :
fieldsExpected.entrySet()) {
        if (!includingId && entry.getKey().equals("id")) {
            continue;
        }
    }
}

```

```

        String fieldName = entry.getKey();
        Object actualValue = entry.getValue();
        Object expectedValue = fieldsExpected.get(fieldName);

        assertEquals(expectedValue, actualValue,
            "The values of the field " + fieldName + " do
not match: " + expectedValue + " != " + actualValue);
    }
}

@Test
@DisplayName("Find an Entrant in the database by id")
void testFindById() {
    Entrant actual = operations.findById(Entrant.class,
testEntrant.getId());
    compareEntrantObjects(testEntrant, actual);
}

@Test
@DisplayName("Find an Entrant in the database by case number")
void testFindByCaseNumber() {
    entrantList =
operations.findByCaseNumber(testEntrant.getCaseNumber());
    compareEntrantObjects();
}

@Test
@DisplayName("Find an Entrant in the database by surname")
void testFindBySurname() {
    entrantList = operations.findBySurname("Репешко");
    operations.findByKey(Entrant.class, "surname", "Репешко");
    compareEntrantObjects();
}

@Test

```

```

    @DisplayName("Find an Entrant in the database by surname & name
& patronymic")
    void testFindBySurnameNamePatronymic() {
        entrantList =
operations.findBySurnameNamePatronymic("Репешко", "Інесса",
"Віталіївна");
        compareEntrantObjects();
    }

    @Test
    @DisplayName("Find an Entrant in the database by birthday")
    void testFindByBirthday() {
        entrantList =
operations.findByBirthday(testEntrant.getBirthday());
        compareEntrantObjects();
    }

    @Test
    @DisplayName("Find an Entrant in the database by gender")
    void testFindByGender() {
        entrantList = operations.getAllList(Entrant.class, true);
        System.out.println("entrantList" + entrantList);

        entrantList =
operations.findByGender(testEntrant.getGender());
        entrantList.forEach(System.out::println);
        compareEntrantObjects(testEntrant, entrantList.get(1));
    }

    @Test
    @DisplayName("Find an Entrant in the database by rating score")
    void testFindByRatingScore() {
        entrantList =
operations.findByRatingScore(testEntrant.getRatingScore());
        entrantList.forEach(System.out::println);
        compareEntrantObjects();
    }

```



```

    }

    @Test
    @DisplayName("Get a list of records with filtering and
    sorting")
    void testGetFilteredAndUnsortedList() {
        Map<String, Object> filters = new HashMap<>();
        filters.put("surname", "a");
        filters.put("ratingScore", Range.between(150.0, 170.0));
        filters.put("birthday", Range.between(
            LocalDate.parse("01.04.2005",
                DateTimeFormatter.ofPattern("dd.MM.yyyy")),
            LocalDate.parse("01.12.2005",
                DateTimeFormatter.ofPattern("dd.MM.yyyy"))
        ));

        Map<String, String> sortOrders = new HashMap<>();
        sortOrders.put("surname", "ASC");

        List<Entrant> resultList =
            operations.getFilteredAndSortedEntrantList(Entrant.class, true,
                filters, sortOrders, null);

        List<Long> expectedIds = new ArrayList<>();
        expectedIds.add(1L);
        expectedIds.add(21L);
        expectedIds.add(23L);
        expectedIds.add(14L);

        assertEquals(expectedIds.size(), resultList.size());
        assertEquals(expectedIds,
            resultList.stream().map(Entrant::getId).toList());
        resultList.forEach(System.out::println);
    }
}

```

ДОДАТОК В

Реалізація JSP сторінки таблиці з пошуком, сортуванням та фільтрацією даних та сервлета до неї

В.1 Код сторінки EntrantTable.jsp

```
<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" language="java" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>
<%@ taglib prefix="fn"
uri="http://java.sun.com/jsp/jstl/functions" %>
<c:set var="contextPath"
value="${pageContext.request.contextPath}"/>
<c:set var="buttonPath"
value="${contextPath}/view/image/button/">

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Entrant Table</title>
    <link rel="icon" type="image/x-icon"
href="${contextPath}/view/image/icon/favicon_square.png">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
```

```

        crossorigin="anonymous">
</head>
<body>

<jsp:include page="../component/NavigationBar.jsp"/>

<div class="mx-3 mt-3 mb-5 font-monospace">
    <div class="row">
        <h1 class="text-info fs-1 my-3 fw-semibold text-center">
            Entrant Table
        </h1>
    </div>

    <div class="row align-items-start">
        <div class="col-2 mx-2">
            <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
                <div class="card-body col">
                    <div class="card-title row">
                        <p class="text-info fs-5 fw-bold p-0
m-0">Search</p>
                        <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">Search by case number,
                            surname</p>
                    </div>
                    <form class="card-text col mt-3"
id="formSearch" method="GET">
                        <div class="row mb-3">
                            <label for="inputSearch"
class="visually-hidden">Search</label>
                            <input type="search"
class="form-control border border-2 border-info-subtle"
                                id="inputSearch" name="search"
placeholder="Type to search...">
                        </div>

```

```

        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a type="button" id="btn-search"
onclick="applySearch(this) "
                class="btn btn-outline-info
border-2" title="Apply search">
                
            </a>
            <a type="button" id="btn-search-reset"
onclick="resetSearch(this) "
                class="btn btn-outline-info border-2
button-reset" title="Reset search">
                
            </a>
        </div>
    </form>
</div>
</div>

    <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
        <div class="card-body col">
            <div class="card-title row">
                <p class="text-info fs-5 fw-bold p-0
m-0">Filters</p>
            </div>
            <div class="card-subtitle row mt-2"
id="filterBirthday">
                <p class="text-info fs-6 fw-bold p-0
m-0">Birthday</p>
                <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
                    Filtering by birthdate interval</p>
            </div>

```

```

        <form class="card-text col mt-2"
id="formFilterBirthday" method="GET">
            <div class="row mb-2">
                <label for="inputBirthdayStart"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                style="font-size:
0.8rem;">Start</label>
                <input type="date" class="form-control
border border-2 border-info-subtle"
                id="inputBirthdayStart"
name="birthdayStart">
            </div>
            <div class="row mb-3">
                <label for="inputBirthdayEnd"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                style="font-size:
0.8rem;">End</label>
                <input type="date" class="form-control
border border-2 border-info-subtle"
                id="inputBirthdayEnd"
name="birthdayEnd">
            </div>
            <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
                <a type="button"
id="btn-filter-birthday" onclick="applyFilterBirthday(this)"
                class="btn btn-outline-info
border-2" title="Apply filter">
                    
                </a>
                <a type="button"
id="btn-filter-birthday-reset" onclick="resetFilterBirthday(this)"
                class="btn btn-outline-info border-2
button-reset" title="Reset filter">

```

```

                                
                                </a>
                                </div>
                                </form>

                                <div class="card-subtitle row mt-4"
id="filterRatingScore">
                                <p class="text-info fs-6 fw-bold p-0
m-0">RatingScore</p>
                                <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
                                Filtering by rating score interval</p>
                                </div>
                                <form class="card-text col mt-2"
id="formFilterRatingScore" method="GET">
                                <div class="row mb-2">
                                <label for="inputRatingScoreMin"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                                style="font-size:
0.8rem;">Start</label>
                                <input type="number"
class="form-control border border-2 border-info-subtle"
                                id="inputRatingScoreMin"
name="ratingScoreMin" min="120.001" max="200.000"
                                step="0.001">
                                </div>
                                <div class="row mb-3">
                                <label for="inputRatingScoreMax"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                                style="font-size:
0.8rem;">End</label>
                                <input type="number"
class="form-control border border-2 border-info-subtle"
                                id="inputRatingScoreMax"
name="ratingScoreMax" min="120.001" max="200.000"

```

```

                step="0.001">
            </div>
            <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
                <a type="button"
id="btn-filter-rating-score"
onclick="applyFilterRatingScore(this)"
                class="btn btn-outline-info
border-2" title="Apply filter">
                    
                </a>
                <a type="button"
id="btn-filter-rating-score-reset"
onclick="resetFilterRatingScore(this)"
                class="btn btn-outline-info border-2
button-reset" title="Reset filter">
                    
                </a>
            </div>
        </form>
    </div>
</div>

    <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
        <div class="card-body col">
            <div class="card-title row">
                <p class="text-info fs-5 fw-bold p-0
m-0">Add row</p>
                <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">Add new row to the end
                    of the table</p>
            </div>

```

```

        <form class="card-text col mt-3"
id="formInsert" method="GET">
            <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
                <a
href="{contextPath}/entrants/create" id="btn-add-entrant"
                class="btn btn-outline-info
border-2" title="Add record" role="button">
                    
                </a>
            </div>
        </form>
    </div>
</div>
</div>

<div class="col">
    <table id="entrantTable"
        class="table table-sm table-hover font-monospace
table-bordered border border-3 border-info-subtle shadow-lg">
        <thead class="table-info text-center fw-medium
text-secondary-emphasis">
            <tr>
                <th class="text-center
align-content-center">#</th>
                <c:forEach var="fieldName"
items="{fieldNames}">
                    <th class="text-center
align-content-center" data-column="{fieldName[0]}">
                        <div class="d-flex p-0 m-0">
                            <div class="flex-grow-1
align-content-center justify-content-center p-0 m-0">
                                <span class="">
                                    <a class="sortLink
link-info link-offset-2 link-offset-3-hover link-underline

```



```

link-underline-opacity-0 link-underline-opacity-75-hover"

onclick="applySort('${fieldName[0]}')" id="${fieldName[0]}"
                                >${fieldName[1]}</a>
                                </span>
                                </div>
                                <div class="d-none flex-shrink-1
align-content-center justify-content-end ms-2 p-0 m-0"
                                style="width: 20px;"
                                <button type="button"
class="btn bg-transparent p-0 m-0" title="Sort records">
                                
                                
                                </button>
                                </div>
                                </div>
                                </th>
                                </c:forEach>
                                <th class="text-center
align-content-center">Operations</th>
                                </tr>
                                </thead>
                                <tbody class="table-light fw-light
text-primary-emphasis">
                                <c:forEach var="entrant" items="${entrantList}"
varStatus="status">
                                <tr data-id="${entrant.id}">
                                <td class="text-center
align-content-center">${status.index + 1}</td>

```

```
  |
```

```

                                <a type="button"
href="{contextPath}/entrants/{entrant.id}/delete"
                                class="p-0 m-0 border border-0
bg-transparent button-delete"
                                data-id="{entrant.id}"
title="Delete record">
                                
                                </a>
                                </div>
                                </td>
                                </tr>
                                </c:forEach>
                                </tbody>
                                </table>
                                <br>
                                </div>
                                </div>

                                <div class="d-flex justify-content-center">
                                <a href="{contextPath}/home" class="btn btn-outline-info
me-5" type="button">
                                Go Home
                                </a>
                                </div>
                                </div>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setBirthdayLimit();
        setInputValueFromAttr();
    });

    function setBirthdayLimit() {

```

```

document.getElementById('inputBirthdayStart').setAttribute('min',
getBirthdayMinValue());

document.getElementById('inputBirthdayStart').setAttribute('max',
getBirthdayMaxValue());

document.getElementById('inputBirthdayEnd').setAttribute('min',
getBirthdayMinValue());

document.getElementById('inputBirthdayEnd').setAttribute('max',
getBirthdayMaxValue());

function getBirthdayMinValue() {
    const today = new Date();
    return new Date(today.getFullYear() - 110,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
}

function getBirthdayMaxValue() {
    const today = new Date();
    return new Date(today.getFullYear() - 16,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
}

function setInputValueFromAttr() {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);
    const inputFields = document.getElementsByTagName("input");

    for (const input of inputFields) {
        if (params.has(input.name)) {
            input.value = params.get(input.name);
        }
    }
}

```

```

    }

}

function applySearch(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const searchValue = document.getElementById("inputSearch");
    setFilteringParameter(params, searchValue);

    const newURL = currentURL.pathname + '?' +
params.toString();
    linkButton.setAttribute('href', newURL);
}

function resetSearch(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const searchValue = document.getElementById("inputSearch");
    resetFilteringParameter(params, searchValue);

    const newURL = currentURL.pathname + '?' +
params.toString();
    const form = document.getElementById("formSearch");
    form.reset();
    linkButton.setAttribute('href', newURL);
}

function resetFilterBirthday(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

```

```

        const dateStart =
document.getElementById("inputBirthdayStart");
        const dateEnd =
document.getElementById("inputBirthdayEnd");
        resetFilteringParameter(params, dateStart);
        resetFilteringParameter(params, dateEnd);

        const newURL = currentURL.pathname + '?' +
params.toString();
        const form = document.getElementById("formFilterBirthday");
        form.reset();
        linkButton.setAttribute('href', newURL);
    }

    function resetFilterRatingScore(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        const scoreMin =
document.getElementById("inputRatingScoreMin");
        const scoreMax =
document.getElementById("inputRatingScoreMax");
        resetFilteringParameter(params, scoreMin);
        resetFilteringParameter(params, scoreMax);

        const newURL = currentURL.pathname + '?' +
params.toString();
        const form =
document.getElementById("formFilterRatingScore");
        form.reset();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterBirthday(linkButton) {
        const currentURL = new URL(window.location.href);

```

```

        const params = new
URLSearchParams(currentURL.searchParams);

        const dayStart =
document.getElementById("inputBirthdayStart");
        const dayEnd = document.getElementById("inputBirthdayEnd");
        setFilteringParameter(params, dayStart);
        setFilteringParameter(params, dayEnd);

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterRatingScore(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        const scoreMin =
document.getElementById("inputRatingScoreMin");
        const scoreMax =
document.getElementById("inputRatingScoreMax");
        setFilteringParameter(params, scoreMin);
        setFilteringParameter(params, scoreMax);

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function setFilteringParameter(params, inputElement) {
        const paramName = inputElement.name;
        const paramValue = inputElement.value.toString().trim();

        if (paramValue.length > 0) {

```

```

        params.set(paramName, paramValue);
    } else {
        params.delete(paramName);
    }
}

function resetFilteringParameter(params, inputElement) {
    const paramName = inputElement.name;
    params.delete(paramName);
}

function applySort(fieldName) {
    const linkElement = document.getElementById(fieldName);

    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const paramName = "sort";
    const paramFieldName = linkElement.id;
    let paramSortOrder = 'asc';
    const currentSortValue = params.get(paramName);

    if (currentSortValue) {
        const currentSortFieldName =
currentSortValue.split("-")[0];
        const currentSortOrder = currentSortFieldName ===
paramFieldName ? currentSortValue.split("-")[1] : '';
        paramSortOrder = currentSortOrder === 'asc' ? 'desc' :
'asc';
    }

    params.set(paramName, paramFieldName + '-' +
paramSortOrder);

```



```

        const newURL = currentURL.pathname + '?' +
params.toString();

        linkElement.setAttribute('href', newURL);
    }
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>

<jsp:include page="../component/Footer.jsp"/>

</body>
</html>

```

B.2 Код сторінки PageNotFound.jsp

```

<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Page Not Found Error</title>
    <link rel="icon" type="image/x-icon"
href="${pageContext.request.contextPath}/view/image/icon/favicon_s
quare.png">

```

```

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
    rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
    crossorigin="anonymous">
</head>
<body>
<div class="container-fluid p-0 m-0 vh-100 align-content-end
justify-content-center">
    <div class="row align-content-center justify-content-center
my-5">
        <div class="card border border-2 border-danger-subtle
            shadow bg-danger-subtle rounded mt-5 w-50 p-0">
            <div class="card-header bg-danger-subtle border-bottom
border-2 border-danger-subtle
                font-monospace text-center
text-secondary-emphasis fw-bolder">
                <span class="badge text-bg-danger">404</span>
                Page Not Found Error
            </div>

            <div class="card-body bg-light font-monospace">
                <div class="card-text text-secondary-emphasis
fw-light text-center">
                    <p>Oops! Looks like the page doesn't exist.</p>
                </div>
            </div>
        </div>

        <div class="d-flex justify-content-center mt-5">
            <button class="btn btn-outline-info" type="button"
onclick="window.history.back()">Go Back</button>
        </div>
    </div>

```

```

</div>

<div class="row align-content-end p-0 m-0">
    <jsp:include page="../component/Footer.jsp"/>
</div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

    crossorigin="anonymous"></script>
</body>
</html>

```

B.3 Код сервлета EntrantTableServlet.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.controller.entity.entrant;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletConfig;
import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import ntukhpi.csit.semit.riv.webappsrivlab2.model.DAO.EntrantDAO;

```

```

import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
;
import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
List;

import java.io.IOException;
import java.util.List;
import java.util.Map;

/**
 * EntrantTableServlet class handles requests for displaying a
 * table of entrant.
 * It allows for filtering, sorting, and searching entrant based on
 * parameters passed in the request.
 *
 * The servlet supports the following features:
 * - Initial loading of entrant into the database during servlet
 * initialization.
 * - Displaying all entrant or a filtered and sorted list based on
 * request parameters.
 * - Forwarding the request to the EntrantTable.jsp view to display
 * the entrant in a table format.
 *
 * This servlet interacts with {@link EntrantDAO} to retrieve
 * entrant from the database and
 * {@link EntrantFiltersAndSorts} to process filters, sorts, and
 * search criteria from the request.
 *
 * @author Inessa Repeshko CS-222a
 */
@WebServlet(name = "EntrantTableServlet", value = "/entrants")
public class EntrantTableServlet extends HttpServlet {
    private static final String VIEW_PATH =
"/view/entrant/EntrantTable.jsp";

```

```

private static final EntrantDAO operations = new EntrantDAO();
private EntrantFiltersAndSorts filtersAndSortsGetter;

@Override
public void init(ServletConfig config) throws ServletException
{
    super.init(config);

    List<Entrant> initialEntrantList = new
EntrantList().getEntrants();
    operations.insertAll(initialEntrantList);
}

@Override
protected void doGet(HttpServletRequest request,
HttpServletRequest response) throws ServletException, IOException
{
    List<Entrant> entrantList;

    filtersAndSortsGetter = new
EntrantFiltersAndSorts(request);

    Map<String, Object> filters = null;
    Map<String, String> sortOrders = null;
    Map<String, String> search = null;

    String[] currentSort = null;

    if (request.getParameterNames().hasMoreElements()) {
        try {
            filters =
filtersAndSortsGetter.getFiltersFromRequest();
            sortOrders =
filtersAndSortsGetter.getSortsFromRequest();
            search =
filtersAndSortsGetter.getSearchFromRequest();
        } catch (IllegalArgumentException e) {

```

```

        System.err.println(e.getMessage());
    }

    entrantList =
operations.getFilteredAndSortedList(filters, sortOrder, search);
    } else {
        entrantList = operations.getAllList();
    }

    request.setAttribute("entrantList", entrantList);

    Entrant entrant = new Entrant();
    List<String[]> fieldNames =
entrant.getFieldNamesAsFormattedString();
    request.setAttribute("fieldNames", fieldNames);

    response.setStatus(HttpServletResponse.SC_OK);
    response.setCharacterEncoding("UTF-8");

    ServletContext servletContext = getServletContext();
    RequestDispatcher requestDispatcher =
servletContext.getRequestDispatcher(VIEW_PATH);
    requestDispatcher.forward(request, response);
}
}

```

B.4 Код класу EntrantFiltersAndSorts.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.controller.entity.entrant;

import jakarta.servlet.http.HttpServletRequest;
import org.apache.commons.lang3.Range;

```

```

import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.Enumeration;
import java.util.HashMap;
import java.util.Map;
import java.time.format.DateTimeFormatter;

/**
 * EntrantFiltersAndSorts class is responsible for extracting and
 * processing filtering, sorting,
 * and searching parameters from an HTTP request.
 *
 * It supports the following operations:
 * - Filtering based on entrant attributes such as case number,
 * name, surname, patronymic, birthday range, gender, and rating
 * score.
 * - Sorting based on specified fields with ASC or DESC order.
 * - Searching entrant by case number or surname.
 *
 * The filters, sorts, and search criteria are extracted from the
 * request and stored in maps
 * for use in database queries.
 *
 * @author Inessa Repeshko CS-222a
 */
public class EntrantFiltersAndSorts {
    private static final DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("yyyy-MM-dd");

    private HttpServletRequest request;
    private Map<String, Object> filters = new HashMap<>();
    private Map<String, String> sortOrders = new HashMap<>();
    private Map<String, String> search = new HashMap<>();
    private Enumeration<String> parameterNames;
    private Map<String, String> attributes = new HashMap<>();

```

```

public EntrantFiltersAndSorts(HttpServletRequest request) {
    if (request == null) {
        throw new IllegalArgumentException("Request cannot be
null.");
    }

    this.request = request;
    parameterNames = this.request.getParameterNames();
}

public Map<String, Object> getFiltersFromRequest() {
    setFiltersFromRequest();
    return this.filters;
}

public Map<String, String> getSortsFromRequest() {
    setSortsFromRequest();
    return this.sortOrders;
}

public Map<String, String> getSearchFromRequest() {
    setSearchFromRequest();
    return this.search;
}

public Map<String, String> getAttributesFromRequest() {
    setAttributesFromRequest();
    return this.attributes;
}

public void setFiltersFromRequest() {
    while (parameterNames.hasMoreElements()) {
        String paramName = parameterNames.nextElement();

        switch (paramName) {
            case "caseNumber":

```



```

        case "surname":
        case "name":
        case "patronymic":
            addStringFilter(paramName);
            break;

        case "birthdayStart":
        case "birthdayEnd":
            addBirthdayFilter();
            break;

        case "gender":
            addGenderFilter();
            break;

        case "ratingScoreMin":
        case "ratingScoreMax":
            addRatingScoreFilter();
            break;

        case "sort":
        case "search":
            break;

        default:
            throw new IllegalArgumentException("Unknown
parameter passed: " + paramName + ".");
    }
}

public void setSortsFromRequest() {
    String sortParam = request.getParameter("sort");

    if (sortParam != null && !sortParam.isBlank()) {
        String[] sortParts = sortParam.split("-");
    }
}

```

```

        if (sortParts.length == 2) {
            String fieldName = sortParts[0];
            String sortDirection = sortParts[1].toUpperCase();

            if (sortDirection.equals("ASC") ||
sortDirection.equals("DESC")) {
                addSort(fieldName, sortDirection);
            } else {
                throw new IllegalArgumentException("The sort
direction must be 'ASC' or 'DESC'.");
            }
        } else {
            throw new IllegalArgumentException("Invalid sort
parameter format '" + sortParam + "'. Expected format:
'fieldName-order'.");
        }
    }
}

public void setSearchFromRequest() {
    String searchParam = request.getParameter("search");

    if (searchParam != null && !searchParam.isBlank()) {
        addSearch(searchParam);
    }
}

public void setAttributesFromRequest() {
    while (parameterNames.hasMoreElements()) {
        String paramName = parameterNames.nextElement();

        switch (paramName) {
            case "caseNumber":
            case "surname":
            case "name":

```

```

        case "patronymic":
        case "birthdayStart":
        case "birthdayEnd":
        case "gender":
        case "ratingScoreMin":
        case "ratingScoreMax":
        case "sort":
        case "search":
            attributes.put(paramName,
request.getParameter(paramName));
            break;

        default:
            throw new IllegalArgumentException("Unknown
parameter passed: " + paramName + ".");
    }
}

private void addStringFilter(String fieldName) {
    String filterValue = request.getParameter(fieldName);

    if (filterValue != null && !filterValue.isBlank()) {
        filters.put(fieldName, filterValue);
    } else {
        throw new IllegalArgumentException("Invalid string
value passed for filtering by " + fieldName + ".");
    }
}

private void addBirthdayFilter() {
    String birthdayMin = request.getParameter("birthdayStart");
    String birthdayMax = request.getParameter("birthdayEnd");

    try {

```

```

        if (birthdayMin != null && !birthdayMin.isBlank() &&
birthdayMax != null && !birthdayMax.isBlank()) {
            LocalDate minDate = LocalDate.parse(birthdayMin,
formatter);
            LocalDate maxDate = LocalDate.parse(birthdayMax,
formatter);
            filters.put("birthday", Range.between(minDate,
maxDate));
        } else if (birthdayMin != null &&
!birthdayMin.isBlank()) {
            LocalDate minDate = LocalDate.parse(birthdayMin,
formatter);
            LocalDate maxDate = LocalDate.now();
            filters.put("birthday", Range.between(minDate,
maxDate));
        } else if (birthdayMax != null &&
!birthdayMax.isBlank()) {
            LocalDate minDate =
LocalDate.now().minusYears(110);
            LocalDate maxDate = LocalDate.parse(birthdayMax,
formatter);
            filters.put("birthday", Range.between(minDate,
maxDate));
        }
    } catch (DateTimeParseException e) {
        e.printStackTrace();
        throw new IllegalArgumentException("Invalid date value
passed for filtering by birthday.");
    }
}

private void addGenderFilter() {
    String filterValue = request.getParameter("gender");

    if (filterValue != null
        && !filterValue.isBlank())

```

```

        && (filterValue.equalsIgnoreCase("TRUE")
        || filterValue.equalsIgnoreCase("FALSE"))) {
            filters.put("gender",
Boolean.parseBoolean(filterValue));
        } else {
            throw new IllegalArgumentException("Invalid string
value passed for filtering by gender.");
        }
    }

    private void addRatingScoreFilter() {
        String ratingScoreMin =
request.getParameter("ratingScoreMin");
        String ratingScoreMax =
request.getParameter("ratingScoreMax");

        try {
            if (ratingScoreMin != null && ratingScoreMax != null &&
!ratingScoreMin.isBlank() && !ratingScoreMax.isBlank()) {
                double min = Double.parseDouble(ratingScoreMin);
                double max = Double.parseDouble(ratingScoreMax);
                filters.put("ratingScore", Range.between(min,
max));
            } else if (ratingScoreMin != null &&
!ratingScoreMin.isBlank()) {
                double min = Double.parseDouble(ratingScoreMin);
                double max = Double.valueOf(200.000);
                filters.put("ratingScore", Range.between(min,
max));
            } else if (ratingScoreMax != null &&
!ratingScoreMax.isBlank()) {
                double min = Double.valueOf(120.001);
                double max = Double.parseDouble(ratingScoreMax);
                filters.put("ratingScore", Range.between(min,
max));
            }
        }
    }

```

```

    } catch (NumberFormatException e) {
        e.printStackTrace();
        throw new IllegalArgumentException("Invalid double
value passed for filtering by rating score.");
    }
}

private void addSort(String fieldName, String sortOrder) {
    switch (fieldName) {
        case "caseNumber":
        case "surname":
        case "name":
        case "patronymic":
        case "birthday":
        case "gender":
        case "ratingScore":
            sortOrders.put(fieldName, sortOrder);
            break;
        default:
            throw new IllegalArgumentException("Unknown field
name passed for sorting: " + fieldName + ".");
    }
}

private void addSearch(String value) {
    String string = value.trim();
    search.put("caseNumber", string);
    search.put("surname", string);
}
}

```

B.5 Код сервлета UnauthorizedErrorServlet.java

```

package ntukhpi.csit.semit.riv.webappsrivlab2.controller.error;

```

```

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

/**
 * UnauthorizedErrorServlet class handles requests to the
 * "/unauthorized-error" URL.
 * It forwards the request to the Unauthorized.jsp view, which
 * displays an error page
 * informing the user that they are not authorized to access a
 * specific resource.
 *
 * This servlet only handles GET requests, and it sets the response
 * status to HTTP 200 (OK)
 * before forwarding the request to the error page.
 *
 * @author Inessa Repeshko CS-222a
 */
@WebServlet(name = "UnauthorizedErrorServlet", value =
"/unauthorized-error")
public class UnauthorizedErrorServlet extends HttpServlet {
    private static final String VIEW_PATH =
"/view/error/Unauthorized.jsp";

    @Override
    public void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
        response.setStatus(HttpServletResponse.SC_OK);

```

```

        RequestDispatcher dispatcher =
request.getRequestDispatcher(VIEW_PATH);
        dispatcher.forward(request, response);
    }
}

```

B.6 Код класу PageNotFoundErrorServlet.java

```

package ntukhpi.csit.semit.riv.webappsrivlab2.controller.error;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

/**
 * PageNotFoundErrorServlet class handles requests to the
 * "/page-not-found-error" URL.
 * It forwards the request to the PageNotFound.jsp view, which
 * displays a 404 error page
 * informing the user that the requested page could not be found.
 *
 * This servlet handles GET requests, setting the response status
 * to HTTP 200 (OK)
 * before forwarding the request to the error page view.
 *
 * This is typically used to handle "404 Not Found" errors in the
 * application.
 *
 */

```



```
* @author Inessa Repeshko CS-222a
*/
@WebServlet(name = "PageNotFoundErrorServlet", value =
"/page-not-found-error")
public class PageNotFoundErrorServlet extends HttpServlet {
    private static final String VIEW_PATH =
"/view/error/PageNotFound.jsp";

    @Override
    public void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
        response.setStatus(HttpServletResponse.SC_OK);
        RequestDispatcher dispatcher =
request.getRequestDispatcher(VIEW_PATH);
        dispatcher.forward(request, response);
    }
}
```

ДОДАТОК Г

Реалізація JSP сторінки форми додавання та редагування записів і сервлетів до них

Г.1 Код сторінки EntrantForm.jsp

```
<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" language="java" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>
<c:set var="contextPath"
value="${pageContext.request.contextPath}"/>
<c:set var="buttonPath"
value="${contextPath}/view/image/button/">
<c:set var="requestTypeIsUpdate" value="${requestType ==
'UPDATE'}"/>
<c:set var="requestTypeIsDelete" value="${requestType ==
'DELETE'}"/>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>${requestTypeIsUpdate ? 'Update Entrant' :
(requestTypeIsDelete ? 'Delete Entrant' : 'Create
Entrant')}</title>
    <link rel="icon" type="image/x-icon"
href="${contextPath}/view/image/icon/favicon_square.png">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
```

```

        rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>

<jsp:include page="../component/NavigationBar.jsp"/>

<div class="container w-50 text-center justify-content-center">
    <div class="card border border-3 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0">
        <div class="card-header bg-info-subtle border-bottom
border-3 border-info-subtle
            font-monospace text-center
text-secondary-emphasis fw-bolder">
            ${requestTypeIsUpdate ? 'Update Entrant' :
(requestTypeIsDelete ? 'Delete Entrant' : 'Create Entrant')}
        </div>

        <form method="POST" id="entrantForm"
            action="${contextPath}/entrants/${requestTypeIsUpdate
or requestTypeIsDelete ? entrant.getId() :
''}${requestTypeIsUpdate ? '/update' : (requestTypeIsDelete ?
'/delete' : 'create')}"
            class="card-body bg-light font-monospace p-5 m-0">
            <div class="card-text mb-4 d-none">
                <div class="input-group border border-1
border-info-subtle rounded ">
                    <label for="inputId" class="col-3
input-group-text text-info">Id</label>
                    <input class="col-auto form-control text-dark"
type="text" id="inputId" name="id"
                        minlength="1" maxlength="100" required
${requestTypeIsDelete ? 'readonly' : ''}

```

```

        value="${requestTypeIsUpdate or
requestTypeIsDelete ? entrant.getId() : 'null'}">
    </div>
</div>
<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded ">
        <label for="inputCaseNumber" class=" col-3
input-group-text text-info">Case Number</label>
        <input class="col-auto form-control text-dark"
type="text" id="inputCaseNumber" name="caseNumber"
        minlength="5" maxlength="20" required
${requestTypeIsDelete ? 'readonly' : ''}
        value="${requestTypeIsUpdate or
requestTypeIsDelete ? entrant.getCaseNumber() : ''}">
    </div>
</div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputSurname" class=" col-3
input-group-text text-info">Surname</label>
            <input class="col-auto form-control text-dark"
type="text" id="inputSurname" name="surname"
            minlength="1" maxlength="50" required
${requestTypeIsDelete ? 'readonly' : ''}
            value="${requestTypeIsUpdate or
requestTypeIsDelete ? entrant.getSurname() : ''}">
        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">

```

```

        <label for="inputName" class=" col-3
input-group-text text-info">Name</label>

        <input class="col-auto form-control text-dark"
type="text" id="inputName" name="name"

                minlength="1" maxlength="50" required
${requestTypeIsDelete ? 'readonly' : ''}

                value="${requestTypeIsUpdate or
requestTypeIsDelete ? entrant.getName() : ''}">

    </div>
</div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputPatronymic" class=" col-3
input-group-text text-info">Patronymic</label>

            <input class="col-auto form-control text-dark"
type="text" id="inputPatronymic" name="patronymic"

                    minlength="1" maxlength="50" required
${requestTypeIsDelete ? 'readonly' : ''}

                    value="${requestTypeIsUpdate or
requestTypeIsDelete ? entrant.getPatronymic() : ''}">

        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputBirthday" class=" col-3
input-group-text text-info">Birthday</label>

            <input class="col-auto form-control text-dark"
type="date" id="inputBirthday" name="birthday"

                    value="${requestTypeIsUpdate or
requestTypeIsDelete ? entrant.getBirthdayAsDefaultDate() : ''}"

                    required ${requestTypeIsDelete ?
'readonly' : ''}>

```

```

        </div>
    </div>

    <div class="card-text d-flex align-content-center
justify-content-start mb-4">
        <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">
            <span class="d-flex flex-fill input-group-text
text-info">Gender</span>
        </div>
        <div class="col-auto btn-group" role="group"
aria-label="Gender radio toggle button group">
            ${requestTypeIsDelete ? (entrant.getGender() ?
'<input type="hidden" name="gender" value="true">' : '<input
type="hidden" name="gender" value="false">') : ''}
            <input type="radio" class="btn-check"
name="gender" autocomplete="off"
id="inputGenderMale" value="true"
            ${requestTypeIsUpdate ? (entrant.getGender() ?
'checked' : '') : 'checked'}
            ${requestTypeIsDelete ? (entrant.getGender() ?
'checked' : '') : 'checked'}
            ${requestTypeIsDelete ? 'disabled' : ''}>
            <label class="btn btn-outline-info"
for="inputGenderMale">Male</label>

            <input type="radio" class="btn-check"
name="gender" autocomplete="off"
id="inputGenderFemale" value="false"
            ${requestTypeIsUpdate ? (entrant.getGender() ?
'' : 'checked') : ''}
            ${requestTypeIsDelete ? (entrant.getGender() ?
'' : 'checked') : ''}
            ${requestTypeIsDelete ? 'disabled' : ''}>
            <label class="btn btn-outline-info"
for="inputGenderFemale">Female</label>

```

```

        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputRatingScore" class="col-3
input-group-text text-info">Rating score</label>
            <input class="col-auto form-control text-dark"
type="number" id="inputRatingScore"
                name="ratingScore" min="120.001"
max="200.000" step="0.001" required
                value="{requestTypeIsUpdate or
requestTypeIsDelete ? entrant.getRatingScore() : 120.001}"
                {requestTypeIsDelete ? 'readonly' : ''}>
            </div>
        </div>

        <div class="d-flex justify-content-evenly">
            <button type="submit" class="btn btn-outline-info
border-2 button-apply
                {requestTypeIsDelete ? 'd-none' : ''}"
title="Save record" id="btn-form-save"
                onclick="submitConfirmation('Are you sure
you want to update this record?')">
                
            </button>
            <button type="reset" class="btn btn-outline-info
border-2 button-reset
                {requestTypeIsDelete ? 'd-none' : ''}"
title="Reset fields" id="btn-form-reset"
                onclick="form.reset()">
                
            </button>
        </div>
    </div>

```

```

        <button type="button" class="btn btn-outline-info
border-2 button-delete
                ${requestTypeIsDelete ? '' : 'd-none'}"
title="Delete record" id="btn-form-delete"
                onclick="submitConfirmation('Are you sure
you want to delete that record?')">
                
        </button>
    </div>
</form>
</div>
<div class="d-flex justify-content-center mt-5">
    <a href="${contextPath}/entrants" class="btn
btn-outline-info" type="button">
        Go to Table
    </a>
</div>
</div>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setAllInputsValidity();
    });

    function setAllInputsValidity() {
        setPatternAttr();
        setBirthdayLimit();

        setStringInputValidity();
        setBirthdayValidity();
        setRatingScoreValidity();

        const form = document.getElementById('entrantForm');

```



```

document.getElementById('entrantForm').addEventListener('submit',
function (event) {
    if (!form.checkValidity()) {
        event.preventDefault();
        event.stopPropagation();
    }
});

function setPatternAttr() {
    const regexCaseNumber = new
RegExp("^(?=.{5,20}$)[А-ЩЮЯҒҒІİ]{1,10}\\d{2}[\\-]\\d{1,4}$");
    const regexNames = new
RegExp("^(?=.{1,50}$)[А-ЩЮЯҒҒІİ](?:[А-ЯҒҒІİа-яҒҒіі]*[А-ЯҒҒІİа-яҒҒіі])?(?:[
\\-][А-ЩЮЯҒҒІİ](?:[А-ЯҒҒІİа-яҒҒіі]*[А-ЯҒҒІİа-яҒҒіі])?)*$");

    const inputCaseNumber =
document.getElementById('inputCaseNumber');
    inputCaseNumber.setAttribute('pattern',
regexCaseNumber.source);

    const inputSurname =
document.getElementById('inputSurname');
    inputSurname.setAttribute('pattern',
regexNames.source);

    const inputName = document.getElementById('inputName');
    inputName.setAttribute('pattern', regexNames.source);

    const inputPatronymic =
document.getElementById('inputPatronymic');
    inputPatronymic.setAttribute('pattern',
regexNames.source);
}

```

```

function setBirthdayLimit() {

document.getElementById('inputBirthday').setAttribute('min',
getBirthdayMinValue());

document.getElementById('inputBirthday').setAttribute('max',
getBirthdayMaxValue());

function getBirthdayMinValue() {
    const today = new Date();
    return new Date(today.getFullYear() - 110,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
}

function getBirthdayMaxValue() {
    const today = new Date();
    return new Date(today.getFullYear() - 16,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
}

function setBirthdayValidity() {
    const regexpDate = new
RegExp("^(?=.{10}$)\\d{4}\\-\\d{2}\\-\\d{2}");
    const messages = [
        "Birthday cannot be empty.",
        "Value must be date from 110 to 16 years ago
today."
    ];
    const inputBirthday =
document.getElementById('inputBirthday');

    inputBirthday.addEventListener('submit', function () {
        const minDate = new Date(this.min);
        const maxDate = new Date(this.max);
        const selectedDate = new Date(this.value);

```

```

        if (!regexDate.test(selectedDate.toString())) {
            console.log("Invalid input for " + this.name +
": \"" + this.value + "\"");
            console.log(messages[0]);
            this.setCustomValidity(messages[0]);
        } else if (selectedDate < minDate || selectedDate >
maxDate) {
            console.log("Invalid input for " + this.name +
": \"" + this.value + "\"");
            console.log(messages[1]);
            this.setCustomValidity(messages[1]);
        } else {
            console.log("Valid input for " + this.name + ":
\"" + this.value + "\"");
            this.setCustomValidity("");
        }

        this.reportValidity();
    });
}

function setRatingScoreValidity() {
    const message = "Value must be decimal number in range
from 120.001 to 200.000 inclusive.";
    const inputRatingScore =
document.getElementById('inputRatingScore');

    inputRatingScore.addEventListener('input', function ()
{
        const minVal = parseFloat(this.min);
        const maxVal = parseFloat(this.max);
        const selectedVal = parseFloat(this.value);

        if (selectedVal < minVal || selectedVal > maxVal) {

```

```

        console.log("Invalid input for " + this.name +
": \"" + this.value + "\"");
        console.log(message);
        this.setCustomValidity(message);
    } else {
        console.log("Valid input for " + this.name + ":
\"\" + this.value + "\"");
        this.setCustomValidity("");
    }

    this.reportValidity();
});
}

function setStringInputValidity() {
    const messages = [
        "Value must be Ukrainian char string between 5 and
20 chars long in format 'LLLLLLLLLLNN-NNNN'.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe."
    ];

    const form = document.getElementById('entrantForm');
    const inputs = form.querySelectorAll('input[pattern]');

    inputs.forEach((input, i) => {
        input.addEventListener('input', function () {
            if (this.validity.patternMismatch) {
                console.log("Invalid input for " +
this.name + ": \"" + this.value + "\"");
                console.log(messages[i]);
                this.setCustomValidity(messages[i]);
            }
        });
    });
}

```

```

        } else {
            console.log("Valid input for " + this.name
+ ": \"" + this.value + "\"");
            this.setCustomValidity('');
        }

        this.reportValidity();
    });
});
}

}

function submitConfirmation(message) {
    const form = document.getElementById('entrantForm');

    if (confirm(message != null ? message : 'Are you sure you
want to do this action?')) {
        form.submit();
    }
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>

<jsp:include page="../component/Footer.jsp"/>

</body>
</html>

```

Г.2 Код сервлета CreateEntrantServlet.java

```
package
ntukhpi.csit.semit.riv.webappsrivlab2.controller.entity.entrant;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import ntukhpi.csit.semit.riv.webappsrivlab2.model.DAO.EntrantDAO;
import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
;

import java.io.IOException;

/**
 * CreateEntrantServlet class handles requests for creating a new
entrant record.
 * It processes both GET and POST requests for the
"/entrants/create" URL.
 *
 * The servlet supports the following operations:
 * - For GET requests, it displays the form for creating a new
entrant by forwarding to the EntrantForm.jsp view.
 * - For POST requests, it captures the form data, creates a new
entrant, and inserts it into the database.
 * After successful insertion, the user is redirected to the
entrant's update page.
 *
```

```

* This servlet interacts with the {@link EntrantDAO} class to
perform database operations.
*
* @author Inessa Repeshko CS-222a
*/
@WebServlet(name = "CreateEntrantServlet", value =
"/entrants/create")
public class CreateEntrantServlet extends HttpServlet {
    private static final String VIEW_PATH =
"/view/entrant/EntrantForm.jsp";
    private static final EntrantDAO operations = new EntrantDAO();
    private static final String REQUEST_TYPE = "CREATE";

    @Override
    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
        request.setAttribute("requestType", REQUEST_TYPE);
        ServletContext servletContext = getServletContext();
        RequestDispatcher requestDispatcher =
servletContext.getRequestDispatcher(VIEW_PATH);
        requestDispatcher.forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws IOException {
        Entrant entrantToInsert = new Entrant(
            request.getParameter("caseNumber"),
            request.getParameter("surname"),
            request.getParameter("name"),
            request.getParameter("patronymic"),
            request.getParameter("birthday"),
            request.getParameter("gender"),
            request.getParameter("ratingScore")
        );

```

```

        Entrant insertedEntrant =
operations.insert(entrantToInsert);

        response.setStatus(HttpServletResponse.SC_CREATED);
        response.sendRedirect(request.getContextPath() +
"/entrants/" + insertedEntrant.getId().toString() + "/update");
    }
}

```

Г.3 Код сервлета UpdateAndDeleteEntrantServlet.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab2.controller.entity.entrant;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import ntukhpi.csit.semit.riv.webappsrivlab2.model.DAO.EntrantDAO;
import
ntukhpi.csit.semit.riv.webappsrivlab2.model.entity.entrant.Entrant
;

import java.io.IOException;

/**
 * UpdateAndDeleteEntrantServlet class handles requests to update
or delete an entrant record.
 * It processes both GET and POST requests for URLs matching the
pattern "/entrants/*".
 *

```



```

* The servlet supports the following operations:
* - Update an entrant when the URL matches "{id}/update".
* - Delete an entrant when the URL matches "{id}/delete".
*
* For GET requests, it retrieves the entrant information based on
the provided ID and forwards
* the data to the EntrantForm.jsp view for further actions.
* For POST requests, it either updates or deletes the entrant
based on the URL pattern.
*
* If the entrant is not found or an error occurs, the servlet
redirects to a 404 page.
*
* This servlet interacts with the {@link EntrantDAO} class to
perform database operations.
*
* @author Inessa Repeshko CS-222a
*/
@WebServlet(name = "UpdateAndDeleteEntrantServlet", value =
    "/entrants/*")
public class UpdateAndDeleteEntrantServlet extends HttpServlet {
    private static final String VIEW_PATH =
        "/view/entrant/EntrantForm.jsp";
    private static final EntrantDAO operations = new EntrantDAO();

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        String req_type = "GET";
        String pathInfo = request.getPathInfo();
        ServletContext servletContext = getServletContext();

        try {
            if (pathInfo != null
                && (pathInfo.matches("/\\d+/update")

```

```

        || pathInfo.matches("/\\d+/delete")) ) {
            if (pathInfo.matches("/\\d+/update")) {
                req_type = "UPDATE";
            } else if (pathInfo.matches("/\\d+/delete")) {
                req_type = "DELETE";
            }

            String idFromURL = pathInfo.split("/") [1];
            Long idToFind = Long.parseLong(idFromURL);

            Entrant foundEntrant =
operations.findById(idToFind);

            if (foundEntrant != null) {
                request.setAttribute("requestType", req_type);
                request.setAttribute("entrant", foundEntrant);
                RequestDispatcher requestDispatcher =
servletContext.getRequestDispatcher(VIEW_PATH);
                requestDispatcher.forward(request, response);
            }
        }
    } catch (IllegalArgumentException e) {
        e.printStackTrace();
        response.setStatus(HttpServletResponse.SC_NOT_FOUND);
        response.sendRedirect(request.getContextPath() +
"/page-not-found-error");
    }
}

@Override
protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws IOException {
    String pathInfo = request.getPathInfo();
    Entrant entrantToUpdate = new Entrant(
        request.getParameter("id"),
        request.getParameter("caseNumber"),

```

```

        request.getParameter("surname"),
        request.getParameter("name"),
        request.getParameter("patronymic"),
        request.getParameter("birthday"),
        request.getParameter("gender"),
        request.getParameter("ratingScore")
    );

    if (pathInfo != null) {
        if (pathInfo.matches("/\\d+/update")) {
            Entrant updatedEntrant =
operations.update(entrantToUpdate);

            response.setStatus(HttpServletResponse.SC_CREATED);
            response.sendRedirect(request.getContextPath() +
"/entrants/" + updatedEntrant.getId().toString() + "/update");
        }

        else if (pathInfo.matches("/\\d+/delete")) {
            boolean isDeleted =
operations.delete(entrantToUpdate);

            response.setStatus(HttpServletResponse.SC_CREATED);
            response.sendRedirect(request.getContextPath() +
"/entrants");
        }
    }
}
}
}

```